

ICPC算法模板



ICPC Algorithm Template

ICPC Algorithm Template	1
1. Basic Algorithms	5
Monotonic Stack	5
Monotonic Queue	6
Tower of Hanoi	7
Counting Sort and Radix Sort	8
2. Dynamic Programming	10
Binary Lifting and Sparse Table	10
Rerooting DP	12
Longest Increasing Subsequence (LIS)	13
Contour Line DP	15
3. String	17
Aho-Corasick Automaton	17
KMP Algorithm	19
Manacher's Longest Palindromic Substring	21
Z Algorithm (Extended KMP)	22
String Hashing	24
4. Mathematics	25
0/1 Fractional Programming	25
GCD and LCM	26
Miller-Rabin Primality Test	27
Raney's Lemma	28
Chinese Remainder Theorem (CRT)	29
Extended Chinese Remainder Theorem (EXCRT)	31
Fast Exponentiation	32
Linear Diophantine Equation	33
Binomial Inversion	35
Binomial Theorem	37
Bit Manipulation	38
Catalan Numbers (Prime Factor Counting)	39
Catalan Numbers	41
Linear Basis (Vector Space)	43

Goldbach's Conjecture	44
Bitwise Implementation of Arithmetic Operations.....	44
Sieve of Eratosthenes.....	46
Perfect Shuffle Problem	46
Inclusion-Exclusion Principle	49
Common General Corollaries.....	51
Common Arithmetic and Geometric Sequences	52
Cantor Expansion	52
XOR Linear Basis (Ordinary Elimination)	53
XOR Linear Basis (Gaussian Elimination).....	55
Merging XOR Linear Bases	57
Extended CRT Template Example	57
Extended Euclidean Algorithm for Modular Inverse.....	60
Majority Vote	61
Majority Vote k-Extension	61
Matrix Fast Exponentiation.....	63
Multiplicative Functions.....	65
Josephus Problem	65
Linear Sieve (Euler's Sieve).....	65
Combinatorics	66
Bezout's Theorem and Extended Euclidean Algorithm	67
Trial Division Primality Test.....	68
Prime Factorization	68
Fermat Inverse and Brute-Force Inverse	69
Base Conversion.....	70
Linear Recurrence for Modular Inverses	71
Inverse Cantor Expansion	71
Linear Recurrence for Factorial Inverses	74
Random Number Functions	74
Gaussian Elimination (Congruence Equations)	75
Gaussian Elimination (XOR Equations).....	78
Gaussian Elimination (Linear Equations)	80
High-Precision Multiplication.....	82
Chicken McNugget Theorem	83

5. Graph Theory	84
1. Shortest Path Problems	84
01BFS.....	84
A* Algorithm	85
Bellman-Ford / SPFA Single-Source Shortest Path.....	86
Dijkstra Single-Source Shortest Path	87
Floyd-Warshall All-Pairs Shortest Path	91
Layered Graph Shortest Path	91
TSP Shortest Path Problem	94
Congruence Shortest Path (Two-Loop Method).....	95
Congruence Shortest Path (Dijkstra).....	97
Negative Cycle and Difference Constraints (Floyd).....	99
Negative Cycle and Difference Constraints (SPFA)	100
2. Tree Problems	103
DFS Detection of Negative Cycle.....	103
LCA via Binary Lifting.....	104
LCA via Tarjan.....	106
Tree Difference (Point and Edge).....	108
Tree Diameter (Two DFS).....	110
Tree Diameter (Tree DP)	112
Tree Centroid (Using Subtree Size)	114
Tree Centroid (Using Subtree Node Count)	115
3. Spanning Trees	117
Kruskal Minimum Spanning Tree	117
Prim Minimum Spanning Tree	120
Topological Sort	125
6. Computational Geometry	127
Pick's Theorem for Lattice Points.....	127
3D Geometric Centroid	128
3D Polygon Area.....	129
2D Geometric Centroid	132
2D Polygon Area (Shoelace Formula)	133
Convex Hull (Andrew's Algorithm)	134
Determining the Type of Angle	136

Angle Between Vectors.....	137
Circle-Related (2D)	138
Plane Normal Vector	139
Radian Angle Type Determination and Conversion	140
Dot Product and Cross Product.....	141
Point-Line and Point-Plane Relationship.....	145
Point-Line and Point-Plane Projection	146
Point-to-Line Distance.....	147
Angle Between Lines	148
Line-Plane Angle and Projection	150
7. Data Structures	152
1. Trie	152
Basic Trie	152
Persistent Trie	154
2. Handwritten Heap.....	154
Handwritten Reverse-Indexed Heap Optimization.....	154
Handwritten Heap.....	156
3. Leftist Heap	158
K-th Shortest Path	158
Persistent Leftist Heap	164
Leftist Heap	166
4. Disjoint Set Union (DSU)	169
Weighted DSU	169
Disjoint Set Union	170
5. Ordered Map Topics	171
AVL Balanced Tree	171
FHQ Persistent Treap	177
Splay Tree.....	181
Treap FHQ (Non-Frequency-Compressed).....	186
Treap FHQ (Frequency-Compressed).....	191
Treap Rotation Tree	195
Persistent Literary Balanced Tree	200
Leftist Heap	204
Literary Balanced Tree (FHQ Lazy Tag)	207

Scapegoat Tree (Frequency-Compressed)	209
Scapegoat Tree (Non-Frequency-Compressed)	214
Cartesian Tree	218
Skip List	220
6. Fenwick Tree	226
Point Update, Range Query (2D).....	226
Point Update, Range Query	226
Fenwick Tree	227
Range Update, Point Query	232
Range Update, Range Query (2D)	233
Range Update, Range Query	234
7. Segment Tree	235
Persistent Segment Tree (Range MEX)	235
Persistent Segment Tree (K-th Smallest)	238
Persistent Segment Tree (Lazy Update).....	241
Persistent Segment Tree (Marker Permanence)	243
Persistent Segment Tree (Floating Upper Median)	246
Segment Tree Range Min/Max	250
Segment Tree Historical Min/Max.....	253
Segment Tree and Sweep Line (Perimeter Union).....	257
Segment Tree and Sweep Line (Area Union)	260

1. Basic Algorithms

Monotonic Stack

```
// The monotonic stack has a wide range of applications:
// greedy, DP optimization, and many other scenarios

// template:
// Monotonic stack finds the first strictly smaller position on the left
// and right
stack<int>sta;
const int MAXN = 2e5;
int right[MAXN + 1];
int left[MAXN + 1];
```

```

int a[MAXN + 1];
void solve(int n) {
    // -1 marks nonexistence
    // the stack must always keep the top as the maximum
    for (int i = 1; i <= n; i++) {
        // the right side is maintained when popping
        while (sta.size() && a[i] < a[sta.top()]) {
            int t = sta.top();
            ::right[t] = i;
            sta.pop();
        }

        // the left side is maintained when pushing
        // if the same element exists, align with the first same position
        on the left
        if (sta.size()) {
            if (a[i] == a[sta.top()]) {
                ::left[i] = ::left[sta.top()];
            }
            else {
                ::left[i] = sta.top();
            }
        }
        else {
            ::left[i] = -1;
        }
        sta.push(i);
    }
    while (sta.size()) {
        int t = sta.top();
        ::right[t] = -1;
        sta.pop();
    }
}

```

Monotonic Queue

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

int a[500000];

void prove() {
    int n = 1000;
    // randomArray(n, a);
}

```

```

    deque<int>q;//monotonic queue maintaining the range maximum
    int len = 2;
    //Note: be sure to maintain the real [l, r] interval inside the loop
    //the left and right endpoints inside the queue serve the maximum
    value, not necessarily the real endpoints
    //if the problem involves information about the real [l, r], be sure
    to maintain it
    for (int l = 1, r = 1; r <= n; r++) {
        if (r - l + 1 > len) {
            l++;
        }
        //maintain the largest candidate value on the right
        while (q.size() && a[q.back()] < a[r]) {
            q.pop_back();
        }
        q.push_back(r);//enqueue the new node

        //maintain the valid index on the left
        while (q.size() && q.front() < l) {
            q.pop_front();
        }

        //output the range maximum
        //with the constraint below, the interval length is strictly =
        len, otherwise <= len
        if (r - l + 1 == len) {
            cout << l << "---" << r << ' ' << a[q.front()] << '\n';
        }
    }
}

```

Tower of Hanoi

```

#include<bits/stdc++.h>
using namespace std;
class TOH {
public:
    //Classic Tower of Hanoi problem:
    //move all disks on pile A to C, using B as the auxiliary pile
    //each disk must be strictly larger than the disk below it
    using ll = long long;
    ll f(ll n, char cur, char tool, char goal) {
        //currently moving cur to goal, tool is the auxiliary pile
        if (n == 0) {
            return 0;
        }
    }
}

```

```

    }
    //recursion
    ll ans = 0;
    ans = ans + f(n - 1, cur, goal, tool); //move [1, n - 1] to the
    tool pile, using the goal pile as the actual auxiliary pile

    ans = ans + 1; //+1 means the last n-th disk moves to the goal pile
    cout << cur << "->" << goal << '\n'; //output the move of the n-th
    disk

    ans = ans + f(n - 1, tool, cur, goal); //finally treat tool as the
    new cur pile, move the n-1 disks on it to goal

    return ans;
}
};

```

Counting Sort and Radix Sort

```

#include<bits/stdc++.h>
using namespace std;
//Counting sort, time and space complexity O(n)
//has strict requirements on the range of the data
class CountSort {
private:
    static const int MAXN = 1e7;
    int cnt[MAXN + 1] = { 0 };
    //unordered_map<int,int>cnt
public:

    void Sort(int* a, int n, int maxval) {
        fill(cnt, cnt + maxval + 1, 0);
        for (int i = 1; i <= n; i++) {
            cnt[a[i]]++;
        }
        int idx = 0;
        for (int i = 0; i <= maxval; i++) {
            for (int j = 1; j <= cnt[i]; j++) {
                a[++idx] = i;
            }
        }
    }
};

//Radix sort RadixSort
//can sort non-negative integers in different bases

```

```

//time complexity O(n)
class RadixSort {
private:
    static const int MAXN = 1e7;
    int help[MAXN + 1]; //auxiliary array
    int bit = 0;
    int GetHeightBit(int* a, int n, int base) {

        //get the highest digit
        int maxval = 0;
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            maxval = max(maxval, a[i]);
        }
        while (maxval) {
            maxval /= base;
            ans++;
        }
        return ans;
    }

    void copy(int n, int* a, int* b) {
        //a->b
        for (int i = 1; i <= n; i++) {
            b[i] = a[i];
        }
    }

public:
    void Sort(int* a, int n, int base) {
        //valid indices 1~n
        bit = GetHeightBit(a, n, base); //get the most significant digit in
BASE
        for (int i = 1, offset = 1; i <= bit; i++, offset *= base) {

            //process each digit in turn
            vector<int> cnt(base, 0); //0~base-1
            for (int i = 1; i <= n; i++) {
                cnt[(a[i] / offset) % base]++;
            }

            //prefix sum in base notation
            for (int i = 1; i < base; i++) {
                cnt[i] += cnt[i - 1];
            }

            //sort

```

```

        for (int i = n; i >= 1; i--) {
            help[cnt[(a[i] / offset) % base]--] = a[i];
        }
        copy(n, help, a);
    }
}

};

```

2. Dynamic Programming

Binary Lifting and Sparse Table

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//Core ideas for understanding binary lifting and sparse table:
//1. binary numbers can be decomposed
//2. binary math naturally guarantees composability
//3. the maintained information must be mergeable
//4. dynamic programming thinking

//Binary lifting table maintains the cyclic shortest path problem
//https://www.luogu.com.cn/problem/P4155
const int MAXN = 2e5;
struct node {
    int idx;
    int st, ed;
}line[2 * MAXN + 1];
int stjump[2 * MAXN + 1][32];
int power = 32;
int ans[MAXN + 1];

//build the binary lifting table, core code
void buildStjump(int n) {
    power = ceil(log2(n));
    int e = n << 1; //number of lines
    for (int i = 1, arrive = 1; i <= e; i++) { //arrive is the reachable line
        index
        while (arrive + 1 <= e && line[arrive + 1].st <= line[i].ed) {
            arrive++; //if reachable
        }
        stjump[i][0] = arrive;
    }
}

```

```

    }
    for (int p = 1; p <= power; p++) {
        for (int i = 1; i <= e; i++) {
            stjump[i][p] = stjump[stjump[i][p - 1]][p - 1];
        }
    }
}

int jump(int i, int m) {
    int aim = line[i].st + m;
    int cur = i, nxt, ans = 1;
    for (int p = power; p >= 0; p--) {
        nxt = stjump[cur][p];
        if (nxt != 0 && line[nxt].ed < aim) {
            ans += (1 << p);
            cur = nxt;
        }
    }
    return ans + 1; // stjump[cur][0], the last step
}

bool cmp(node a, node b) {
    return a.st < b.st;
}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> line[i].st >> line[i].ed;
        line[i].idx = i;
        if (line[i].st > line[i].ed) {
            line[i].ed += m;
        }
    }
    sort(line + 1, line + n + 1, cmp);
    for (int i = 1; i <= n; i++) {
        line[i + n].st += m;
        line[i + n].ed += m;
    }
    buildStjump(n);
    for (int i = 1; i <= n; i++) {
        ans[line[i].idx] = jump(i, m);
    }
    for (int i = 1; i <= n; i++) {
        cout << ans[i] << ' ';
    }
}

```

```

    }
}

```

Rerooting DP

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e6;

// Rerooting DP: solves the problem of quickly querying some information
// with each node as the root
// needs to maintain several pieces of information for each node
// generally requires two DFS passes
// the first DFS obtains information of all nodes bottom-up during
// recursive backtracking, and obtains the answer for the root node
// the second DFS obtains answers for all nodes through a top-down
// transition equation with strict positional dependency
// core: analyze the state transition between a parent and its child
// through several pieces of information, finally obtaining the key
// information

ll dp[MAXN + 1]; // sum of distances from all nodes to i, with i as the
// root
int siz[MAXN + 1]; // size of subtree i
ll sum[MAXN + 1]; // sum of distances from all children of i to i
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];
int idx = 0;
void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}
void dfsDp(int u, int fa, int n) {
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            // !!! Core code follows: !!!
            // node 1's child 2
            // all subtree nodes of 2 walk towards 1, then take one step
            // back (-siz[v]) to reach 2
            // all subtree nodes of 1 on the opposite side of 2 walk
            // towards 1, then take one step forward (n-siz[v]) to reach 2
            dp[v] = dp[u] - siz[v] + (n - siz[v]);
        }
    }
}

```

```

        dfsDp(v, u, n);
    }
}

void dfs(int u, int fa) {
    siz[u] = 1;
    sum[u] = 0;
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfs(v, u);
            siz[u] += siz[v];
            sum[u] = sum[v] + siz[v];
        }
    }
}

void solve() {
    // https://www.luogu.com.cn/problem/P3478
    int n;
    cin >> n;
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(1, 0);
    dp[1] = sum[1];
    dfsDp(1, 0, n);
    int best = 0;
    ll dist = -1e18;
    for (int i = 1; i <= n; i++) {
        if (dp[i] > dist) {
            dist = dp[i];
            best = i;
        }
    }
    cout << best << '\n';
}

```

Longest Increasing Subsequence (LIS)

```

#include<bits/stdc++.h>
using std::cin, std::cout;
const int MAXN = 1e6;
using ll = long long;

```

```

//LIS: longest increasing subsequence (strictly increasing)

int dp[MAXN + 1]; //dp[i] is the longest increasing subsequence ending at i
int end[MAXN + 1]; //the maximum ending element of the increasing
subsequence of length i

int binary_search(int x, int n, int* arr) {
    //find the first position >= x
    //for the longest non-strictly increasing subsequence (i.e. equality
    allowed), find the first position > x
    int l = 1, r = n;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (arr[mid] >= x) {
            //for the longest non-strictly increasing subsequence, r = mid
            - 1 when > x
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return l;
}

//get the length of the longest increasing subsequence
int LIS(int n, int* arr) {
    int R = 1;
    dp[1] = 1;
    end[1] = arr[1];
    for (int i = 2; i <= n; i++) {
        int idx = binary_search(arr[i], R, end);
        if (idx > R) {
            end[++R] = arr[i];
        }
        else {
            end[idx] = arr[i];
        }
        dp[i] = idx;
    }
    return R;
}

```

Contour Line DP

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Contour line DP
// time complexity:  $O(n * m * 2^m)$ 
// ordinary state compression:  $O(n * 2^m * 2^m)$ 
// the core of contour line DP is to compress the information of the
// previous and current rows into one bit state
// usually, for a bit state s, when processing the i-th position, s[0...i
// - 1] represents the decisions newly made in the current row, and s[i...m]
// represents the decisions of the previous row
// after processing, moving to i + 1, the i-th position becomes the
// decision state of the current row
// ordinary state compression: the bit information of the whole row is
// updated separately
// contour line DP: relies on a single cell for step-by-step recurrence
// https://www.luogu.com.cn/problem/P1879

const int mod = 1e8;

ll getMsk(ll x, int i) { return (x >> i) & 1; } //get the state of the i-th
bit
ll revMsk(ll x, int i) { return x ^ (1LL << i); } //flip the i-th bit
ll setMsk(ll x, int i, int v) { return v == 0 ? (x & ~(1LL << i)) : (x |
(1LL << i)); } //set the state of the i-th bit

int a[12][12];
int f[12][12][1 << 12]; //space compression
int dp[13][1 << 12]; //space compression
int prepare[1 << 12]; //auxiliary array

int f1(int i, int j, int s, int n, int m) {
    if (i == n) {
        return 1;
    }
    if (j == m) {
        return f1(i + 1, 0, s, n, m);
    }
    if (f[i][j][s] != -1) {
        return f[i][j][s];
    }
    int ans = f1(i, j + 1, setMsk(s, j, 0), n, m) % mod;
    // (position in matrix can be placed) and (first column or nothing
```

```

placed on the left) and (nothing placed above)
    if (a[i][j] == 1 && (j == 0 || getMsk(s, j - 1) == 0) && getMsk(s, j)
== 0) {
        ans = (ans + f1(i, j + 1, setMsk(s, j, 1), n, m)) % mod;
    }
    f[i][j][s] = ans;
    return ans;
}

int dp1(int n, int m) {
    int maxs = 1 << m;
    //prepare is initialized to the value of the n-th row, all 1
    fill(prepare, prepare + maxs, 1);

    for (int i = n - 1; i >= 0; i--) {
        //when j == m, directly reuse the value of the next row at j = 0
        (row i + 1, j = 0)
        for (int s = 0; s < maxs; s++) {
            dp[m][s] = prepare[s];
        }

        //j[m-1,0]
        for (int j = m - 1; j >= 0; j--) {
            for (int s = 0; s < maxs; s++) {
                dp[j][s] = dp[j + 1][setMsk(s, j, 0)];
                if (a[i][j] == 1 && (j == 0 || getMsk(s, j - 1) == 0) &&
getMsk(s, j) == 0) {
                    dp[j][s] = (dp[j][s] + dp[j + 1][setMsk(s, j, 1)]) %
mod;
                }
            }
        }

        //set prepare, for the previous row (row i - 1) to depend on
        for (int s = 0; s < maxs; s++) {
            prepare[s] = dp[0][s];
        }
    }

    return dp[0][0];
}

void solve() {
    int n, m;
    cin >> n >> m;
    int maxs = 1 << m;
    for (int i = 0; i < n; i++) {

```

```

        for (int j = 0; j < m; j++) {
            fill(f[i][j], f[i][j] + maxs, -1);
            cin >> a[i][j];
        }
    }
    // cout << f1(0, 0, 0, n, m) << '\n';
    cout << dp1(n, m) << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    solve();
    return 0;
}

```

3. String

Aho-Corasick Automaton

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Aho-Corasick automaton (optimized version): counts the occurrences of
target strings in the query string
// tree (direct table): Trie transition table, directly stores the node to
jump to after encountering a character, already merged with fail to avoid
back-jumping
// fail: points to the prefix node corresponding to the longest valid
suffix of the current path, similar to KMP's next
// (maintains the prefix node that has the same valid suffix as the path
ending at the current character (and the longest), similar to the kmp next
array)
// counting: traverse the query string and accumulate counts at the
reached nodes
// aggregation: build the graph in reverse along fail, accumulate counts
bottom-up to the root

// The AC automaton can implement functions other than collecting word
frequencies, customizable per problem
// e.g. alert (a bool that reports the discovery of target information,
which can be propagated along fail)
// its core is to build the tree direct table from the target strings,

```

```

then build the fail table

// call order: insert -> setFail -> buildTree / query
// complexity: O(S+T+L)
// S = query string length, T = total target string length, L = number of
tree nodes
// template problem: https://www.luogu.com.cn/problem/P5357

const int MAXN = 2e5;
const int MAXS = 2e6;

int ed[MAXN + 1]; // end position of each target character
int fail[MAXS + 1]; // fail pointer
int times[MAXS + 1]; // frequency
int tree[MAXS + 1][26]; // tree direct table
int head[MAXS + 1]; // build the graph
int nxt[MAXS + 1];
int to[MAXS + 1];
int idx = 0;
int cnt = 0;

void addEdge(int u, int v) { // build the graph
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void insert(int i, string s) { // insert string into the prefix tree tree
    int cur = 0;
    for (char ch : s) {
        int path = ch - 'a';
        if (tree[cur][path] == 0) {
            tree[cur][path] = ++cnt;
        }
        cur = tree[cur][path];
    }
    ed[i] = cur;
}

void setFail() { // set the fail pointer and the tree direct table
    queue<int> q;
    fail[0] = 0;
    for (int path = 0; path <= 25; path++) {
        if (tree[0][path] > 0) {
            fail[tree[0][path]] = 0;
            q.push(tree[0][path]);
        }
    }
    while (q.size()) {

```

```

        int cur = q.front();
        q.pop();
        for (int path = 0; path <= 25; path++) {
            if (tree[cur][path] == 0) {
                tree[cur][path] = tree[fail[cur]][path];
            }
            else {
                fail[tree[cur][path]] = tree[fail[cur]][path];
                q.push(tree[cur][path]);
            }
        }
    }
}

void buildTree(int n) { //build the fail reverse graph
    for (int i = 1; i <= n; i++) {
        addEdge(fail[i], i);
    }
}

void Get(string s) { //read the large article, get the initial single-node
word frequency
    int cur = 0;
    for (char ch : s) {
        int path = ch - 'a';
        cur = tree[cur][path];
        times[cur]++;
    }
}

void f(int cur) { //recursively count the true word frequency
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        f(to[edge]);
        times[cur] += times[to[edge]];
    }
}
}

```

KMP Algorithm

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
class KMP {
private:
    /*
    Overall time complexity  $O(n + m)$ 
    kmp :  $O(n)$ 
    nextArr :  $O(m)$ 

```

kmp returns the starting position of string t as a substring of s
if you want to return the starting position of the first matching
substring, modify as follows

```
while (i < n && j < m) {  
    .....  
    (no need for the special check if:j == m)  
    return j == m ? i - j : -1;  
}
```

In the nxt array, nxt[0] serves as a sentinel position, and nxt[m] is
used to represent the whole string t

nxt[i] represents (not including the current position i) the length of
the longest identical prefix and suffix

e.g. aaavfvdfaaad, nxt[12] = 3, "aaa" = "aaa"

generate the nxt array of the target string t

*/

```
static const int MAXN = 1e6;
```

```
int nxt[MAXN + 2];
```

```
vector<int> kmp(string s, string t) {  
    vector<int>res;  
    int x = 0, y = 0;// x is the pointer of s, y is the pointer of t  
    int n = s.size(), m = t.size();  
    while (x < n) {  
        if (s[x] == t[y]) {  
            x++, y++;// matched successfully  
        }  
        else if (y == 0) {  
            x++;// y points to the start of t  
        }  
        else {// j jumps  
            y = nxt[y];  
        }  
        if (y == m) {// satisfied  
            res.push_back(x - y);  
            y = nxt[y];  
        }  
    }  
    return res;  
}
```

```
void nextArr(string s) {  
    int n = s.size();  
    nxt[0] = -1;  
    nxt[1] = 0;  
    if (n == 1) {  
        return;  
    }
```

```

    }
    int i = 2;
    int cn = 0; // matching position on the left
    while (i <= n) {
        // when matching, the current position is not included
        if (s[i - 1] == s[cn]) {
            nxt[i++] = ++cn;
        }
        else if (cn > 0) {
            cn = nxt[cn]; // jump
        }
        else { // cn == 0
            nxt[i++] = 0;
        }
    }
}
};

```

Manacher's Longest Palindromic Substring

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//manacher finds the longest palindromic substring
//p[i]: the radius of the palindromic substring centered at i
//r: the maximum position among the rightmost cut-off positions of
palindromic substrings of all determined points
//c: the earliest center whose palindromic substring cut-off position
equals r
//case 1: current position i is not covered by r, expand centered at i
//case 2: current i is covered by r, symmetric point i2 is inside the
large palindrome, p[i]=p[i2]
//case 3: current i is covered by r, symmetric point i2 is outside the
large palindrome, p[i]=r-i
//case 4: current i is covered by r, the palindrome radius of symmetric
point i2 hits the boundary, expand from r
class Man {
private:
    static const int MAXN = 2e5;
    static int p[2 * MAXN + 2];
public:
    int manacher(string t) {
        string s = "#";
        for (char ch : t) {
            s += ch; //cannot write as s+=ch+'#'
            s += '#'; //writing this way would add the ASCII codes of ch

```

```

and #, producing one character
    }
    int maxlen = 0;
    //the actual palindrome length = palindrome radius - 1
    for (int i = 0, r = 0, c = 0, len; i < s.size(); i++) {
        len = r > i ? min(p[2 * c - i], r - i) : 1; //if covered by r,
get the answer, otherwise at least len=1
        while (i + len < s.size() && i - len >= 0 && s[i + len] == s[i
- len]) {
            //cases 2 and 3 cannot execute this while loop
            //cases 1 and 4 may execute this while loop
            len++;
        }
        if (i + len > r) {
            //determine the longest palindrome radius
            r = i + len;
            c = i;
        }
        maxlen = max(maxlen, len);
        p[i] = len;
        int st = (i - len + 1) / 2; //the actual mapped starting
position of this substring
        int ed = (i + len - 3) / 2; //the actual mapped ending position
of this substring
    }
    return maxlen - 1;
}
};

```

Z Algorithm (Extended KMP)

```

#include<bits/stdc++.h>
using namespace std;
//Z algorithm
//z[i] represents the length of the longest matching prefix between
s[i...n] and s[0...n]
//r represents the farthest known matching distance
//c represents the earliest position that reaches the farthest matching
distance
//case 1: i is not covered by r, expand directly from i
//case 2: i is covered by r, z[i-c] is inside the expansion region,
z[i]=z[i-c]
//case 3: i is covered by r, z[i-c] is outside the expansion region,
z[i]=r-i
//case 4: i is covered by r, z[i-c] is at the boundary of the expansion
region, expand from r

```

```

//e[i] represents the length of the longest matching prefix between
a[i...n] and b[0...n]
//all cases of e are analogous to the four cases of z
class Z {
private:
    static const int MAXN = 2e5;
    static int z[MAXN + 2];
    static int e[MAXN + 2];
public:
    void zArray(string s) {
        z[0] = s.size();
        for (int i = 1, c = 1, r = 1, len; i < s.size(); i++) {
            len = r > i ? min(r - i, z[i - c]) : 0; //get cases 1 and 3,
otherwise assign 0
            while (i + len < s.size() && s[i + len] == s[len]) {
                //the while loop only executes normally in cases 1 and 4
                len++;
            }
            if (i + len > r) {
                r = i + len;
                c = i;
            }
            z[i] = len;
        }
    }
    void eArray(string a, string b) {
        int n = a.size();
        int m = b.size();
        zArray(b); //generate the e array of a with respect to b, depends
on the z array of b
        for (int i = 0, c = 0, r = 0, len; i < n; i++) {
            len = r > i ? min(r - i, z[i - c]) : 0;
            while (i + len < n && len < m && a[i + len] == b[len]) {
                len++;
            }
            if (i + len > r) {
                r = i + len;
                c = i;
            }
            e[i] = len;
        }
    }
};

```

String Hashing

```
#include<bits/stdc++.h>
using namespace std;
class HashString {
    //string hashing
    //input: string
    //return result: a specific numeric value
    //string hashing requires choosing a base
    //e.g. 433,499,599,1e9+7,31,131,1313,13131,131313
    //assigning characters ch -> positive integer (do not use 0!),
    customizable
private:
    using ll = long long;
    using ull = unsigned long long;
    static const ull base = 599;
    //digits + uppercase + lowercase
    static const int MAXN = 2e6;
    static ull pow[MAXN + 1];
    static ull hash[MAXN + 1];
    ull value(const char& ch) {
        if (ch >= 'a' && ch <= 'z') {
            return ch - 'a' + 37;
        }
        else if (ch >= 'A' && ch <= 'Z') {
            return ch - 'A' + 11;
        }
        else if (ch >= '0' && ch <= '9') {
            return ch - '0' + 1;
        }
        return 0;//invalid return result
    }
public:
    ull HashAll(const string& s) {
        ull ans = value(s[0]);
        for (ull i = 1; i < s.size(); i++) {
            ans = ans * base + value(s[i]);
        }
        return ans;
    }

    void build(const string& s, int n) {
        pow[0] = 1;
        for (int i = 1; i < n; i++) {
            pow[i] = pow[i - 1] * base;
        }
        hash[0] = value(s[0]);
```

```

        for (int i = 1; i < n; i++) {
            hash[i] = hash[i - 1] * base + value(s[i]);
        }
    }
    ull HashRange(int l, int r) {
        //compute the hash value of the interval [l, r]
        //call build manually before calling HashRange
        if (l == 0) return hash[r];
        return hash[r] - hash[l - 1] * pow[r - l + 1];
    }
};

```

4. Mathematics

0/1 Fractional Programming

```

#include<bits/stdc++.h>
using namespace std;

// 0/1 fractional programming, cost-performance problem
// given arrays a, b
// choose some indices so that sum(a) / sum(b) is maximized
// formula transformation:
//  $\text{sum}(a) / \text{sum}(b) = x \rightarrow \text{sum}(a) - x * \text{sum}(b) = 0 \rightarrow \text{sum}(a[i] - b[i] * x)$ 
//  $= 0$ 
// solve with binary search on the answer

// https://www.luogu.com.cn/problem/P10505
const int MAXN = 1e3;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;
struct stc {
    ld a, b;
    ld val;
}a[MAXN + 1];
bool cmp(stc s1, stc s2) {
    return s1.val > s2.val;
}
bool check(ld x, int n, int k) {
    for (int i = 1; i <= n; i++) {
        a[i].val = a[i].a - a[i].b * x;
    }
    sort(a + 1, a + n + 1, cmp);
    ld sum = 0;
    for (int i = 1; i <= k; i++) {

```

```

        sum += a[i].val;
    }
    return sum >= eps;
}

bool solve() {
    int n, k;
    cin >> n >> k;
    cout << fixed << setprecision(16);
    if (n == 0 && k == 0) {
        return false;
    }
    ld l = 0, r = 0;
    k = n - k;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].a;
        r += a[i].a;
    }
    for (int i = 1; i <= n; i++) {
        cin >> a[i].b;
    }
    while (l < r && r - l > eps) {
        ld mid = (l + r) / 2.0;
        if (check(mid, n, k)) {
            l = mid + eps;
        }
        else {
            r = mid - eps;
        }
    }
    cout << (ll)round(l * 100.0) << '\n';
    return true;
}

```

GCD and LCM

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//greatest common divisor
//Euclidean algorithm
//gcd(a,b)=gcd(b,a%b)
ll gcd(ll a, ll b) {
    while (b) {

```

```

        ll tmp = a;
        a = b;
        b = tmp % b;
    }
    return a;
}

//least common multiple
ll lcm(ll a, ll b) {
    if (a == 0 || b == 0) {
        return 0;
    }
    return a / gcd(a, b) * b;
}

```

Miller-Rabin Primality Test

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//fast exponentiation
ll pw(ll a, ll b, ll mod) {//...
}
//Miller_Rabin algorithm for primality testing
//auxiliary verification function
bool witness(ll a, ll n) {
    ll u = n - 1;
    int t = 0;
    while ((u & 1) == 0) {
        t++;
        u >>= 1;
    }
    ll x1 = pw(a, u, n), x2;
    for (int i = 1; i <= t; i++) {
        x2 = pw(x1, 2, n);
        if (x2 == 1 && x1 != 1 && x1 != n - 1) {
            return true;
        }
        x1 = x2;
    }
    if (x1 != 1) {
        return true;
    }
    return false;
}
//MR_p test array

```

```

11 MR_p[] = { 2,3,5,7,11,13,17,19,23,29,31,37 };
bool Miller_Rabin(11 n) {
    //template problem: https://www.luogu.com.cn/problem/U148828
    //determine whether x is prime
    if (n <= 2) {
        return n == 2;
    }
    if ((n & 1) == 0) {
        return false;
    }
    for (11 p : MR_p) {
        if (p >= n)break;
        if (witness(p, n)) {
            return false;
        }
    }
    return true;
}

```

Raney's Lemma

```

#include<bits/stdc++.h>
using namespace std;
//Raney's lemma: if an integer sequence (indices 1~m) has each term not
//exceeding 1 (a[i]<=1), and the total sum is the positive integer n,
//then among its m cyclic shifts, exactly n shifts have all non-empty
//prefix sums positive
void Raney() {
    cout << "Raney:" << '\n';
    const int m = 22;
    int arr[m + 1] = { 0,1,1,1,-2,1,-1,1,1,-3,1,1,
        -2,1,1,1,1,1,1,-1,-2,1,1 };
    int n = 0;
    for (int i = 1;i <= m;i++) {
        n += arr[i];
    }
    cout << n << ' ' << m << '\n';
    int cnt = 0;
    for (int i = 1;i <= m;i++) {
        int sum = 0;
        int flg = 1;
        for (int j = i;j <= i + m - 1;j++) {
            sum += arr[(j - 1) % m + 1];
            if (sum <= 0)flg = 0;
        }
        if (flg)cnt++;
    }
}

```

```

    }
    cout << "n: " << n << '\n';
    cout << "cnt: " << cnt << '\n';
}

```

Chinese Remainder Theorem (CRT)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
// Chinese remainder theorem
// general form: requires m1, m2, ...mn to be pairwise coprime
// x% m1 = r1, x% m2 = r2, ..., x% mn = rn
// lcm = m1 * m2 * m3*...*mn(coprime)
// ai = lcm / mi
// ci = (ri * ai * inv(ai, mi)) % lcm
// each ci satisfies ci% mi = ri, ci% mj = 0(j != i)
// x = c1 + c2 + c3 + ... + cn
// smallest integer solution minx = x % lcm
// general solution X = k * lcm + minx(k >= 0)

const int MAXN = 2e5;
ll r[MAXN + 1];
ll m[MAXN + 1];
ll c[MAXN + 1];
ll a[MAXN + 1];

ll add(ll a, ll b, ll mod) {
    return b == 0 ? a : add(a ^ b, ((a & b) << 1) % mod, mod);
}

ll multiply(ll a, ll b, ll ans, ll mod) {
    return b == 0 ? ans : multiply((a << 1) % mod, (ull)b >> 1, (b & 1) ?
add(ans, a, mod) : ans, mod);
}

ll mul(ll a, ll b, ll mod) { return multiply(a, b, 0, mod); }

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

```

```

    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

ll inv(ll a, ll mod) {
    ll x, y;
    exgcd(a, mod, x, y);
    x = (x % mod + mod) % mod;
    return x;
}

// template: https://www.luogu.com.cn/problem/P1495
// note:
// use the __int128_t data type to avoid intermediate data overflowing the
// ll type
// or implement multiplication with bit operations, taking the modulus at
// each step
// both can AC, C++ recommends __int128_t, more efficient and safe

void solve() {
    int n;
    cin >> n;
    ll lcm = 1;
    for (int i = 1; i <= n; i++) {
        cin >> m[i] >> r[i];
        lcm *= m[i];
    }
    ll x = 0;

    for (int i = 1; i <= n; i++) {
        a[i] = lcm / m[i];
        // c[i] = mul(r[i], mul(a[i], inv(a[i], m[i]), lcm), lcm);
        c[i] = (ll)r[i] * (ll)a[i] % lcm * (ll)inv(a[i], m[i]) % lcm;
        x = (x + c[i]) % lcm;
    }
    x %= lcm;
    cout << x << '\n';
}

```

Extended Chinese Remainder Theorem (EXCRT)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;

// Chinese remainder theorem
// extension: m1, m2, m3, ..., mn need not be strictly pairwise coprime
// x% m1 = r1, x% m2 = r2, ..., x% mn = rn
// supplement the initial modulus m0 = 1, lcm = 1, tail = 0, giving ans1 =
x * lcm + tail
// for mi, ri, giving ans2 = y * mi + ri
// obviously ans1 = ans2, eliminating gives lcm* x + mi * y = ri - tail,
i.e. ax + by = c
// solve with exgcd, if no solution exists then there is no solution

// general solution x = x0 + (b / d) * n, substitute into ans = lcm * x +
tail, ans = lcm * (b / d) * n + (lcm * x0 + tail)
// i.e. perform the following operations:
// get lcm1 = lcm * (b / d), tail1 = (lcm * x0 + tail) % lcm1
// get ans = lcm1 * x + tail1
// keep iterating until the end, the final tail is the answer (smallest
integer solution)

const int MAXN = 2e5;
ll r[MAXN + 1];
ll m[MAXN + 1];

ll add(ll a, ll b, ll mod) {
    return b == 0 ? a : add(a ^ b, ((a & b) << 1) % mod, mod);
}

ll multiply(ll a, ll b, ll ans, ll mod) {
    return b == 0 ? ans : multiply((a << 1) % mod, (ull)b >> 1, (b & 1) ?
add(ans, a, mod) : ans, mod);
}

ll mul(ll a, ll b, ll mod) { return multiply(a, b, 0, mod); }

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
}
```

```

    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

// template: https://www.luogu.com.cn/problem/P4777
// C++ recommends using __int128_t, safer and more efficient than the bit-
operation implementation
ll excrt(int n) {
    ll tail = 0, lcm = 1, tmp, a, b, c, x0;
    for (int i = 1; i <= n; i++) {
        // here a, b, c may be customized according to the problem
        a = lcm;
        b = m[i];
        c = ((r[i] - tail) % b + b) % b;

        // the following code is the generic part
        ll x, y;
        ll d = exgcd(a, b, x, y);
        if (c % d != 0) {
            return -1; // no solution
        }

        // x0 guarantees a non-negative particular solution
        x0 = ((ll)x * (ll)(c / d) % (b / d) + ll(b / d)) % (b / d);
        tmp = lcm * (b / d); // the tmp here is the new lcm1
        tail = ((ll)tail + ll(x0) * ll(lcm) % tmp) % tmp;
        lcm = tmp;
    }
    // tail is the smallest non-negative solution returned
    return tail;
}

```

Fast Exponentiation

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define ill __int128_t
//fast exponentiation
ll pw(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {

```

```

        if (b & 1) {
            res = ill(res) * ill(a) % mod;
        }
        a = ill(a) * ill(a) % mod;
        b >>= 1;
    }
    return res;
}

```

Linear Diophantine Equation

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// given a, b not both zero
// d = gcd(a, b)

// ax + by = d
// a particular solution (xx, yy) exists
// infinitely many solutions: x = xx + (b / d) * n
// y = yy - (a / d) * n
// n is any integer

// ax + by = c(c = d * k, k is any integer)
// a particular solution (xx, yy) exists
// x1 = xx * (c / d), y1 = yy * (c / d)
// infinitely many solutions: x = x1 + (b / d) * n
// y = y1 - (a / d) * n
// n is any integer

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

ll a, b, c, d, x, y;

```

```

// if a positive integer solution exists
// when x takes the smallest >= 1, y takes the maximum (the maximum >= 1)
// when y takes the smallest >= 1, x takes the maximum (the maximum >= 1)
// if it does not exist
// when x takes the smallest >= 1, y < 0
// when y takes the smallest >= 1, x < 0

//template: https://www.luogu.com.cn/problem/P5656
void solve() {
    cin >> a >> b >> c;
    d = gcd(a, b);
    if (c % d != 0) {
        cout << -1 << '\n';
        return;
    }
    exgcd(a, b, x, y);
    // solution
    x = x * (c / d);
    y = y * (c / d);
    // increment
    ll xd = b / d;
    ll yd = a / d;
    if (x <= 0) {
        ll n = (1 - x + xd - 1) / xd;
        x += n * xd;
        y -= n * yd;
    }
    else {
        ll n = (x - 1) / xd;
        x -= n * xd;
        y += n * yd;
    }
    if (y <= 0) {
        cout << x << ' ';
        ll n = (1 - y + yd - 1) / yd;
        y += n * yd;
        cout << y << '\n';
    }
    else {
        cout << ((y - 1) / yd + 1) << ' ';
        cout << x << ' ';
        cout << y - ((y - 1) / yd) * yd << ' ';
        cout << x + ((y - 1) / yd) * xd << ' ';
        cout << y << ' ';
        cout << '\n';
    }
}
}

```

Binomial Inversion

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Binomial inversion:
// the following f and g functions are sums over the range of i
//  $0 \leq i \leq n$ 
//  $g(n) = (-1)^i * C(n, i) * f(i)$  is equivalent to  $f(n) = (-1)^i * C(n, i) * g(i)$ 
//  $g(n) = C(n, i) * f(i)$  is equivalent to  $f(n) = (-1)^{(n - i)} * C(n, i) * g(i)$ 

//  $n \leq i \leq N$ 
//  $g(n) = (-1)^i * C(i, n) * f(i)$  is equivalent to  $f(n) = (-1)^i * C(i, n) * g(i)$ 
//  $g(n) = C(i, n) * f(i)$  is equivalent to  $f(n) = (-1)^{(i - n)} * C(i, n) * g(i)$ 

// g(i): designate that a certain i elements must be chosen, and the
// intersection formed contains at least these i
//  $2^{(n - i)}$ : all sets that choose the required i elements (i required,
// the items deciding different sets are the other n - i, n - i arbitrary)
//  $2^{(2^{(n - i)})} - 1$ : all sets that choose the required elements, the
// number of schemes of choosing or not (-1 means it cannot be empty)
//  $g(i) = C(n, i) * (2^{(2^{(n - i)})} - 1)$ 

// f(i): after choosing the sets, the intersection formed by the sets is
// exactly i elements
// characteristic form:  $g(k) = f(i) * C(i, k)$  summed ( $k \leq i \leq n$ )
// meaning: from the exactly i elements of f(i), choose k as the at-least-
// k elements designated by g(k)
// matches the fourth inversion form
//  $f(k) = (-1)^{(i - k)} * C(i, k) * g(i)$  summed ( $k \leq i \leq n$ )

// https://www.luogu.com.cn/problem/P10596
// core: this problem uses the transformation of "designate at least k" ->
// "exactly k"
// such transformations are often the core of binomial inversion

const int mod = 1e9 + 7;
const int MAXN = 1e6;
ll fac[MAXN + 1]; // fac[i] is (i!) modulo mod
ll inv[MAXN + 1]; // inv[i] is the modular inverse of (i!) modulo mod
ll g[MAXN + 1];
```

```

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {
    return pw(a, mod - 2);
}

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }
}

ll C(ll n, ll m) {
    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

void prepare(int n) {
    init(n);
    // g(i) = C(n, i) * (2 ^ (2 ^ (n - i))) - 1
    ll tmp = 2;
    for (int i = n; i >= 0; i--) {
        g[i] = tmp;
        tmp = tmp * tmp % mod;
    }
    for (int i = 0; i <= n; i++) {
        g[i] = (g[i] + mod - 1) % mod * C(n, i) % mod;
    }
}

ll f(int k, int n) {
    ll ans = 0;
    // f(k) = (-1)^(i - k) * C(i, k) * g(i) summed (k <= i <= n)

```

```

    for (int i = k; i <= n; i++) {
        if ((i - k) & 1) {
            ans = (ans + C(i, k) * (mod - 1) % mod * g[i] % mod) % mod;
        }
        else {
            ans = (ans + C(i, k) * g[i] % mod) % mod;
        }
    }
    return ans;
}

void solve() {
    int n, k;
    cin >> n >> k;
    prepare(n);
    cout << f(k, n);
}

```

Binomial Theorem

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// combinatorial identity:  $C(n, m) = C(n - 1, m) + C(n - 1, m - 1)$ 
//  $C(n, m) = n! / (m! * (n - m)!)$ 
//  $0! = 1$ , 0 to the 0th power = 1

// combinatorial decomposition formula of binomial coefficients:
//  $C(j, i) * C(i, n) = C(j, n) * C(j - n, j - i)$ 

const int mod = 1e9 + 7;
const int MAXN = 2e5;
ll fac[MAXN + 1]; // fac[i] is (i!) modulo mod
ll inv[MAXN + 1]; // inv[i] is the modular inverse of (i!) modulo mod

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {

```

```

        return pw(a, mod - 2);
    }

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }
}

ll C(ll n, ll m) {
    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

```

Bit Manipulation

```

#include<bits/stdc++.h>
using namespace std;

//quick check for power of 2
void isPwTwo(int n) {
    //if x>0, x&(~x+1)==x then x is a power of 2
    //~x+1=-x
    if (n == 0) {
        cout << 'F' << '\n';
        return;
    }
    if ((n & (-n)) == n) {
        cout << 'T' << '\n';
    }
    else {
        cout << 'F' << '\n';
    }
}

int GetLowBitOne(int a){}
//find the smallest non-negative integer k such that the lowest bit 1 of
a+k is >= minV
int calCost(int a, int minV) {
    int v = GetLowBitOne(a);
    if (v >= minV) return 0;
}

```

```

    int pow2 = 1 << minV;
    int m = (a + pow2 - 1) >> minV;
    int b = m << minV;
    return b - a;
}

// use the set formed by the 1 bits of the given bit state status (i.e.
// all subsets of the bit state status)
// loop traversal, this method guarantees strict descending without
// repetition
void s(){
    for (int i = status; i > 0; i = (i - 1) & status) {
        cout << i << ' ';
    }
}

// the bit-level essence of addition
// for any integers a, b
// always  $a + b = a \oplus b + 2(a \& b)$ 

// quickly extract the rightmost 1 of x (x!=0)
// ll lowBitOne=x&(-x)

```

Catalan Numbers (Prime Factor Counting)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Catalan numbers:
//  $f(n) = C(2n, n) / (n + 1)$ 
// formula 2,  $n \geq 0$ 

// when the data size reaches  $1e6$ , the modulus mod may not be prime
// need to use formula 2 + prime factor counting to solve
// maintain the smallest prime factor table, primes need not be maintained
// (default marked 0, otherwise it will error!)

// time complexity:  $O(n)$ 
// https://www.luogu.com.cn/problem/P3200
// core code follows:

ll f2(int n, int mod) {
    // 2 ~ n : -1
    fill(cnt + 2, cnt + n + 1, -1);
    // n + 2 ~ 2n : 1

```

```

    fill(cnt + n + 2, cnt + 2 * n + 1, 1);
    for (int i = 2 * n; i >= 2; i--) {
        if (minFactor[i] != 0) {
            cnt[minFactor[i]] += cnt[i];
            cnt[i / minFactor[i]] += cnt[i];
            cnt[i] = 0;
        }
    }
    ll ans = 1;
    for (int i = 2; i <= 2 * n; i++) {
        if (cnt[i] != 0) {
            ans = ans * pw(i, cnt[i], mod) % mod;
        }
    }
    return ans;
}

const int MAXN = 1e6;
// !!! allocate double the space !!!
ll minFactor[2 * MAXN + 1]; // minFactor[i] is the smallest prime factor of
i (linear Euler sieve maintains the smallest prime factor table)
bitset<2 * MAXN + 1>isPrime; // mark primes
ll Primes[2 * MAXN + 1];
int cnt[2 * MAXN + 1];
void getPrimes_Linear(ll n) {
    isPrime.set();
    isPrime[0] = isPrime[1] = 0;
    Primes[0] = 0;
    for (ll i = 2; i <= n; i++) {
        // note: for Catalan numbers and prime factor counting, primes
        need not be obtained
        if (isPrime[i]) Primes[++Primes[0]] = i;
        for (ll j = 1, v; j <= Primes[0]; j++) {
            v = Primes[j];
            if (v * i > n) break;
            isPrime[i * v] = 0;
            minFactor[i * v] = v;
            if (i % v == 0) break;
        }
    }
}

ll pw(ll a, ll b, int mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
    }
}

```

```

        b >>= 1;
    }
    return res;
}

```

Catalan Numbers

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Catalan numbers:
// f(n) = C(2n, n) - C(2n, n - 1)
// f(n) = C(2n, n) / (n + 1)
// f(n) = f(n - 1) * ((4n - 2) / (n + 1))
// f(n) = [sum 0 <= i <= n - 1]: f(i) * f(n - 1 - i)
// among them, formulas 1, 3, 4 require n >= 1
// formula 2, n >= 0

// core applications of Catalan numbers:
// 1. push/pop stack, bracket matching problems
// 2. non-crossing pairing problems
// 3. binary tree shape counting problems
// 4. matrix path problems
// 5. convex polygon triangulation
// 6. prefix non-crossing problems (i.e. not violating a specific
condition)
// the core characteristics of the above problems: decomposability,
recursiveness, non-crossing property

// core code follows:
ll f1(int n) {
    f[n] = (C(2 * n, n) - C(2 * n, n - 1) + mod) % mod;
    return f[n];
}

ll f2(int n) {
    f[n] = C(2 * n, n) * invVal[n + 1] % mod;
    return f[n];
}

ll f3(int n) {
    f[0] = f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i - 1] * (4 * i - 2) % mod * invVal[i + 1] % mod;
    }
    return f[n];
}

```

```

}

ll f4(int n) {
    f[0] = f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = 0;
        for (int j = 0; j <= i - 1; j++) {
            f[i] = (f[i] + f[j] * f[i - 1 - j] % mod) % mod;
        }
    }
    return f[n];
}

const int mod = 1e9 + 7;
const int MAXN = 2e5;
ll invVal[MAXN + 10]; // modular inverses of consecutive constants
ll fac[MAXN + 10]; // fac[i] is (i!) modulo mod
ll inv[MAXN + 10]; // inv[i] is the modular inverse of (i!) modulo mod
ll f[MAXN + 10];

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {
    return pw(a, mod - 2);
}

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }
}

```

```

    invVal[1] = 1;
    for (ll i = 2; i <= n; i++) {
        invVal[i] = mod - invVal[mod % i] * (mod / i) % mod;
    }
}

ll C(ll n, ll m) {
    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

```

Linear Basis (Vector Space)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-7;
array<int, 2> cost[501];
ld a[501][501];
int basis[501];

bool insert(int i, int m) {
    for (int j = 1; j <= m; j++) {
        if (fabs1(a[i][j]) >= eps) {
            if (basis[j] == 0) {
                basis[j] = i;
                return true;
            }
            ld rate = a[i][j] / a[basis[j]][j];
            for (int k = j; k <= m; k++) {
                a[i][k] -= rate * a[basis[j]][k];
            }
        }
    }
    return false;
}

bool cmp(array<int, 2>a, array<int, 2>b) {
    return a[0] < b[0];
}

void solve() {
    cout << fixed << setprecision(12);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {

```

```

        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }
    for (int i = 1; i <= n; i++) {
        cin >> cost[i][0];
        cost[i][1] = i;
    }
    sort(cost + 1, cost + n + 1, cmp);
    int cnt = 0;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (insert(cost[i][1], m)) {
            cnt++;
            ans += cost[i][0];
        }
    }
    cout << cnt << ' ' << ans << '\n';
}

```

Goldbach's Conjecture

```

// Strong Goldbach's conjecture: any even number greater than 2 can be
expressed as the sum of two primes; this conjecture has been proven to
hold for integers in [1,1e9]

// Weak Goldbach's conjecture (this conjecture has been completely
proven): any odd number greater than 5 can be expressed as the sum of
three primes
// among them, if n&1 and n-2 is prime then it can be split into at least
2

```

Bitwise Implementation of Arithmetic Operations

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
// implement addition, subtraction, multiplication and division without
arithmetic operations
// pure bit operations

// the overflow problem needs to be solved by taking the modulus yourself

// addition: XOR operation + AND operation
// XOR operation: addition without carry
// AND operation: maintains the carry information
ll add(ll a, ll b) { //    addition without carry    carry information

```

```

    return b == 0 ? a : add(a ^ b, (a & b) << 1);
}

// take the opposite number
// -b = ~b + 1
ll neg(ll b) {
    return add(~b, 1LL);
}

// subtraction
// a - b = a + (-b) = a + (~b + 1)
ll minus(ll a, ll b) {
    return add(a, neg(b));
}

// multiplication
// b must be converted to an unsigned right shift
ll multiply(ll a, ll b, ll ans) {
    return b == 0 ? ans : multiply(a << 1, (ull)b >> 1, (b & 1) ? add(ans,
a) : ans);
}
ll mul(ll a, ll b) { return multiply(a, b, 0); }

// int:0~30
// long long:0~62
// 31 and 63 are the sign bits
// note: a, b cannot be LLONG_MIN (the minimum value of the data type)
ll divide(ll a, ll b) {
    ll x = a < 0 ? neg(a) : a;
    ll y = b < 0 ? neg(b) : b;
    ll ans = 0; // here :: is the global namespace, avoiding
conflict with std
    for (int i = 62; i >= 0; i = ::minus(i, 1)) {
        if ((x >> i) >= y) {
            ans |= (1LL << i);
            x = ::minus(x, y << i);
        }
    }
    return (a < 0) ^ (b < 0) ? neg(ans) : ans;
}

// division when the minimum value of the data type appears
ll div2(ll a, ll b) {
    if (a == LLONG_MIN && b == LLONG_MIN) {
        return 1;
    }
    if (a != LLONG_MIN && b != LLONG_MIN) {

```

```

        return divide(a, b);
    }
    if (b == LLONG_MIN) {
        return 0;
    }
    // a is minimum, b is not minimum
    a = add(a, b > 0 ? b : neg(b));
    ll ans = divide(a, b);
    ll offset = b > 0 ? neg(1) : 1;
    return add(ans, offset);
}

```

Sieve of Eratosthenes

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e7;
bitset<MAXN + 1>isPrime;//mark primes
ll Primes[MAXN + 1];
ll minFactor[MAXN + 1];
void getPrimes_Eratosthenes(ll n) {
    //Sieve of Eratosthenes
    //complexity O(n*log2(log2n))
    //get all primes from 1 to n
    Primes[0] = 0;
    isPrime.set();
    isPrime[0] = isPrime[1] = 0;
    for (ll i = 2; i <= n; i++) {
        if (isPrime[i]) {
            minFactor[i] = i;
            Primes[++Primes[0]] = i;
            for (ll j = i * i; j <= n; j += i) {
                isPrime[j] = 0;
                if (minFactor[j] == 0) {
                    minFactor[j] = i;
                }
            }
        }
    }
}

```

Perfect Shuffle Problem

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

// Perfect shuffle problem:
// given interval [l...r], even length
// mid = (l + r) >> 1, denote [l, mid] as the array { l1,l2,l3,... }
// denote [mid+1, r] as the array { r1,r2,r3,... }
// swap [l, r] into the form { r1,l1,r2,l2,r3,l3,.. }

// core:
// if i <= mid, go(i) = i + (i - l + r)
// if i > mid, go(i) = i - (r - i + 1)
// there exist cyclic swaps of several indices
// when the length k = 3 ^ k - 1
// all cycle starting points are { 3 ^ 0, 3 ^ 1, 3 ^ 2, ..., 3 ^ (k - 1) }
// solve the problem by ternary decomposition

// using the auxiliary array help, it can be solved in O(n) time and O(n)
space
// using ternary decomposition, it can be solved in O(n) time and O(1)
space

const int MAXN = 2e5;
int a[MAXN + 1];
int b[MAXN + 1];
int start[21];
ll split[21]; // ternary decomposition array
// 3 to the 20th power is 3486784401, which is large enough

int prepare(int n) {
    int siz = 0;
    for (ll s = 1, p = 2; p <= n; s *= 3, p = s * 3 - 1) {
        start[++siz] = s; // the starting point is a power of 3
        split[siz] = p; // the split point is power of 3 minus 1
    }
    return siz;
}

int go(int l, int r, int i) {
    int mid = (l + r) >> 1;
    if (i <= mid) return i + (i - l + 1);
    return i - (r - i + 1);
}

// reverse the interval [l, r]
void rev(int l, int r, int* arr) {
    while (l < r) {
        swap(arr[l], arr[r]);
        l++, r--;
    }
}

```

```

}

//swap [l, k], [k + 1, r]
void rotate(int l, int k, int r, int* arr) {
    rev(l, k, arr);
    rev(k + 1, r, arr);
    rev(l, r, arr);
}

void circle(int l, int r, int i, int* arr) {
    for (int j = i; j >= 1; j--) {
        int cur = l + start[j] - 1, next;
        next = go(l, r, cur);
        int curv = arr[cur];
        while (next != l + start[j] - 1) {
            int tmp = arr[next];
            arr[next] = curv;
            curv = tmp;
            cur = next, next = go(l, r, next);
        }
        arr[next] = curv;
    }
}

void shuffle(int l, int r, int* arr) {
    int n = r - l + 1;
    int siz = prepare(n);
    for (int i = siz, mid; n > 0;) {
        if (split[i] <= n) {
            mid = (l + r) >> 1;
            // process the segment of length split[i]
            // respectively formed by concatenating parts from the left
and right
            rotate(l + split[i] / 2, mid, mid + split[i] / 2, arr);
            circle(l, l + split[i] - 1, i, arr);
            l += split[i];
            n -= split[i];
        }
        else {
            i--;
        }
    }
}

void shuffleHelp(int l, int r, int* arr) {
    int n = r - l + 1;
    vector<int>help(n + 1, 0);

```

```

int mid = (l + r) >> 1;
for (int i = 1, j = mid + 1; j <= r; i += 2, j++) {
    help[i] = arr[j];
}
for (int i = 2, j = 1; j <= mid; i += 2, j++) {
    help[i] = arr[j];
}
for (int i = 1, j = 1; j <= r; j++, i++) {
    arr[j] = help[i];
}
}

```

Inclusion-Exclusion Principle

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//Inclusion-exclusion principle

//core:
//+C(k,1) sum - C(k,2) intersection + C(k,3) sum - C(k,4) intersection
+ .....
//odd count +, even count -
//example: https://codeforces.com/problemset/problem/803/F
//example: https://www.luogu.com.cn/problem/P1450

class Inclusion_Exclusion { //Inclusion-exclusion principle
private:
    const static int mod = 1e9 + 7;
    int cnt[100001];
    ll dp[100001];
    //dp[i] represents the number of subsequences with i as the smallest
common divisor
    ll pow2[100001];
    void initial() {
        pow2[0] = 1;
        for (ll i = 1; i <= 100000; i++) {
            cnt[i] = 0;
            pow2[i] = pow2[i - 1] * 2LL % mod;
        }
    }

    ll dp02[100001];

public:

```

```

void solve01() {
    initial();
    int n;
    cin >> n;
    ll max_val = 1;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        cnt[x]++;
        max_val = max(max_val, (ll)x);
    }
    //in the array, a subsequence with gcd=i that contains i itself
    must have all other elements as multiples of i
    for (int i = max_val; i >= 1; i--) {
        ll count = 0;
        for (int j = i; j <= max_val; j += i) {
            count += cnt[j];
        }
        dp[i] = (pow2[count] - 1 + mod) % mod;
        for (int j = i + i; j <= max_val; j += i) {
            dp[i] = (dp[i] - dp[j] + mod) % mod;
        }
    }
    cout << dp[1] << '\n';
}

void solve02() {
    //assuming no count limit, dp[s] complete knapsack represents the
    number of schemes for 1~i to make s yuan
    //dp[s-(di+1)*ci] represents the number of schemes where the i-th
    coin exceeds di coins
    //use the inclusion-exclusion principle to find the number of
    schemes where all of 1~4 exceed
    ll value[4];
    ll cnt[4];
    int n;
    cin >> value[0] >> value[1] >> value[2] >> value[3] >> n;
    dp[0] = 1;
    //complete knapsack problem
    for (int i = 0; i <= 3; i++) {
        for (int j = value[i]; j <= 100000; j++) {
            dp[j] += dp[j - value[i]];
        }
    }
    for (int i = 0; i < n; i++) {
        ll s;
        for (int i = 0; i < 4; i++) {

```

```

        cin >> cnt[i];
    }
    cin >> s;
    //0001->1111(1->15)
    //use bit states to represent the number of schemes of the
violation state of the 1~i-th coins
    ll illegal = 0; //illegal
    for (int i = 1; i <= ((1 << 4) - 1); i++) {
        ll tmp = s; //total value
        int flg = 0; //mark add/subtract
        for (int j = 0; j <= 3; j++) {
            //check the state of each bit
            if ((i >> j) & 1) {
                tmp -= value[j] * (cnt[j] + 1);
                flg ^= 1;
            }
        }
        if (tmp >= 0) {
            illegal += flg ? dp[tmp] : -dp[tmp];
        }
    }
    cout << dp[s] - illegal << '\n';
}
};

```

Common General Corollaries

```

//1、
//positive integers a, b
//if  $a \% c = b$  exists
//then  $a = b$ , or  $a > 2 * b$ 
//all remainders obtainable by positive integer a  $[0 \dots (a-1)/2]$  && [a]
//round down

//2、
//greatest common divisor related
//any positive integer  $a < b$ 
//always  $\gcd(a, b) = \gcd(a, b - a)$ 

//3、
//for any positive integers  $a < b$ 
//if  $a \% m = b \% m$ , then  $(b - a) \% m = 0$ 

//4、

```

```
// if (a+b)%mod=0
// given a, then b%mod=(mod-a%mod)%mod

//5、
// (a+b)%mod==j
// given a, b%mod=((j-a)%mod+mod)%mod

//6、
// (a%m*b%m)%m=(a*b)%mod
// -a%mod=(-a%mod+mod)%mod=(mod-a)%mod
```

Common Arithmetic and Geometric Sequences

```
Geometric sequence:
i>=n
a[i]=i*i
b[i]=i*i-1
sum of first n terms:
Sa[i]=n(n+1)(2n+1)/6
Sb[i]=n(n-1)(2n+5)/6
```

Cantor Expansion

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Cantor expansion:
// given numbers 1~n, given a permutation P of 1~n, find the rank of
permutation P (ascending lexicographic order)
// rank(P) = [sum over 1 <= i <= n]: rightSmall(P[i]) * (n - i)!

// https://www.luogu.com.cn/problem/P5367

const int MAXN = 1e6;
const int mod = 998244353;
int tree[MAXN + 1];
int fac[MAXN + 1];
int p[MAXN + 1];

int lowbit(int x) {
    return x & -x;
}

void add(int i, int n, int v) {
    while (i <= n) {
        tree[i] += v;
        i += lowbit(i);
    }
}
```

```

    }
}

int sum(int r) {
    int ans = 0;
    while (r > 0) {
        ans = 11((11)ans + (11)tree[r]) % mod;
        r -= lowbit(r);
    }
    return ans;
}

int rank(int n) {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = 11((11)ans + (11)sum(p[i] - 1) * (11)fac[n - i] % mod) %
mod;
        add(p[i], n, -1);
    }
    return ans;
}

void prepare(int n) {
    fac[0] = 1;
    for (int i = 1; i <= n; i++) {
        fac[i] = 11((11)fac[i - 1] * (11)i) % mod;
    }
}

void solve() {
    int n;
    cin >> n;
    prepare(n);
    for (int i = 1; i <= n; i++) {
        add(i, n, 1);
        cin >> p[i];
    }
    cout << (::rank(n) + 1) % mod;
}

```

XOR Linear Basis (Ordinary Elimination)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 60;

```

```

ll basis[bit + 1];
ll a[MAXN + 1];
bool zero = false;

// Basis: a minimal linearly independent set that can represent all
// elements of the original space (not unique)
// ordinary elimination or Gaussian elimination

// solve the XOR linear basis by ordinary elimination
// time complexity  $O(n \log V)$ ,  $V$  is the maximum value
// ordinary elimination

// if the size of the linear basis =  $m$ , then the original XOR sums have  $2^{m-1}$  kinds
// maximum XOR sum: greedily from the high bit, if the XOR sum can become
// larger then XOR, otherwise do not XOR
// the  $k$ -th smallest XOR sum needs Gaussian elimination

// if insertion fails, it means it was eliminated by other basis vectors,
// and the original XOR sums contain 0
bool insert(ll x) {
    for (int i = bit; i >= 0; i--) {
        if ((x >> i) & 1) {
            if (basis[i] == 0) {
                basis[i] = x;
                return true;
            }
            x ^= basis[i];
        }
    }
    return false;
}

void buildBasis(ll* arr, int n) {
    zero = false;
    fill(basis, basis + bit + 1, 0);
    for (int i = 1; i <= n; i++) {
        if (!insert(arr[i])) {
            zero = true;
        }
    }
}

// maximum XOR sum
ll GetXorMax() {
    ll res = 0;
    for (int i = bit; i >= 0; i--) {

```

```

        res = max(res, res ^ basis[i]);
    }
    return res;
}

```

XOR Linear Basis (Gaussian Elimination)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 60;

// !! note !!
// since Gaussian elimination needs to copy the original array, a large
memory should be allocated
ll basis[MAXN + 1];
ll a[MAXN + 1];
bool zero = false;
int len;// the size of the linear basis

// Basis: a minimal linearly independent set that can represent all
elements of the original space (not unique)
// ordinary elimination or Gaussian elimination

// solving the linear basis by Gaussian elimination is a reduced version
of solving equations by Gaussian elimination
// 1. no need to reserve positions for free variables 2. no need for the
dependency relationship between pivots and free variables
// time complexity  $O(n \log V)$ ,  $V$  is the maximum value

// if the size of the linear basis =  $m$ , then the original XOR sums have  $2^m - 1$  kinds
// maximum XOR sum: greedily from the high bit, if the XOR sum can become
larger then XOR, otherwise do not XOR
// the  $k$ -th smallest XOR sum: decompose the binary bits of  $k$ , compute the
XOR sum

void gauss(int n) {
    len = 1;
    for (int i = bit; i >= 0; i--) {
        for (int j = len; j <= n; j++) {
            if (basis[j] & (1LL << i)) {
                swap(basis[j], basis[len]);
                break;
            }
        }
    }
}

```

```

    }
    if (basis[len] & (1LL << i)) {
        for (int j = 1; j <= n; j++) {
            if (j != len && (basis[j] & (1LL << i))) {
                basis[j] ^= basis[len];
            }
        }
        len++;
    }
}
len--;
zero = len != n;
}

void buildBasis(ll* arr, int n) {
    zero = false;
    for (int i = 1; i <= n; i++) {
        basis[i] = arr[i];
    }
    gauss(n);
}

// the k-th smallest non-empty XOR sum
ll GetXorKth(ll k) {
    if (k == 1 && zero) {
        return 0;
    }
    if (zero) {
        k = k - 1;
    }
    if (k >= (1LL << len)) {
        return -1;
    }
    ll ans = 0;
    for (int i = len, j = 0; i >= 1; i--, j++) {
        if ((k & (1LL << j))) {
            ans ^= basis[i];
        }
    }
    return ans;
}

// maximum XOR sum
ll GetXorMax() {
    ll res = 0;
    for (int i = 1; i <= len; i++) {
        res = max(res, res ^ basis[i]);
    }
}

```

```

    }
    return res;
}

```

Merging XOR Linear Bases

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 30;

// insert x into b
void insert(ll x, ll* b) {
    for (int i = bit; i >= 0; i--) {
        if ((x >> i) & 1) {
            if (b[i] == 0) {
                b[i] = x;
                return;
            }
            x ^= b[i];
        }
    }
}

// merge the basis of b into t
void merge(ll* b, ll* t) {
    for (int i = bit; i >= 0; i--) {
        if (b[i] != 0) {
            insert(b[i], t);
        }
    }
}

```

Extended CRT Template Example

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;
const int MAXN = 1e5;

// https://www.luogu.com.cn/problem/P4774
// the core of this problem lies in customizing the solution of a, b, c
// during the Chinese remainder theorem process

ll hp[MAXN + 1]; // health
ll recovery[MAXN + 1]; // recovery

```

```

ll attack[MAXN + 1]; // damage of the sword used to attack the i-th dragon
ll reward[MAXN + 1]; // reward
ll maxCnt = 0;

ll exgcd(ll hp, ll attack, ll& x, ll& y) {
    if (attack == 0) {
        x = 1;
        y = 0;
        return hp;
    }
    else {
        ll g = exgcd(attack, hp % attack, x, y);
        ll tmp = y;
        y = x - y * (hp / attack);
        x = tmp;
        return g;
    }
}

ll excrt(int n) {
    // attack[i] * ans ≡ hp[i] (% recovery[i])
    // ans = lcm * x + tail
    // attack[i] * ans = lcm * x * attack[i] + tail * attack[i]
    // attack[i] * ans = recovery[i] * y + hp[i]
    // lcm * attack[i] * x + recovery[i] * y = hp[i] - tail * attack[i]
    ll tail = 0, lcm = 1, tmp, a, b, c, x0;
    for (int i = 1; i <= n; i++) {
        a = ill(lcm * attack[i]) % recovery[i];
        b = recovery[i];
        c = ((hp[i] - tail * attack[i]) % b + b) % b;
        ll x, y;
        ll d = exgcd(a, b, x, y);
        if (c % d != 0) {
            return -1; // no solution
        }
        x0 = ((ill)x * (ill)(c / d) % ill(b / d) + ill(b / d)) % ill(b / d);
        tmp = ill(lcm) * ill(b / d); // the tmp here is the new lcm1
        tail = ill((ill)tail + ill(x0) * ill(lcm) % ill(tmp)) % ill(tmp);
        lcm = tmp;
    }
    // ans = ? * lcm + tail
    if (tail >= maxCnt) {
        return tail;
    }
    return (maxCnt - tail + lcm - 1) / lcm * lcm + tail;
}

```

```

void solve() {
    int n, m;
    cin >> n >> m;
    maxCnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> hp[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> recovery[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> reward[i];
    }
    multiset<ll>st;
    for (int i = 1; i <= m; i++) {
        ll x;
        cin >> x;
        st.insert(x);
    }
    for (int i = 1; i <= n; i++) {
        if (!st.size()) {
            cout << -1 << '\n';
            return;
        }
        auto it = st.lower_bound(hp[i]);
        if (it == st.end()) { // all are smaller than a[i]
            attack[i] = *prev(st.end());
            st.erase(prev(st.end()));
        }
        else {
            if (it == st.begin()) {
                // all >= hp[i]
                attack[i] = *it;
                st.erase(it);
            }
            else {
                if (*it > hp[i]) {
                    it--;
                }
                attack[i] = *it;
                st.erase(it);
            }
        }
        st.insert(reward[i]);
        maxCnt = max(maxCnt, ll(hp[i] + attack[i] - 1) / attack[i]);
    }
}

```

```

        hp[i] %= recovery[i];
    }
    cout << excrt(n) << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}

```

Extended Euclidean Algorithm for Modular Inverse

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

// solve the modular inverse of a modulo mod
// requires a, mod to be coprime
// time complexity  $O((\log \min(a, \text{mod}))^3)$ 
ll inv(ll a, ll mod) {
    ll x, y;
    exgcd(a, mod, x, y);
    x = (x % mod + mod) % mod;
    return x;
}

```

Majority Vote

```
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 2e5;
int a[MAXN + 1];

// Majority vote problem: find the element whose frequency is strictly
greater than n/2 (n/2 rounded down)

bool check(int n, int x) {
    for (int i = 1, j = 0; i <= n; i++) {
        j += (a[i] == x);
        if (j > n / 2) {
            return true;
        }
    }
    return false;
}

int waterNumber(int n) {
    int hp = 0;
    int cand = -1;
    for (int i = 1; i <= n; i++) {
        if (cand == -1) {
            cand = a[i];
            hp = 1;
        }
        else {
            if (a[i] == cand) {
                hp++;
            }
            else {
                if (--hp == 0) {
                    cand = -1;
                }
            }
        }
    }
    if (hp == 0) return -1;
    return check(n, cand) ? cand : -1;
}
```

Majority Vote k-Extension

```
#include<bits/stdc++.h>
using namespace std;
const int MAXN = 2e5;
const int MAXK = 10;
```

```

//Majority vote problem extension, find the element whose occurrences >
n/k in the array
//maintain a candidate table of size k-1
//delete k elements at a time
//k cannot be too large, otherwise the time complexity becomes the
bottleneck, or the problem itself becomes meaningless
int a[MAXN + 1];
int cands[MAXK + 1][2]; //candidate table
void update(int k, int val) {
    //already in the candidate table
    for (int i = 0; i < k; i++) {
        if (cands[i][0] == val && cands[i][1] > 0) {
            cands[i][1]++;
            return;
        }
    }
    //there is an empty slot
    for (int i = 0; i < k; i++) {
        if (cands[i][1] == 0) {
            cands[i][0] = val;
            cands[i][1] = 1;
            return;
        }
    }
    //attack
    for (int i = 0; i < k; i++) {
        if (cands[i][1] > 0) {
            cands[i][1]--;
        }
    }
}
void collect(int k, int n, vector<int>& ans) {
    for (int i = 0, cur, real; i < k; i++) {
        if (cands[i][1] > 0) {
            cur = cands[i][0];
            real = 0;
            for (int j = 1; j <= n; j++) {
                if (cur == a[j]) {
                    real++;
                }
            }
            if (real > n / (k + 1)) {
                ans.push_back(cur);
            }
        }
    }
}
}

```

```

vector<int> waterNumber(int n, int k) {
    k = k - 1; // the size of the maintained candidate table is k-1, this
    step is required!!!
    for (int i = 1; i <= n; i++) {
        update(k, a[i]);
    }
    vector<int> ans;
    collect(k, n, ans);
    return ans;
}

```

Matrix Fast Exponentiation

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int mod = 998244353;

// matrix fast exponentiation, matrix multiplication
// often used for dynamic programming optimization
// matrix fast exponentiation time complexity:
// 1-dim k-order, k-dim 1-order:  $O(\log n * k^3)$ 

// matrix fast exponentiation for dp
// the core is to transfer the strict positional dependency of i - 1 and i
// into the matrix, the relation matrix A
// dp[i] = A * dp[i - 1]

// matrix multiplication: the front columns must equal the back rows, (use
// references to save the cost of copying)
vector<vector<ll>> multiply(const vector<vector<ll>>& a, const
vector<vector<ll>>& b) {
    int n = a.size();
    int m = b[0].size();
    int k = a[0].size();
    vector<vector<ll>> ans(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            for (int c = 0; c < k; c++) {
                ans[i][j] += a[i][c] * b[c][j];
            }
        }
    }
    return ans;
}

```

```

// matrix fast exponentiation: strictly for square matrices, i.e. n * n
matrices (n >= 1)
// here do not use references, copy directly
vector<vector<ll>> MatPow(vector<vector<ll>> a, ll b) {
    int n = a.size();
    vector<vector<ll>>ans(n, vector<ll>(n, 0));
    for (int i = 0;i < n;i++) { // the main diagonal must be 1, others 0
        ans[i][i] = 1;
    }
    while (b) {
        if (b & 1) {
            ans = multiply(ans, a);
        }
        a = multiply(a, a);
        b >>= 1;
    }
    return ans;
}

// example: https://ac.nowcoder.com/acm/contest/139659/F
// note!: this problem involves the modulus operation

// relation mapping, rows represent the i-th position, columns represent
the i - 1-th position
// represents the number of schemes for the i-th position to obtain a
state when the i - 1-th position is in some state
vector<vector<ll>>A = { {1,1,0,1},
                      {1,1,1,0},
                      {0,1,1,1},
                      {1,0,1,1} };

// initial unit state
vector<vector<ll>>v0 = { {1},
                       {0},
                       {0},
                       {0} };

void solve() {
    ll n, r;
    cin >> n >> r;
    vector<vector<ll>>ans = multiply(MatPow(A, n), v0);
    cout << ans[r % 4][0] % mod << '\n';
}

```

Multiplicative Functions

```
// function f(n), n>=1
// multiplicative function: f(1)=1 and for gcd(m,n)=1, f(m*n)=f(n)*f(m)
// completely multiplicative function: f(1)=1 and for n>=1, m>=1,
f(m*n)=f(n)*f(m)
// completely multiplicative function
// besides f(n*m)=f(n)*f(m)
// can also be f(n*m*...*k)=f(n)*f(m)*...f(k) with multiple terms
```

Josephus Problem

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// denote the ring size of a certain operation as n, the killed node
number as s, looping k times
// before the operation the node number is denoted OddPos, after the
operation NewPos
// then: OddPos = (NewPos + k - 1) % n + 1
// for the last operation, NewPos = 1

// https://www.luogu.com.cn/problem/P8671

void Cycle() {
    int n, k;
    cin >> n >> k;
    int ans = 1;
    // the last NewPos = 1
    // enumerate the ring size c of the previous operation
    for (int c = 2; c <= n; c++) {
        ans = (ans + k - 1) % c + 1;
    }
    cout << ans << '\n';
}
```

Linear Sieve (Euler's Sieve)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e7;
ll minFactor[MAXN + 10]; // minFactor[i] is the smallest prime factor of i
(linear Euler sieve maintains the smallest prime factor table)
bitset<MAXN + 1>isPrime; // mark primes
ll Primes[MAXN + 1];
void getPrimes_Linear(ll n) {
```

```

isPrime.set();
isPrime[0] = isPrime[1] = 0;
Primes[0] = 0;
for (ll i = 2; i <= n; i++) {
    if (isPrime[i]) {
        minFactor[i] = i; // the smallest prime factor of a prime is
itself
        Primes[++Primes[0]] = i;
    }
    for (ll j = 1, v; j <= Primes[0]; j++) {
        v = Primes[j];
        if (v * i > n) break;
        isPrime[i * v] = 0;
        minFactor[i * v] = v; // get the smallest prime factor, must be
placed before break
        if (i % v == 0) break;
    }
}
}

```

Combinatorics

```

#include<bits/stdc++.h>
using namespace std;

// m=[0,n]
// sum C(n,m)*2^m=3^n
// C(n,0)*2^0+C(n,1)*2^1+...+C(n,m)*2^m+...+C(n,n)*2^n=3^n

//permutations and combinations
ll C(ll n, ll m) {
    if (m == 0 || m == n) return 1;
    else if (m < 0 || m > n) return 0;
    ll ans = N[n] % mod * inv_(N[m] * N[n - m], mod) % mod;
    return ans;
}

ll f(ll n, ll l, ll k) {
    // n elements (unlimited quantity), require using n elements to form
an array of length = l
    // the interval between two identical elements must be >= k
    // 0 <= k < n <= l
    // the number of possible schemes = n!*(n-k)^(l-k)/(n-k)!
    // n-k to the power of l-k

```

```

    return N[n] * pw(n - k, 1 - k) % mod * inv_(N[n - k], mod);
}

```

Bezout's Theorem and Extended Euclidean Algorithm

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Bezout's theorem:
// if a, b are integers not both zero, then there must exist integers x, y
such that  $ax + by = \gcd(a, b)$ 
// note: x, y only need to be integers
// the greatest common divisor of a and b = the smallest positive
difference obtainable by  $ax$  and  $by$  for arbitrary given integers x, y
// corollaries:
// 1. if a, b are integers not both zero, a and b are coprime, if and only
if there exist integers x, y such that  $ax + by = 1$ 
// 2. if a, b are integers not both zero, and  $ax + by = c$  has a solution,
then c must be an integer multiple of  $\gcd(a, b)$  (0 times also counts)
// 3. Bezout's theorem for two terms a and b can be generalized to
multiple terms
// e.g.  $ax + by + cz = g * k$ , where  $g = \gcd(a, b, c)$ ,  $k \geq 0$ , there must
exist a solution for x, y, z

// extended Euclidean algorithm:
// given  $ax + by = c$ , c is a multiple of  $\gcd(a, b)$ 
//  $g = \gcd(a, b)$ 
// exgcd solves a particular solution  $x_0, y_0$  for x, y
// this particular solution  $x_0, y_0$  is the solution of  $ax + by = \gcd(a, b)$ 
// the solution of the current equation  $ax + by$ :  $(xx, yy) = (x_0 * (c / d),$ 
 $y_0 * (c / d))$ 
// general solution:  $x = x_0 + (b / d) * n$ ,  $y = y_0 - (a / d) * n$ 
// among them, the smallest non-negative solution of x is  $\min x = (x_0 * (c$ 
 $/ d) \% (b / d) + (b / d) \% (b / d)$ 
// time complexity  $O((\log \min(a, b))^3)$ 
// note: the passed a, b must not be negative
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;

```

```

        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

```

Trial Division Primality Test

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
bool is_Prime(ll x) {
    //determine whether x is prime
    //complexity O(sqrt(n))
    if (x <= 1) {
        return false;
    }
    else if (x == 2) {
        return true;
    }
    else if (x % 2 == 0) {
        return false;
    }
    else {
        for (ll r = 3; r * r <= x; r += 2) {
            if (x % r == 0) {
                return false;
            }
        }
    }
    return true;
}

```

Prime Factorization

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e7;
bitset<2 * MAXN + 1>isPrime;//mark primes
vector<ll>Primes;//store primes
vector<ll>factors[2 * MAXN + 1];//factors[i] stores the prime factors of i

void getFactors(ll n) {
    //complexity O(sqrt(n)/log2(n))
    //factorize the prime factors of n
    ll val = n;
    if (isPrime[n] == 1) {

```

```

        factors[n].push_back(n);
        return;
    }
    for (ll p : Primes) {
        if (val == 1)break;
        if ((ll)p * p > (ll)val)break;
        if (val % p != 0)continue;
        factors[n].push_back(p);
        while (val % p == 0)val /= p;
    }
    if (val > 1)factors[n].push_back(val);
}

//get the prime factors of all numbers in the range 1~n
void GetFactors(ll n) {
    for (int p : Primes) {
        for (int i = p; i <= n; i += p) {
            factors[i].push_back(p);
        }
    }
}

```

Fermat Inverse and Brute-Force Inverse

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;
ll pw(ll a,ll b){}

//Fermat's little theorem, find the modular inverse
ll inv_(ll a, ll p) {
    return pw(a, p - 2);
}

//brute-force modular inverse
//only needs gcd(a,mod)=1
//time complexity O(mod), recommend using the extended Euclidean algorithm
ll inv02_(ll a, ll mod) {
    a %= mod;
    for (ll t = 1; t < mod; t++) {
        if ((a * t) % mod == 1) return t;
    }
    return 1;
}

```

Base Conversion

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const string DIGIT =
"0123456789"
"abcdefghijklmnopqrstuvwxyz"
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"
"!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~";

//this DIGIT character set supports up to base-94 conversion
//if limited to digits and letters, it supports up to base-62 conversion

//convert x to base
string toBase(ll x, int base) {
    if (x == 0) {
        return "0";
    }
    string ans;
    while (x) {
        ans += DIGIT[x % base];
        x /= base;
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) {
            res = res * a;
        }
        a = a * a;
        b >>= 1;
    }
    return res;
}

//get the i-th digit of x in base
ll getDigit(ll x, int base, int i) {
    return x / pw(base, i) % base;
}

//set the i-th digit in basej to v
```

```
ll setDigit(ll s, int base, int i, int v) {
    ll p = pw(base, i);
    ll old = s / p % base;
    return s + (v - old) * p;
}
```

Linear Recurrence for Modular Inverses

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//linear recurrence for modular inverses
//example: https://www.luogu.com.cn/problem/P3811
ll inv[3000001];
void initial_inv(ll n, ll p) {
    inv[1] = 1;
    for (ll i = 2; i <= n; i++) {
        inv[i] = p - inv[p % i] * (p / i) % p;
    }
}
```

Inverse Cantor Expansion

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Inverse Cantor expansion:
// given numbers 1~n, given a permutation P of 1~n
// the rank of permutation P is x, find the permutation at rank x + m
// core: factorial number system, the i-th digit uses i! as the base (the
lowest digit is 0)
// the digit of the i-th position is rightSmall[P[i]] (Cantor expansion
content)

// https://www.luogu.com.cn/problem/U72177

const int MAXN = 1e5;
int sum[MAXN << 2 | 1];
ll a[MAXN + 1];
void add(int jobi, int jobv, int l, int r, int i);
int range(int jobl, int jobr, int l, int r, int i);
void build(int l, int r, int i);
int queryAndDelete(int jobk, int l, int r, int i);
// core code:
```

```

void compute(int n, ll m) {
    build(1, n, 1);

    // convert each digit of the permutation into the corresponding
    factorial number system digit
    // method: the number of unused elements smaller than the current
    value
    for (int i = 1; i <= n; i++) {
        int tmp = a[i];
        a[i] = range(1, tmp - 1, 1, n, 1);
        add(tmp, -1, 1, n, 1);
    }

    // factorial carry:
    // compared with the next digit, the extra multiplied value val,
    perform division and modulus carry
    a[n] += m;
    for (int i = n; i >= 1; i--) {
        a[i - 1] += a[i] / ll(n - i + 1);
        a[i] %= ll(n - i + 1);
    }
    build(1, n, 1);
    for (int i = 1; i <= n; i++) {
        a[i] = queryAndDelete(a[i] + 1, 1, n, 1);
    }
}

void build(int l, int r, int i) {
    if (l == r) {
        sum[i] = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1);
        build(mid + 1, r, i << 1 | 1);
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
    }
}

int range(int jobl, int jobr, int l, int r, int i) {
    if (jobl > jobr) return 0;
    if (jobl <= l && r <= jobr) {
        return sum[i];
    }
    else {
        int mid = (l + r) >> 1;
        int ans = 0;

```

```

        if (jobl <= mid) {
            ans += range(jobl, jobr, l, mid, i << 1);
        }
        if (jobr > mid) {
            ans += range(jobl, jobr, mid + 1, r, i << 1 | 1);
        }
        return ans;
    }
}

void add(int jobi, int jobv, int l, int r, int i) {
    if (l == r) {
        sum[i] += jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            add(jobi, jobv, l, mid, i << 1);
        }
        else {
            add(jobi, jobv, mid + 1, r, i << 1 | 1);
        }
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
    }
}

int queryAndDelete(int jobk, int l, int r, int i) {
    if (l == r) {
        sum[i]--;
        return l;
    }
    else {
        int lsum = sum[i << 1];
        int rsum = sum[i << 1 | 1];
        int mid = (l + r) >> 1;
        int ans;
        if (lsum >= jobk) {
            ans = queryAndDelete(jobk, l, mid, i << 1);
        }
        else {
            ans = queryAndDelete(jobk - lsum, mid + 1, r, i << 1 | 1);
        }
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
        return ans;
    }
}

```

```

void solve() {
    ll n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    compute(n, m);
    for (int i = 1; i <= n; i++) {
        cout << a[i] << ' ';
    }
}

```

Linear Recurrence for Factorial Inverses

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//linear recurrence for consecutive factorial inverses
//i.e. the inverse of the factorial
ll invN[200001];
void initial_invN(int n) {
    invN[0] = 1;
    invN[n] = inv_(N[n], mod);
    for (ll i = n - 1; i >= 1; i--) {
        invN[i] = (i + 1) * invN[i + 1] % mod;
    }
}

```

Random Number Functions

```

#include<bits/stdc++.h>
using namespace std;
int main()
{
    random_device rd;//get a true random seed

    //performance means generation speed (cpu time) and memory usage
    //quality means the degree of closeness to true randomness; the higher
    it is, the longer the generation period and the weaker the regularity

    //directly using the seed generated by rd() produces true randomness,
    the best quality, but relatively average performance
    uniform_int_distribution<int> dist(1, 100);
    cout << dist(rd) << '\n';
}

```

```

//use a pseudo-random engine to accept the seed from rd(), thus
generating a pseudo-random seed, pseudo-random but with significantly
better performance
//generate a pseudo-random seed from the true random seed
mt19937 gen(rd());//high quality, excellent performance, period about
1e6001

//mt19937_64 is an optimized choice for generating 64-bit values,
period about 1e6001
//minstd_rand has excellent performance and speed, poor quality,
period about 2^31-2
//ranlux48 has extremely high quality, low performance, period about
1e576

uniform_int_distribution<int> int_dist(1, 100);//uniform integer
distribution
uniform_real_distribution<double> real_dist(0.0, 100.0);
cout << int_dist(gen) << '\n';//the source of the random number is gen
(seed)
cout << real_dist(gen) << '\n';
}

```

Gaussian Elimination (Congruence Equations)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 100;
const int MOD = 7;;
// linear recurrence for inverses, get the inverse table
ll inv[MOD + 1];
ll mat[MAXN + 1][MAXN + 1];
void inv_(int mod) {
    inv[1] = 1;
    for (int i = 2; i < mod; i++) {
        inv[i] = (int)(mod - (ll)inv[mod % i] * ll(mod / i) % mod);
    }
}
ll gcd(ll a, ll b) {
    return b == 0 ? a : gcd(b, a % b);
}

// Gaussian elimination for congruence equations
// time complexity O(n ^ 3)
// pivots depend on free variables, free variables do not depend on any

```

```

other variable
// unique solution: there are n pivots (i.e. all columns are pivot
columns)
// multiple solutions: free variables exist, and all rows of free
variables are  $0 = 0$ 
// contradiction: a free variable row is  $0 = b(b \neq 0)$ 

// n variables, n expressions
// the n + 1-th column is the value
// if the number of equations is less than n, zero rows must be padded; if
the equations are redundant (linearly dependent), free variables naturally
appear after elimination
// when checking solutions, only check rows [1, n] of the actual number of
variables n

// when processing the pivot of row i column i
// note!: to strictly distinguish unique solution, contradiction, and
multiple solutions, the non-pivot rows with  $j < i$  must also participate in
the computation
// if only judging whether a unique solution exists, starting from row i
is enough

// !!! note !!!
// if any coefficient is read as negative, use the following formula to
guarantee the initial coefficients of gauss elimination are all positive
//  $-a \% \text{mod} = (-a \% \text{mod} + \text{mod}) \% \text{mod}$ 

void gauss(int n, int mod) {
    for (int i = 1; i <= n; i++) {
        // process the pivot of row i column i
        // get the row where position i equals 1
        // ! note !: to strictly distinguish unique solution,
contradiction, and multiple solutions, the non-pivot rows with  $j < i$  must
also participate in the computation
        // if only judging whether a unique solution exists, starting from
row i is enough
        for (int j = 1; j <= n; j++) {
            if (j < i && mat[j][i] != 0) { // pivot row, skip
                continue;
            }
            if (mat[j][i] != 0) { // for congruence equations, swap
directly as long as a non-zero value is found
                swap(mat[i], mat[j]);
                break;
            }
        }
    }
}

```

```

        if (mat[i][i] != 0) {
            for (int j = 1; j <= n; j++) {
                if (i != j && mat[j][i] != 0) {
                    ll g = gcd(mat[j][i], mat[i][i]);
                    ll a = mat[i][i] / g;
                    ll b = mat[j][i] / g;

                    if (j < i && mat[j][j] != 0) {
                        // !! note !!
                        // if row j has a pivot, multiply all coefficients
                        // from column j to i - 1 by * a
                        // maintain the correct relationship between
                        pivots and free variables
                        for (int k = j; k < i; k++) {
                            mat[j][k] = (mat[j][k] * a) % mod;
                        }
                    }
                    //normal elimination
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] = ((mat[j][k] * a - mat[i][k] * b) % mod
+ mod) % mod;
                    }
                }
            }
        }

        bool sign = false;
        // determine the solution
        for (int i = 1; i <= n; i++) {
            if (mat[i][i] != 0) {
                // pivot row, determine whether it is affected by free
                variables
                bool flg = false;
                for (int j = i + 1; j <= n; j++) {
                    if (mat[i][j] != 0) {
                        flg = true;
                        break;
                    }
                }
                if (!flg) {
                    // not affected by free variables, determine the value of
                    the unique solution
                    mat[i][n + 1] = (mat[i][n + 1] * inv[mat[i][i]]) % mod;
                    mat[i][i] = 1;
                }
            }
        }
    }

```

```

        else {
            if (mat[i][n + 1] != 0) {
                cout << "矛盾, 无解" << '\n';
                return;
            }
            sign = true;
        }
    }
    if (sign) {
        cout << "多解" << '\n';
    }
    else {
        cout << "存在唯一解" << '\n';
    }
}

```

Gaussian Elimination (XOR Equations)

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 100;
int mat[MAXN + 1][MAXN + 1];

// Gaussian elimination for XOR equations
// time complexity  $O(n^3)$ 
// pivots depend on free variables, free variables do not depend on any
// other variable
// unique solution: there are n pivots (i.e. all columns are pivot
// columns)
// multiple solutions: free variables exist, and all rows of free
// variables are  $0 = 0$ 
// contradiction: a free variable row is  $0 = b(b \neq 0)$ 

// the core of the application of Gaussian elimination: convert
// relationships into 0/1 XOR relationships
// for example, changes in parity

// n variables, n expressions
// the n + 1-th column is the value
// if the number of equations is less than n, zero rows must be padded; if
// the equations are redundant (linearly dependent), free variables naturally
// appear after elimination
// when checking solutions, only check rows [1, n] of the actual number of
// variables n

// when processing the pivot of row i column i
// note!: to strictly distinguish unique solution, contradiction, and

```

multiple solutions, the non-pivot rows with $j < i$ must also participate in the computation

// if only judging whether a unique solution exists, starting from row i is enough

```
void gauss(int n) {
    for (int i = 1; i <= n; i++) {
        // process the pivot of row i column i
        // get the row where position i equals 1
        // note!: to strictly distinguish unique solution, contradiction,
        // and multiple solutions, the non-pivot rows with  $j < i$  must also
        // participate in the computation
        // if only judging whether a unique solution exists, starting from
        // row i is enough
        for (int j = 1; j <= n; j++) {
            if (j < i && mat[j][j] == 1) { // pivot row, skip
                continue;
            }
            if (mat[j][i] == 1) { // for XOR equations, swap directly as
                // long as 1 is found
                swap(mat[i], mat[j]);
                break;
            }
        }

        if (mat[i][i] == 1) {
            for (int j = 1; j <= n; j++) {
                if (i != j && mat[j][i] == 1) {
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] ^= mat[i][k];
                    }
                }
            }
        }
    }
}

// determine the type of solution
bool sign = false;
for (int i = 1; i <= n; i++) {
    if (mat[i][i] == 0) { // free variable
        if (mat[i][n + 1] != 0) {
            cout << "矛盾, 无解" << '\n';
            return;
        }
    }
    else {
```

```

        sign = true;
    }
}
}
if (sign) {
    cout << "多解" << '\n';
    return;
}
else {
    cout << "存在唯一解" << '\n';
}
for (int i = 1; i <= n; i++) {
    cout << mat[i][n + 1] << ' ';
}
cout << '\n';
}
}

```

Gaussian Elimination (Linear Equations)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const int MAXN = 100;
const ld eps = 1e-12;
ld mat[MAXN + 1][MAXN + 1];

// Gaussian elimination for linear equations
// time complexity  $O(n^3)$ 
// pivots depend on free variables, free variables do not depend on any
// other variable
// unique solution: there are n pivots (i.e. all columns are pivot
// columns)
// multiple solutions: free variables exist, and all rows of free
// variables are  $0 = 0$ 
// contradiction: a free variable row is  $0 = b(b \neq 0)$ 

// n variables, n expressions
// the n + 1-th column is the value
// if the number of equations is less than n, zero rows must be padded; if
// the equations are redundant (linearly dependent), free variables naturally
// appear after elimination
// when checking solutions, only check rows [1, n] of the actual number of
// variables n

// when processing the pivot of row i column i
// note!: to strictly distinguish unique solution, contradiction, and
// multiple solutions, the non-pivot rows with  $j < i$  must also participate in

```

```

the computation
// if only judging whether a unique solution exists, starting from row i
is enough

void gauss(int n) {
    cout << fixed << setprecision(16);
    for (int i = 1; i <= n; i++) {
        // process the pivot of row i column i
        int maxpick = i;
        // get the row with the largest position i
        // note!: to strictly distinguish unique solution, contradiction,
and multiple solutions, the non-pivot rows with j < i must also
participate in the computation
        // if only judging whether a unique solution exists, starting from
row i is enough
        for (int j = 1; j <= n; j++) {
            if (j < i && fabs(mat[j][j]) >= eps) { // pivot row, skip
                continue;
            }
            if (fabs(mat[j][i]) > fabs(mat[maxpick][i])) {
                maxpick = j;
            }
        }
        // swap the largest row
        swap(mat[i], mat[maxpick]);
        // there is a non-zero maximum, start elimination
        if (fabs(mat[i][i]) >= eps) {
            ld tmp = mat[i][i];
            for (int j = i; j <= n + 1; j++) {
                mat[i][j] /= tmp;
            }
            for (int j = 1; j <= n; j++) {
                if (i != j) {
                    ld rate = mat[j][i] / mat[i][i];
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] -= mat[i][k] * rate;
                    }
                }
            }
        }
    }
}

// determine the type of solution
bool sign = false;

```

```

    for (int i = 1; i <= n; i++) {
        if (fabsl(mat[i][i]) < eps) { // free variable
            if (fabsl(mat[i][n + 1]) >= eps) {
                cout << "矛盾, 无解" << '\n';
                return;
            }
            else {
                sign = true;
            }
        }
    }
    if (sign) {
        cout << "多解" << '\n';
        return;
    }
    else {
        cout << "存在唯一解" << '\n';
    }
    for (int i = 1; i <= n; i++) {
        cout << mat[i][n + 1] << ' ';
    }
    cout << '\n';
}

```

High-Precision Multiplication

```

#include<bits/stdc++.h>
using namespace std;

// a * b
// the array stores digits in reverse order
// from 1 ~ n are low to high digits

// high-precision multiplication, use arrays to store digits, manually
simulate multiplication
// for the final result, the digit length does not exceed the sum of the
digit lengths of a, b
const int MAXN = 300;
int a[MAXN + 1];
int b[MAXN + 1];
int res[2 * MAXN + 1];

void multiply(string s1, string s2) {
    int n = s1.size();
    int m = s2.size();
    for (int i = 1; i <= n; i++) {
        a[i] = s1[n - i] - '0';
    }
}

```

```

    }
    for (int i = 1; i <= m; i++) {
        b[i] = s2[m - i] - '0';
    }
    fill(res, res + n + m + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            res[j + i - 1] += a[i] * b[j];
            if (res[j + i - 1] >= 10) {
                res[j + i] += res[j + i - 1] / 10;
                res[j + i - 1] %= 10;
            }
        }
    }
    bool isZero = true;
    int idx = n + m;
    for (int i = idx; i >= 0; i--) {
        if (res[i] != 0) {
            idx = i;
            isZero = false;
            break;
        }
    }
    if (isZero) {
        cout << 0 << '\n';
        return;
    }
    for (int i = idx; i >= 1; i--) {
        cout << res[i];
    }
    cout << '\n';
}

```

Chicken McNugget Theorem

```

#include<bits/stdc++.h>
using namespace std;

// given two coprime positive integers a, b
// the largest integer that cannot be obtained by  $ax + by = c$  is  $ab - a - b$ 

void MLJK(int a, int b) {
    if (gcd(a, b) != 1) {
        cout << "No delicious!!!" << '\n';
    }
    else {

```

```

        cout << a * b - a - b << '\n';
        cout << "Perfect!!!" << '\n';
    }
}

```

5. Graph Theory

1. Shortest Path Problems

01BFS

```

#include<bits/stdc++.h>
using namespace std;

//0-1 BFS handles graphs whose edge weights (costs) are all 0 or 1
//find the shortest distance from the source to any point
//solved with bfs + deque

//process (core): enqueue, traverse children
//if dist[son]>dist[father]+w, update
//dist[son]=dist[father]+w
//w==0, enqueue at the front
//w==1, enqueue at the back

class Tmp {
private:

    static const int INF = 1e9;

    int mat[501][501]; //2D matrix
    int dist[501][501]; //shortest path

    pair<int, int> dir[4] = { {1,0},{-1,0},{0,1},{0,-1} };

    void bfs(int rx, int ry, int n, int m) {
        dist[rx][ry] = 0; //start path = 0
        deque<array<int, 2>>q;
        q.push_front({ rx,ry });
        while (q.size()) {
            //pop from the front
            auto [x, y] = q.front();
            q.pop_front();

            for (auto [di, dj] : dir) {
                int nx = x + di, ny = y + dj;
                if (nx<1 || nx>n || ny<1 || ny>m) {

```

```

        continue;
    }
    //get the edge weight according to the problem definition
    int w = mat[x][y] == mat[nx][ny] ? 0 : 1;
    if (dist[nx][ny] > dist[x][y] + w) {
        dist[nx][ny] = dist[x][y] + w;

        //core code: 0 enqueue at front, 1 enqueue at back
        if (w == 0) {
            q.push_front({ nx,ny });
        }
        else {
            q.push_back({ nx,ny });
        }
    }
}

}

void init(int n, int m) {
    //initialization.... omitted .....
}

public:
    //template problem: https://www.luogu.com.cn/problem/P4554
    void solve() {
        //input, omitted .....
    }
};

```

A* Algorithm

```

//A* algorithm is an extension of the Djk algorithm
//differences:
//A* algorithm targets the shortest path between a specified start and end
//when pushing into the heap, it does not add the real distance, but real
distance + estimated distance
//A* algorithm relies on a powerful heuristic estimation function to
achieve better path selection
//making the search space closer to the shortest path (like fitting)
//thus achieving a constant-factor optimization of time complexity
//the distance computed by the estimation function <= the real distance

```

Bellman-Ford / SPFA Single-Source Shortest Path

```
#include<bits/stdc++.h>
using namespace std;

//BellmanFord and SPFA
//single-source shortest path algorithm
//queue optimization
//dequeue -> relaxation -> enqueue (if not in the queue) -> dequeue
//requires no negative cycle

//relaxation operation (core):
//use edges and points to check whether to update a smaller dist
//dist[son]=min(dist[father]+wight)

const int MAXN = 2e3;
const int MAXM = 3e3;
const int INF = 1e9;

int idx = 0;
int head[MAXN + 1];
int nxt[2 * MAXM + 1];
int to[2 * MAXM + 1];
int weight[2 * MAXM + 1];

void addEdge(int u, int v, int w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

int roundCnt[MAXN + 1];
int dist[MAXN + 1];
bool vis[MAXN + 1];
//start is the source
int SPFA(int start, int n) {
    queue<int>q;
    q.push(start);
    vis[start] = true;
    roundCnt[start] = 1;//round
    dist[start] = 0;
    while (q.size()) {
        int cur = q.front();
        vis[cur] = false;
        q.pop();
```

```

        for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            int w = weight[edge];
            if (dist[cur] + w < dist[son]) {
                dist[son] = dist[cur] + w;
                if (!vis[son]) {
                    //only when not in the queue, it counts as one round
of relaxation
                    if (++roundCnt[son] >= n) {
                        //detect negative cycle, stuck in a loop
                        return -1;
                    }
                    vis[son] = true;
                    q.push(son);
                }
            }
        }
    }
    return 1;//successfully got the result
}

```

Dijkstra Single-Source Shortest Path

```

#include<bits/stdc++.h>
using namespace std;

//Djk algorithm solves the single-source shortest path problem on graphs
with non-negative edge weights
//directed/undirected graphs, with/without cycles, non-negative edge
weights
//all the above types of graphs can be solved

//reverse-indexed heap optimization
//time complexity: n points, m edges
//O((n+m)logn)
//unoptimized time complexity: O((n+m)logm)

//the difference between Djk and Bfs
//Djk marks when popping
//ordinary bfs marks when enqueueing
//the reason is that all edge weights in bfs are the same, so level-by-
level traversal that arrives earlier is always better
//while complex graphs need Djk to mark when popping, to obtain the
optimal update

class Tmp {

```

```

private:

    // core code follows:

    // reverse-indexed heap optimization is used here
    // ordinary Djk, maintain a min-heap
    // each time it needs to check that it has not been dequeued and
dist[v] is greater than dist[u]+weight
    // if satisfied, update dist[v] and enqueue
    // mark when dequeuing
    void Djk(int s) {
        HeapInsert(0, s);
        dist[s] = 0;
        vis[s] = true;
        while (siz > 0) {
            pop();
            //popping means the optimal solution of this point has been
found
            dist[heapPoint] = heapVal;
            if(vis[heapPoint])continue;
            vis[heapPoint] = true;
            for (int edge = head[heapPoint];edge > 0;edge = nxt[edge]) {
                int son = to[edge];
                ll val = weight[edge];
                //already popped, not accepted
                if (vis[son]) {
                    continue;
                }
                //the HeapInsert function itself provides add and update
options
                HeapInsert(heapVal + val, son);
            }
        }

        using ll = long long;
        static const int MAXN = 1e5;
        static const int MAXM = 2e5;
        static const ll INF = 1e18;

        int head[MAXN + 1];
        int nxt[MAXM + 1];
        int to[MAXM + 1];
        int idx = 0;
        ll weight[MAXM + 1];

```

```

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

int heapPoint;
ll heapVal;
int siz = 0;
int f[MAXN + 1]; // point -> position in the heap
struct {
    ll val;
    int point;
} heap[MAXN + 1];

void HeapSwap(int i, int j) {
    swap(f[heap[i].point], f[heap[j].point]);
    swap(heap[i], heap[j]);
}

void siftUp(int i) {
    while (i > 0 && heap[i].val < heap[(i - 1) >> 1].val) {
        HeapSwap(i, (i - 1) >> 1);
        i = (i - 1) >> 1;
    }
}

void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].val < heap[l].val ? r : l;
        if (heap[best].val >= heap[i].val) {
            return;
        }
        HeapSwap(i, best);
        i = best;
        l = 2 * i + 1, r = l + 1;
    }
}

void update(ll val, int point) {
    int idx = f[point];
    // better, update
    if (heap[idx].val > val) {
        heap[idx].val = val;
        siftUp(idx);
    }
}

```

```

    }
}

void HeapInsert(ll val, int point) {
    if (f[point] != -1) {
        //already inserted, update
        update(val, point);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].point = point;
    heap[cur].val = val;
    f[point] = cur;
    siftUp(cur);
}

void pop() {
    if (siz < 1) {
        heapPoint = -1;
        return;
    }
    HeapSwap(0, --siz);
    heapVal = heap[siz].val;
    heapPoint = heap[siz].point;
    f[heapPoint] = -1; //delete mapping
    siftDown(0);
}

ll dist[MAXN + 1];
bool vis[MAXN + 1];

void init(int n) {
    idx = siz = 0;
    for (int i = 1; i <= n; i++) {
        dist[i] = INF;
        head[i] = 0;
        f[i] = -1;
        vis[i] = false;
    }
}

public:
    //template problem: https://www.luogu.com.cn/problem/P4779
    void solve() {
        //input, omitted .....
    }
}

```

```

    }
};

```

Floyd-Warshall All-Pairs Shortest Path

```

//all-pairs shortest path algorithm
//time complexity  $O(n^3)$ , n is the number of nodes
//enumerating the bridge pivot must be in the outermost loop!!!
//applies to any graph except those with negative cycles (i.e. no cycle
whose sum is negative)
const int MAXN=100;
const int INF=1e9;
int dist[MAXN+1][MAXN+1];
for(int bridge=1;bridge<=n;bridge++){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            //no path, illegal
            if(dist[i][bridge]==INF||dist[bridge][j]==INF)continue;
            dist[i][j]=min(dist[i][j],dist[i][bridge]+dist[bridge][j]);
        }
    }
}

```

Layered Graph Shortest Path

```

#include<bits/stdc++.h>
using namespace std;

//layered graph shortest path (state space graph)
//used when the states of points in the graph are unclear, with different
actual states due to different path choices
//abstract a point into the form {coordinate, state}
//specific to each problem
//for example: {coordinate, bit information} is treated as a point
//run Djk based on this

class Tmp {

private:

    static const int MAXN = 1e4;
    static const int MAXM = 5e4;
    static const int INF = 1e9;

    //build the graph
    int head[MAXN + 1] = { 0 };
    int nxt[2 * MAXM + 1] = { 0 };
    int to[2 * MAXM + 1] = { 0 };

```

```

int idx = 0;
int weight[2 * MAXM + 1];

void addEdge(int u, int v, int w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

//handwritten heap, reverse-indexed heap optimization
int heapCnt;
int heapPoint;
int heapVal;
int siz = 0;
int f[MAXN + 1][11]; //{point, state} -> position in the heap
struct {
    int val; //cost
    int cnt; //free-use count state
    int point; //point
} heap[MAXN + 1];

void HeapSwap(int i, int j) {
    //swap mapping -> swap data
    swap(f[heap[i].point][heap[i].cnt],
f[heap[j].point][heap[j].cnt]);
    swap(heap[i], heap[j]);
}

void siftUp(int i) {
    while (i > 0 && heap[i].val < heap[(i - 1) >> 1].val) {
        HeapSwap(i, (i - 1) >> 1);
        i = (i - 1) >> 1;
    }
}

void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].val < heap[l].val ? r : l;
        if (heap[best].val >= heap[i].val) {
            return;
        }
        HeapSwap(i, best);
        i = best;
        l = 2 * i + 1, r = l + 1;
    }
}

```

```

}

void update(int val, int point, int cnt) {
    int idx = f[point][cnt];
    //better, update
    if (heap[idx].val > val) {
        heap[idx].val = val;
        siftUp(idx);
    }
}

void HeapInsert(int val, int point, int cnt) {
    if (f[point][cnt] != -1) {
        //already inserted, update
        update(val, point, cnt);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].point = point;
    heap[cur].val = val;
    heap[cur].cnt = cnt;
    f[point][cnt] = cur;
    siftUp(cur);
}

void pop() {
    if (siz < 1) {
        heapPoint = -1;
        return;
    }
    HeapSwap(0, --siz);
    heapVal = heap[siz].val;
    heapPoint = heap[siz].point;
    heapCnt = heap[siz].cnt;
    f[heapPoint][heapCnt] = -1; //delete mapping
    siftDown(0);
}

bool vis[MAXN + 1][11] = { false };

int Djk(int st, int ed, int k) {
    vis[st][0] = true;
    HeapInsert(0, st, 0);
    while (siz > 0) {
        pop();
        vis[heapPoint][heapCnt] = true;
        if (heapPoint == ed) {

```

```

        return heapVal;
    }
    for (int edge = head[heapPoint]; edge > 0; edge = nxt[edge]) {
        int son = to[edge];
        int cost = weight[edge];
        if (vis[son][heapCnt]) continue;
        //not using a free pass
        //the HeapInsert function itself provides add, update, and
no-change checks
        HeapInsert(cost + heapVal, son, heapCnt);
        if (heapCnt < k) {
            if (vis[son][heapCnt + 1]) continue;
            HeapInsert(heapVal, son, heapCnt + 1);
        }
    }
}

}

}

public:

    //template problem: https://www.luogu.com.cn/problem/P4568
    void solve() {
        //omitted .....
    }
};

```

TSP Shortest Path Problem

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 20;

// core: state compression
// f(status,i) represents the shortest path when in state status and
village i is the last one reached
// enumerate the next village j to reach
// dp[s][i]=min(dp[s|(1<<j)][j]+dist[i][j])

ll dist[MAXN + 1][MAXN + 1];
ll dp[1 << MAXN][MAXN + 1];
ll solve() {
    //template problem: https://www.luogu.com.cn/problem/P1171
    int n;
    cin >> n;
    memset(dp, 0x7f7f7f7f7f7f7f7f, sizeof(dp));
}

```

```

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> dist[i][j];
        }
    }
    //n=4,status=111(bit states only store the non-start points)
    int status = (1 << (n - 1)) - 1;
    for (int j = 2; j <= n; j++) {
        dp[status][j] = dist[j][1];
    }
    ll ans = 1e18;
    //the bit state of s represents 2~n
    for (int s = status; s >= 0; s--) {
        for (int i = 2; i <= n; i++) {
            if ((s >> (i - 2)) & 1) {
                //position i is the last reached
                for (int j = 2; j <= n; j++) {
                    if (!((s >> (j - 2)) & 1)) {
                        //the next position reached is j
                        dp[s][i] = min(dp[s][i], dp[s | (1 << (j - 2))][j]
+ dist[i][j]));
                    }
                }
            }
        }
    }
    //enumerate the first position reached
    for (int i = 2; i <= n; i++) {
        ans = min(ans, dp[1 << (i - 2)][i] + dist[1][i]);
    }
    return ans;
}

```

Congruence Shortest Path (Two-Loop Method)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int inf = 1e9;
const ll INF = 1e18;

const int MAXN = 12;
const int MAXV = 5e5;
int a[MAXN + 1];
ll dist[MAXV + 1];

int gcd(int a, int b) {

```

```

    return b == 0 ? a : gcd(b, a % b);
}

void Cycle(int n) {
    int x = a[1];
    fill(dist, dist + x, INF);
    dist[0] = 0;
    for (int i = 2; i <= n; i++) {
        int d = gcd(a[i], x);
        for (int st = 0; st < d; st++) {
            for (int cur = st, next, round = 0; round < 2; round += (cur ==
st ? 1 : 0)) {
                next = (cur + a[i]) % x;
                if (dist[cur] != INF) {
                    dist[next] = min(dist[next], dist[cur] + a[i]);
                }
                cur = next;
            }
        }
    }
}

ll Cal(ll r, int x) {
    if (r < 0) return 0;
    ll ans = 0;
    for (int i = 0; i < x; i++) {
        if (r >= dist[i]) {
            ans += max(0LL, (r - dist[i]) / x + 1);
        }
    }
    return ans;
}

void solve() {
    ll n, l, r;
    cin >> n >> l >> r;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    int siz = 0;
    sort(a + 1, a + n + 1);
    for (int i = 1; i <= n; i++) {
        if (a[i] != 0) {
            a[++siz] = a[i];
        }
    }
    if (siz == 0) {

```

```

        cout << 0 << '\n';
    }
    else {
        Cycle(siz);
        cout << Cal(r, a[1]) - Cal(1 - 1, a[1]) << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}

```

Congruence Shortest Path (Dijkstra)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// https://www.luogu.com.cn/problem/P3403

// solve the complete knapsack problem with large data
// group by the remainder of a specific value
// note: group by remainder [0, x - 1], so it is based on 0 rather than 1
// depending on the problem, the knapsack capacity may need customization,
// e.g. n = n - 1
// the finally computed dist[i] is the smallest reachable value with %
// base = i

const int MAXN = 1e5;
const ll INF = LLONG_MAX;
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
ll dist[MAXN + 1];
bool vis[MAXN + 1];

```

```

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

void Djk() {
    dist[0] = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>>q;
    q.push({ 0,0 });
    while (q.size()) {
        auto [d, u] = q.top();
        q.pop();
        if (vis[u])continue;
        dist[u] = d;
        vis[u] = true;
        for (int edge = head[u];edge > 0;edge = nxt[edge]) {
            int v = to[edge];
            if (!vis[v] && dist[v] > dist[u] + (ll)weight[edge]) {
                dist[v] = dist[u] + weight[edge];
                q.push({ dist[v],v });
            }
        }
    }
}

void solve() {
    ll h, x, y, z;
    cin >> h >> x >> y >> z;
    h = h - 1;
    // make the minimum value x, to reduce the cost
    if (y > z)swap(y, z);
    if (x > y)swap(x, y);
    fill(dist, dist + x + 1, INF);
    for (int i = 0;i < x;i++) {
        addEdge(i, (i + y) % x, y);
        addEdge(i, (i + z) % x, z);
    }
    Djk();
    ll ans = 0;
    for (int i = 0;i < x;i++) {
        if (dist[i] <= h) {
            ans += (h - dist[i]) / x + 1;
        }
    }
}

```

```

    cout << ans << '\n';
}

```

Negative Cycle and Difference Constraints (Floyd)

```

#include<bits/stdc++.h>
using namespace std;

// template: https://www.luogu.com.cn/problem/P2474
// the core of the Floyd algorithm is that the u, v relationship is
// bridged through bridge
// this problem only requires judging the relationship of the two pairs a,
// b and u, v
// and any relationship between any two is given, so the Floyd algorithm
// can be used to make the relationship strict

int dmin[51][51]; // the minimum possible difference between i and j
int dmax[51][51]; // the maximum possible difference between i and j

void solve() {
    int n, a, b;
    cin >> n >> a >> b;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            char r;
            cin >> r;
            if (r == '=') {
                dmin[i][j] = dmax[i][j] = 0;
            }
            else if (r == '-') {
                // i < j
                dmin[i][j] = -2;
                dmax[i][j] = -1;
            }
            else if (r == '+') {
                // i > j
                dmin[i][j] = 1;
                dmax[i][j] = 2;
            }
            else {
                dmin[i][j] = -2;
                dmax[i][j] = 2;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        dmin[i][i] = dmax[i][i] = 0;
    }
}

```

```

    }
    // bridge, make the pairwise relationships stricter
    for (int bridge = 1; bridge <= n; bridge++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                dmin[i][j] = max(dmin[i][j], dmin[i][bridge] +
dmin[bridge][j]);
                dmax[i][j] = min(dmax[i][j], dmax[i][bridge] +
dmax[bridge][j]);
            }
        }
    }
    int ans1 = 0, ans2 = 0, ans3 = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if (i != a && j != a && i != b && j != b) {
                if (dmin[a][i] > dmax[j][b] || dmin[a][j] > dmax[i][b]) {
                    ans1++;
                }
                if (dmax[a][i] < dmin[j][b] || dmax[a][j] < dmin[i][b]) {
                    ans3++;
                }
                if (dmin[a][i] == dmax[a][i] && dmin[j][b] == dmax[j][b]
&& dmin[a][i] == dmin[j][b]) {
                    ans2++;
                }
                else if (dmin[b][i] == dmax[b][i] && dmin[j][a] ==
dmax[j][a] && dmin[j][a] == dmin[b][i]) {
                    ans2++;
                }
            }
        }
    }
    cout << ans1 << ' ' << ans2 << ' ' << ans3 << '\n';
}
int main() {
    solve();
    return 0;
}

```

Negative Cycle and Difference Constraints (SPFA)

```

#include<bits/stdc++.h>
using namespace std;

// two forms of difference constraints
// 1.  $x_i - y_i \leq c_i$  is equivalent to  $x_i \leq y_i + c_i$ 

```

```

// ci is a constant
// build edge yi->xi, ci is the edge weight, initialize all point weights
to (+INF)
// through SPFA, keep relaxing, if a negative cycle appears, no solution

// 2.  $x_i - y_i \geq c_i$  is equivalent to  $x_i \geq y_i + c_i$ 
// ci is a constant
// build edge yi->xi, ci is the edge weight, initialize all point weights
to (-INF)
// through SPFA, keep expanding, if a positive cycle appears, no solution

// connect the super source:
// when building the graph, a super source root is needed as the initial
point connecting all points
// the two can be converted into each other, the difference lies in
relaxation and expansion, negative cycle and positive cycle
// ! note ! because of the super source root, the judgment condition is
not  $\geq n$ , but  $> n$ 

// restrict the super source:
// if the values of several variables are already determined
// build a restricted super source limitRoot, connect positive and
negative edges to the variables with determined values respectively
// the super source root and the restricted super source must be separated

// SPFA
// time complexity:  $O(n * m)$ 
// n: number of points  m: number of edges

// template: https://www.luogu.com.cn/problem/P5960

const int MAXN = 5e3;
using ll = long long;
const ll INF = 1e18;

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
int roundCnt[MAXN + 1];
ll dist[MAXN + 1];
bool vis[MAXN + 1];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;

```

```

    to[idx] = v;
    weight[idx] = w;
}

void SPFA(int n) {
    int root = 0;
    queue<int>q;
    vis[root] = true;
    dist[root] = 0;
    roundCnt[root] = 1;
    q.push(root);
    while (q.size()) {
        int u = q.front();
        vis[u] = false;
        q.pop();
        for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
            int v = to[edge];
            if (dist[v] > dist[u] + weight[edge]) {
                dist[v] = dist[u] + weight[edge];
                if (!vis[v]) {
                    // !!! note !!!
                    // here because of the super source root
                    // the judgment condition is not >= n, but > n
                    if (++roundCnt[v] > n) {
                        cout << "NO" << '\n';
                        return;
                    }
                }
                q.push(v);
                vis[v] = true;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        cout << dist[i] << ' ';
    }
}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        addEdge(0, i, 0);
        head[i] = 0;
        vis[i] = false;
        dist[i] = INF;
        roundCnt[i] = 0;
    }
}

```

```

    }
    for (int i = 1; i <= m; i++) {
        ll x, y, c;
        cin >> x >> y >> c;
        // x - y <= c
        // x <= y + c
        // y->x, c
        addEdge(y, x, c);
    }
    SPFA(n);
}

```

2. Tree Problems

DFS Detection of Negative Cycle

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// directed weighted graph
// dfs to determine whether a negative cycle exists

const int MAXN = 3e3;
const int MAXM = 1e4;
int head[MAXN + 1];
int nxt[MAXM + 1];
int to[MAXM + 1];
ll weight[MAXM + 1];
ll value[MAXN + 1];
bool vis[MAXN + 1];
int idx = 0;

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

bool dfs(int u, int n) {
    vis[u] = true;
    if (u == 0) {
        for (int i = 1; i <= n; i++) {
            if (dfs(i, n)) {
                return true;
            }
        }
    }
}

```

```

    }
    else {
        for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
            int v = to[edge];
            ll w = weight[edge];
            if (value[v] > value[u] + w) {
                value[v] = value[u] + w;
                if (vis[v] || dfs(v, n)) {
                    return true;
                }
            }
        }
    }
    vis[u] = false;
    return false;
}

bool check(int n) {
    // determine whether a negative cycle exists, set the accumulated path
    // weights of all points to 0, ensuring only negative numbers trigger updates
    fill(value, value + n + 1, 0);
    fill(vis, vis + n + 1, false);
    // start from super source 0 to reach each point
    return dfs(0, n);
}

```

LCA via Binary Lifting

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 5e5;
//binary lifting on the tree to find LCA
//preprocessing O(nlogn)
//single online query O(logn)
//overall time complexity: O(nlogn)

//https://www.luogu.com.cn/problem/P3379

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int power = 21;
int deep[MAXN + 1];
int stjump[MAXN + 1][21];

```

```

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void dfs(int cur, int fa) {
    deep[cur] = deep[fa] + 1;
    stjump[cur][0] = fa;
    for (int p = 1; p <= power; p++) {
        stjump[cur][p] = stjump[stjump[cur][p - 1]][p - 1];
    }
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(to[edge], cur);
        }
    }
}

void buildStjump(int n, int root) {
    power = ceil(log2(n));
    dfs(root, 0);
}

int lca(int a, int b) {
    if (deep[a] < deep[b]) {
        swap(a, b);
    }
    for (int p = power; p >= 0; p--) {
        if (deep[stjump[a][p]] >= deep[b]) {
            a = stjump[a][p];
        }
    }
    if (a == b) {
        return a;
    }
    for (int p = power; p >= 0; p--) {
        if (stjump[a][p] != stjump[b][p]) {
            a = stjump[a][p];
            b = stjump[b][p];
        }
    }
    return stjump[a][0];
}

void solve() {
    int n, q, root;

```

```

    cin >> n >> q >> root;
    idx = 0;
    fill(head, head + n + 1, 0);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    buildStjump(n, root);
    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << lca(a, b) << '\n';
    }
}

```

LCA via Tarjan

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 5e5;

//offline processing
//offline processing: O(n)
//single offline query: O(1)
//overall complexity: O(n)
//core code: dfs + union-find
//template problem: https://www.luogu.com.cn/problem/P3379

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int ans[MAXN + 1];
vector<array<int, 2>>query[MAXN + 1];
int fa[MAXN + 1];
bool vis[MAXN + 1];
int find(int u) {
    return fa[u] == u ? u : fa[u] = find(fa[u]);
}

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

```

```

void tarjan(int cur, int father) {
    // !!! once entered, immediately mark as arrived
    vis[cur] = true;
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != father) {
            tarjan(to[edge], cur);
            //!!! note !!!
            //!!! setting the parent must be written after the child node
            recursion backtracks !!!
            fa[to[edge]] = cur;
        }
    }
    // !!! after all child nodes are traversed, then query the answer
    int a = cur;
    for (auto& [b, i] : query[a]) {
        if (vis[b]) {
            ans[i] = find(b);
        }
    }
}

void solve() {
    int n, q, root;
    cin >> n >> q >> root;
    idx = 0;
    fill(head, head + n + 1, 0);
    fill(vis, vis + n + 1, 0);
    for (int i = 1; i <= n; i++) {
        fa[i] = i;
    }
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    for (int i = 1; i <= q; i++) {
        int a, b;
        cin >> a >> b;
        query[a].push_back({ b, i });
        query[b].push_back({ a, i });
    }
    tarjan(root, 0);
    for (int i = 1; i <= q; i++) {
        cout << ans[i] << '\n';
    }
}

```

```

    }
}

```

Tree Difference (Point and Edge)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// tree point and edge difference, offline algorithm

// the root node needs a virtual node added as its parent
// given a tree and tree nodes u, v (u != v)
// add + val to the weights of points on the path u->v
// array a is the real point weight
// lc = lca(u, v)
// a[u] += val, a[v] += val, a[lc] -= val, a[fa[lc]] -= val

// add + val to the weights of edges on the path u->v
// array b is the temporary point-weight array for auxiliary edge-weight
// modification, not the real point weight
// lc = lca(u, v)
// b[u] += val, b[v] += val, b[lc] -= 2 * val
// finally traverse the tree once with dfs

const int MAXN = 5e4;
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll a[MAXN + 1];
ll weight[2 * MAXN + 1];
ll b[MAXN + 1];
int idx = 0;
int power = 32;
int deep[MAXN + 1];
int stjump[MAXN + 1][32];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

void dfsStjump(int cur, int fa) {
    deep[cur] = deep[fa] + 1;
    stjump[cur][0] = fa;

```

```

    for (int p = 1;p <= power;p++) {
        stjump[cur][p] = stjump[stjump[cur][p - 1]][p - 1];
    }
    for (int edge = head[cur];edge > 0;edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfsStjump(to[edge], cur);
        }
    }
}

void buildStjump(int n, int root) {
    power = ceil(log2(n));
    deep[0] = 0;
    dfsStjump(root, 0);
}

int lca(int a, int b) {
    if (deep[a] < deep[b]) {
        swap(a, b);
    }
    for (int p = power;p >= 0;p--) {
        if (deep[stjump[a][p]] >= deep[b]) {
            a = stjump[a][p];
        }
    }
    if (a == b) {
        return a;
    }
    for (int p = power;p >= 0;p--) {
        if (stjump[a][p] != stjump[b][p]) {
            a = stjump[a][p];
            b = stjump[b][p];
        }
    }
    return stjump[a][0];
}

//point difference core code
void dfsPoint(int u, int fa) {
    for (int v, edge = head[u];edge > 0;edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfsPoint(v, u);
            a[u] += a[v];
        }
    }
}

```

```

void addPoint(int u, int v, ll val) {
    a[u] += val;
    a[v] += val;
    int lc = lca(u, v);
    a[lc] -= val;
    a[stjump[lc][0]] -= val;
}

//edge difference core code
void addRoad(int u, int v, ll val) {
    b[u] += val;
    b[v] += val;
    int lc = lca(u, v);
    b[lc] -= 2 * val;
}

void dfsRoad(int u, int fa) {
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfsRoad(v, u);
            b[u] += b[v];
            weight[edge] += b[v];
        }
    }
}

```

Tree Diameter (Two DFS)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// tree diameter: the path formed by the two farthest points on the tree
// property: based on edge weights all  $\geq 0$  (non-negative)
// 1. if there are multiple diameters, they must share a common middle
part (a point or a path segment)
// 2. for any point p on the tree, in the set of points farthest from p,
at least one of the two endpoints of the diameter is in it

// tree merging, shortest diameter edge weight problem:
// condition: all edge weights must be non-negative and exactly equal
// two trees, the number of edges on the diameters are A, B, then after
merging the longest diameter =  $A+B+1$ , the shortest =  $\max(A, B, \lceil A/2 + B/2 + 1 \rceil)$ ,
each division rounded up

```

```

// solution:
// 1. two dfs, can obtain the diameter, points along the way.... and other
information
// condition: non-negative edge weights
// 2. tree dp, can only obtain the diameter
// condition: applies to any tree

const int MAXN = 5e5;
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int lst[MAXN + 1]; // parent node of the path
ll dist[MAXN + 1];
int idx = 0;
int st, ed; ll diameter;
void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}
void initial(int n) {
    idx = 0;
    fill(head, head + n + 1, 0);
}
void dfs(int u, int fa) {
    lst[u] = fa;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dist[to[edge]] = dist[u] + weight[edge];
            dfs(to[edge], u);
        }
    }
}
void GetMaxRoad(int n) {
    dist[1] = 0;
    dfs(1, 0);
    st = 1;
    for (int i = 2; i <= n; i++) {
        if (dist[i] > dist[st]) {
            st = i;
        }
    }
    fill(lst, lst + n + 1, 0);
    dist[st] = 0;
}

```

```

    dfs(st, 0);
    ed = 1;
    for (int i = 2; i <= n; i++) {
        if (dist[i] > dist[ed]) {
            ed = i;
        }
    }
    diameter = dist[ed];
}

void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v; ll w;
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    GetMaxRoad(n);
    cout << st << "->" << ed << " " << diameter << "\n";
}

```

Tree Diameter (Tree DP)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// tree diameter: the path formed by the two farthest points on the tree
// property: based on edge weights all  $\geq 0$  (non-negative)
// 1. if there are multiple diameters, they must share a common middle
// part (a point or a path segment)
// 2. for any point p on the tree, in the set of points farthest from p,
// at least one of the two endpoints of the diameter is in it

// tree merging, shortest diameter edge weight problem:
// condition: all edge weights must be non-negative and exactly equal
// two trees, the number of edges on the diameters are A, B, then after
// merging the longest diameter = A+B+1, the shortest =  $\max(A, B, \lceil \frac{A+B}{2} \rceil)$ ,
// each division rounded up

// solution:
// 1. two dfs, can obtain the diameter, points along the way.... and other
// information
// condition: non-negative edge weights
// 2. tree dp, can only obtain the diameter

```

```

// condition: applies to any tree

const ll INF = 1e18;
const int MAXN = 5e5;
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
ll dist[MAXN + 1]; // the longest path obtained by going downward with u as
the head (single journey)
ll ans[MAXN + 1]; // the longest path that must include u (u as the head)
ll diameter = -INF;
void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}
void initial(int n) {
    idx = 0;
    fill(head, head + n + 1, 0);
    fill(ans, ans + n + 1, 0);
    fill(dist, dist + n + 1, 0);
}
void dfs(int u, int fa) {
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        int v = to[edge];
        if (v != fa) {
            dfs(v, u);
        }
    }
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        int v = to[edge];
        if (v != fa) {
            // historical longest vs the new path length (historical
            longest single journey + new path length)
            // diameter = max(diameter, dist[u] + dist[v] + weight[edge]);
            simplified code here
            ans[u] = max(ans[u], dist[u] + dist[v] + weight[edge]);
            // get the longest single journey path (downward)
            dist[u] = max(dist[u], dist[v] + weight[edge]);
        }
    }
}
void GetMaxRoad(int n) {
    dfs(1, 0);
}

```

```

        for (int i = 1; i <= n; i++) {
            diameter = max(diameter, ans[i]);
        }
    }
}

void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    GetMaxRoad(n);
    cout << diameter << '\n';
}

```

Tree Centroid (Using Subtree Size)

```

#include<bits/stdc++.h>
using namespace std;

// tree centroid: node p
// 1. with node p as the root, the maximum number of nodes among all
subtrees is minimized
// 2. with node p as the root, the number of nodes of each subtree is <=
half of the total number of nodes (rounded down)
// 3. with node p as the root, the total number of edges from all nodes to
p is the smallest
// properties:
// 1. there are at most two centroids, which must be adjacent
// 2. if a leaf node is added to or removed from the tree, the centroid
after the change moves at most one edge
// 3. if two trees are connected, the new centroid must be on the path of
the original centroids
// 4. if all edge weights on the tree are >= 0, there must exist a
centroid with the smallest total distance from all nodes

//http://poj.org/problem?id=1655
const int MAXN = 2e4;
int best, G;
int siz[MAXN + 1];
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;

```

```

void initial(int n) {
    best = INT_MAX;
    G = INT_MAX;
    idx = 0;
    fill(siz, siz + n + 1, 0);
    fill(head, head + n + 1, 0);
}

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void dfs(int n, int u, int fa) {
    siz[u] = 1;
    int maxsub = 0;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(n, to[edge], u);
            siz[u] += siz[to[edge]];
            maxsub = max(maxsub, siz[to[edge]]);
        }
    }
    maxsub = max(maxsub, n - siz[u]);
    if (maxsub < best || (maxsub == best && u < G)) {
        G = u;
        best = maxsub;
    }
}

void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(n, 1, 0);
    cout << G << ' ' << best << '\n';
}

```

Tree Centroid (Using Subtree Node Count)

```

#include<bits/stdc++.h>
using namespace std;

```

```

// tree centroid: node p
// 1. with node p as the root, the maximum number of nodes among all
subtrees is minimized
// 2. with node p as the root, the number of nodes of each subtree is <=
half of the total number of nodes (rounded down)
// 3. with node p as the root, the total number of edges from all nodes to
p is the smallest
// properties:
// 1. there are at most two centroids, which must be adjacent
// 2. if a leaf node is added to or removed from the tree, the centroid
after the change moves at most one edge
// 3. if two trees are connected, the new centroid must be on the path of
the original centroids
// 4. if all edge weights on the tree are >= 0, there must exist a
centroid with the smallest total distance from all nodes

//http://poj.org/problem?id=3107
const int MAXN = 5e4;
int siz[MAXN + 1];
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int G[3];
int maxsub[MAXN + 1];
void initial(int n) {
    idx = 0;
    fill(G, G + 3, 0);
    fill(maxsub, maxsub + n + 1, 0);
    fill(siz, siz + n + 1, 0);
    fill(head, head + n + 1, 0);
}
void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}
void dfs(int n, int u, int fa) {
    siz[u] = 1;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(n, to[edge], u);
            siz[u] += siz[to[edge]];
            maxsub[u] = max(maxsub[u], siz[to[edge]]);
        }
    }
    maxsub[u] = max(maxsub[u], n - siz[u]);
}

```

```

}
void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(n, 1, 0);
    for (int i = 1; i <= n; i++) {
        if (maxsub[i] <= n / 2) {
            G[++G[0]] = i;
        }
    }
    for (int i = 1; i <= G[0]; i++) {
        cout << G[i] << ' ';
    }
    cout << '\n';
}

```

3. Spanning Trees

Kruskal Minimum Spanning Tree

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//minimum spanning tree, Kruskal algorithm
//sort by edge weight from small to large
//if connecting the current edge does not form a cycle then connect,
otherwise skip
//reaching n-1 edges completes
//the complexity depends on the number of edges
//n points, m edges, time complexity O(mlogn)

//template problem: https://www.luogu.com.cn/problem/P3366

const int MAXN = 5e3;
const int MAXM = 2e5;

array<int, 3> edge[MAXM + 1];

//union-find
int father[MAXN + 1];

```

```

int siz[MAXN + 1];
int road[MAXN + 1];

int find(int u) {
    int cnt = -1;
    while (u != father[u]) {
        road[++cnt] = u;
        u = father[u];
    }
    while (cnt >= 0) {
        father[road[cnt]] = u;
        cnt--;
    }
    return u;
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if (ru == rv) return false;
    if (siz[ru] <= siz[rv]) {
        father[ru] = rv;
        siz[rv] += siz[ru];
    }
    else {
        father[rv] = ru;
        siz[ru] += siz[rv];
    }
    return true;
}

void initial(int n) {
    //omitted .....
}

bool cmp(array<int, 3>a, array<int, 3>b) {
    return a[2] < b[2];
}

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//minimum spanning tree, Kruskal algorithm
//sort by edge weight from small to large
//if connecting the current edge does not form a cycle then connect,
otherwise skip

```

```

//reaching n-1 edges completes
//the complexity depends on the number of edges
//n points, m edges, time complexity  $O(m \log n)$ 

//template problem: https://www.luogu.com.cn/problem/P3366

const int MAXN = 5e3;
const int MAXM = 2e5;

array<int, 3> edge[MAXM + 1];

//union-find
int father[MAXN + 1];
int siz[MAXN + 1];
int road[MAXN + 1];

int find(int u) {
    int cnt = -1;
    while (u != father[u]) {
        road[++cnt] = u;
        u = father[u];
    }
    while (cnt >= 0) {
        father[road[cnt]] = u;
        cnt--;
    }
    return u;
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if (ru == rv) return false;
    if (siz[ru] <= siz[rv]) {
        father[ru] = rv;
        siz[rv] += siz[ru];
    }
    else {
        father[rv] = ru;
        siz[ru] += siz[rv];
    }
    return true;
}

void initial(int n) {
    //omitted .....
}

```

```

bool cmp(array<int, 3>a, array<int, 3>b) {
    return a[2] < b[2];
}
void solve() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int n, m;
    cin >> n >> m;
    initial(n);
    for (int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edge[i][0] = u;
        edge[i][1] = v;
        edge[i][2] = w;
    }
    sort(edge + 1, edge + m + 1, cmp);
    ll ans = 0;
    ll cnt = 0;
    for (int i = 1; i <= m; i++) {
        int u = edge[i][0], v = edge[i][1];
        ll w = edge[i][2];
        if (merge(u, v)) {
            ans += w;
            cnt++;
        }
        if (cnt == n - 1) break;
    }
    if (cnt == n - 1) {
        cout << ans << '\n';
    }
    else {
        cout << "orz" << '\n';
    }
}

```

Prim Minimum Spanning Tree

```

#include<bits/stdc++.h>
using namespace std;

//minimum spanning tree, Prim algorithm
//template problem: https://www.luogu.com.cn/problem/P3366

```

```

//Prim algorithm, reverse-indexed heap optimization
//time complexity, n points, m edges
//the optimized implementation needs a handwritten heap Heap
//O(n+m)+O((n+m)*logn)
class PrimBetter {
private:
    using ll = long long;

    static const int MAXN = 5e3;
    static const int MAXM = 2e5;

    int head[MAXN + 1];
    int nxt[2 * MAXM + 1];
    int to[2 * MAXM + 1];
    ll w[2 * MAXM + 1];
    int idx = 0;

    bool vis[MAXN + 1];

    void addEdge(int u, int v, ll z) {
        nxt[++idx] = head[u];
        head[u] = idx;
        to[idx] = v;
        w[idx] = z;
    }

    void init(int n) {
        fill(head, head + n + 1, 0);
        fill(vis, vis + n + 1, false);
        fill(f, f + n + 1, -1);
        siz = 0;
    }

    int f[MAXN + 1]; //reverse index table of the heap, the position of
point i in the heap
    pair<ll, int> heap[MAXN + 1]; //heap: {value, point}
    int siz = 0; //size of the heap

    int heapIdx;
    ll heapVal;

    //float up
    void siftUp(int i) {
        while (i > 0 && heap[i].first < heap[(i - 1) >> 1].first) {
            swap(f[heap[i].second], f[heap[(i - 1) >> 1].second]);
            swap(heap[i], heap[(i - 1) >> 1]);
            i = (i - 1) >> 1;
        }
    }
};

```

```

    }
}

//sink down
void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].first < heap[l].first ? r : l;
        if (heap[best].first >= heap[i].first) {
            return;
        }
        swap(f[heap[best].second], f[heap[i].second]);
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
        r = l + 1;
    }
}

//insert a new value
void HeapInsert(ll val, int idx) {
    if (f[idx] != -1) {
        //already in the heap, perform the update operation
        update(idx, val);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].first = val;
    heap[cur].second = idx;
    f[idx] = cur;
    siftUp(cur);
}

//update the value
void update(int idx, ll val) {
    int i = f[idx]; //get the position of this point in the heap
    if (val < heap[i].first) {
        heap[i].first = val;
        siftUp(i);
    }
}

//pop the top of the heap
void pop() {
    if (siz == 0) heapIdx = -1;
    swap(f[heap[0].second], f[heap[--siz].second]);
    swap(heap[0], heap[siz]);
}

```

```

        heapIdx = heap[siz].second;
        heapVal = heap[siz].first;
        f[heap[siz].second] = -1;//delete
        siftDown(0);
    }

    ll Prim(int n) {
        int cnt = 1;
        vis[1] = true;
        ll ans = 0;
        for (int edge = head[1]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            ll val = w[edge];
            HeapInsert(val, son);
        }
        while (siz > 0) {
            pop();
            if (vis[heapIdx]) {
                continue;
            }
            ans += heapVal;
            vis[heapIdx] = true;//mark as popped, i.e. on the tree
            cnt++;
            for (int edge = head[heapIdx]; edge > 0; edge = nxt[edge]) {
                int son = to[edge];
                ll val = w[edge];
                if (vis[son]) {
                    //popped, i.e. on the tree
                    continue;
                }
                HeapInsert(val, son);
            }
        }

        if (cnt != n) return -1;
        return ans;
    }

public:

    void solve() {
        //omitted .....
    }
};

class PrimNormal {
    //unoptimized, general time complexity

```

```

//n points, m edges, O(n+m)+O(mlogm)
private:

    using ll = long long;

    static const int MAXN = 5e3;
    static const int MAXM = 2e5;

    int head[MAXN + 1];
    int nxt[2 * MAXM + 1];
    int to[2 * MAXM + 1];
    ll w[2 * MAXM + 1];
    int idx = 0;

    bool vis[MAXN + 1];

    void addEdge(int u, int v, ll z) {
        nxt[++idx] = head[u];
        head[u] = idx;
        to[idx] = v;
        w[idx] = z;
    }

    void init(int n) {
        fill(head, head + n + 1, 0);
        fill(vis, vis + n + 1, false);
    }

    ll Prim(int n) {
        int cnt = 1;
        ll ans = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
greater<pair<ll, int>>>q;
        vis[1] = true;
        for (int edge = head[1]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            ll val = w[edge];
            q.push({ val, son });
        }
        while (q.size()) {
            auto [val, cur] = q.top();
            q.pop();
            if (!vis[cur]) {
                ans += val;
                cnt++;
                vis[cur] = true;
                for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {

```

```

        int son = to[edge];
        ll val = w[edge];
        q.push({ val, son });
    }
}
}
if (cnt < n) ans = -1;
return ans;
}

public:
    void solve() {
        //omitted .....
    }
};

```

Topological Sort

```

//topological sort can be used to query cycles (only determine whether a
cycle exists)
//after the topological sort operation completes
//the remaining points with degree > 0 are the points on the cycle
//core code, in bfs only enqueue when indegree = 0

class TopoSort {
private:
    //pure topological sort template code
    static const int MAXN = 2e5;

    //build the graph with forward star
    int idx = 0;
    int head[MAXN + 1];
    int next[2 * MAXN + 1];
    int to[2 * MAXN + 1];
    int indegree[MAXN + 1]; //indegree
    int rank[MAXN + 1];
    int cnt = 0;

    void add_edge(int u, int v) {
        next[++idx] = head[u];
        head[u] = idx;
        to[idx] = v;
        indegree[v]++;
    }

    bool TopoSort(int n) {

```

```

        //topological sort
        //indegree deletion method, enqueue when indegree is 0, delete the
influence in turn

        //if you need to output the lexicographically smallest order
        //change the queue to a min-heap
        //priority_queue<int,vector<int>,greater<int>>q;
        queue<int>q;
        for (int i = 1;i <= n;i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        while (q.size()) {
            int cur = q.front();//q.top()
            q.pop();
            rank[++cnt] = cur;
            for (int edge = head[cur];edge > 0;edge = next[edge]) {
                int son = to[edge];
                //core code, enqueue when indegree = 0
                if (--indegree[son] == 0) {
                    q.push(son);
                }
            }
        }

        return cnt == n;
    }

    void PrintTopo() {
        for (int i = 1;i <= cnt;i++) {
            cout << rank[i] << ' ';
        }
        cout << '\n';
    }

    void init(int n) {
        fill(head, head + n + 1, 0);
        cnt = idx = 0;
        fill(indegree, indegree + n + 1, 0);
    }

public:

    void solve() {
        int n, m;

```

```

        cin >> n >> m;
        for (int i = 1; i <= m; i++) {
            int u, v;
            cin >> u >> v;
            add_edge(u, v);
        }
        if (!TopoSort(n)) { // sort failed
            cout << -1 << '\n';
            return;
        }
        PrintTopo();
    }
};

```

6. Computational Geometry

Pick's Theorem for Lattice Points

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;

// given segment (x1, y1), (x2, y2)
// the number of lattice points on the segment = gcd(| x1 - x2 | , | y1 - y2 | ) + 1
// (including endpoints)

// polygon area: S
// lattice points on the boundary: B
// lattice points inside: I
//  $S = I + B / 2 - 1$ 

// the area can be computed with the shoelace formula

struct pointll {
    ll x, y;
};

ll crossLL(pointll a, pointll b) {
    return a.x * b.y - a.y * b.x;
}

//  $S = I + B / 2 - 1$ 
// Pick's theorem to get the number of boundary lattice points
// boundary lattice points B: each edge contributes  $\gcd(|dx|, |dy|)$  points
// (only counting the start point, not the end point)

```

```
// note: here integer doubled arithmetic is used to avoid precision issues
ll boundaryPoints(const vector<pointll>& p) {
    int n = p.size();
    ll B = 0;
    for (int i = 0; i < n; i++) {
        pointll a = p[i], b = p[(i + 1) % n];
        ll dx = llabs(a.x - b.x);
        ll dy = llabs(a.y - b.y);
        B += std::gcd(dx, dy);
    }
    return B;
}
```

3D Geometric Centroid

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point {
    ld x, y, z;
};

point add(point a, point b) {
    return { a.x + b.x, a.y + b.y, a.z + b.z };
}

point mul(point a, ld k) {
    return { a.x * k, a.y * k, a.z * k };
}

point cross(point a, point b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

ld dot(point a, point b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

// triangle face
```

```

struct Triangle {
    point a, b, c;
};

// 3D polyhedron centroid
// all triangle faces, with consistent vertex order (counterclockwise from
the outside)
point polyhedronCentroid(const vector<Triangle>& tris) {
    point C = { 0, 0, 0 };
    ld V = 0;

    for (auto [a, b, c] : tris) {
        ld vol = dot(a, cross(b, c)) / 6.0;    // signed volume
        V += vol;

        // tetrahedron centroid = (0 + a + b + c) / 4
        point ci = mul(add(add(a, b), c), 1.0 / 4.0);
        C = add(C, mul(ci, vol));
    }

    if (fabs(V) < 1e-12) return { 0, 0, 0 };    // degenerate
    return mul(C, 1.0 / V);
}

```

3D Polygon Area

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 3D:
struct point3 {
    ld x, y, z;
};

point3 cross(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

point3 add(point3 a, point3 b) {
    return { a.x + b.x, a.y + b.y, a.z + b.z };
}

ld dot(point3 a, point3 b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

```

```

}
ld len(point3 a) {
    return sqrt(dot(a, a));
}
// 3D polygon area, p is the vertices (arranged in order and coplanar)
ld polygonArea3D(vector<point3> p) {
    int n = p.size();
    if (n < 3) return 0;

    point3 N = { 0, 0, 0 }; // accumulate area normal vectors
    for (int i = 0; i < n; i++) {
        point3 a = p[i];
        point3 b = p[(i + 1) % n];
        N = add(N, cross(a, b));
    }
    return len(N) / 2.0;
}

// 3D space vertex ordering:
// 3D coplanar point ordering (counterclockwise order, based on convex
hull)
// premise: the input points must be coplanar and form a convex polygon
(or return the convex hull vertices after computing the hull)
// output: sorted 3D points (counterclockwise), directly usable for
polygonArea3D
vector<point3> sortPoints3D(vector<point3> p) {
    int n = p.size();
    if (n <= 3) return p; // triangle returns directly

    // 1. compute the normal vector of the plane where the polygon lies
(Newell's method)
    point3 N = { 0, 0, 0 };
    for (int i = 0; i < n; i++) {
        point3 a = p[i];
        point3 b = p[(i + 1) % n];
        N = add(N, cross(a, b));
    }

    // 2. choose the projection plane: ignore the component with the
largest absolute value in the normal vector
    int dropAxis;
    if (fabs(N.x) >= fabs(N.y) && fabs(N.x) >= fabs(N.z)) dropAxis = 0;
// ignore x
    else if (fabs(N.y) >= fabs(N.z)) dropAxis = 1;
// ignore y
    else dropAxis = 2;

```

```

// ignore z

// 3. project to 2D
vector<pair<ld, ld>> pts2D(n);
for (int i = 0; i < n; i++) {
    if (dropAxis == 0) pts2D[i] = { p[i].y, p[i].z };
    else if (dropAxis == 1) pts2D[i] = { p[i].x, p[i].z };
    else pts2D[i] = { p[i].x, p[i].y };
}

// 4. compute the convex hull in 2D (Andrew's algorithm), returning
the indices of points
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0);
sort(idx.begin(), idx.end(), [&](int i, int j) {
    if (pts2D[i].first != pts2D[j].first) return pts2D[i].first <
pts2D[j].first;
    return pts2D[i].second < pts2D[j].second;
});

vector<int> hullIdx;
// lower hull
for (int i : idx) {
    while (hullIdx.size() >= 2) {
        int m = hullIdx.size();
        auto v1 = make_pair(pts2D[hullIdx[m - 1]].first -
pts2D[hullIdx[m - 2]].first,
pts2D[hullIdx[m - 1]].second - pts2D[hullIdx[m -
2]].second);
        auto v2 = make_pair(pts2D[i].first - pts2D[hullIdx[m -
2]].first,
pts2D[i].second - pts2D[hullIdx[m - 2]].second);
        ld cr = v1.first * v2.second - v1.second * v2.first; // 2D
cross product
        if (cr <= 0) hullIdx.pop_back();
        else break;
    }
    hullIdx.push_back(i);
}
// upper hull
int lowerSize = hullIdx.size();
for (int i = n - 2; i >= 0; i--) {
    int idxVal = idx[i];
    while ((int)hullIdx.size() > lowerSize) {
        int m = hullIdx.size();
        auto v1 = make_pair(pts2D[hullIdx[m - 1]].first -
pts2D[hullIdx[m - 2]].first,

```

```

                pts2D[hullIdx[m - 1]].second - pts2D[hullIdx[m -
2]].second);
            auto v2 = make_pair(pts2D[idxVal].first - pts2D[hullIdx[m -
2]].first,
                pts2D[idxVal].second - pts2D[hullIdx[m - 2]].second);
            ld cr = v1.first * v2.second - v1.second * v2.first;
            if (cr <= 0) hullIdx.pop_back();
            else break;
        }
        hullIdx.push_back(idxVal);
    }
    hullIdx.pop_back(); // remove the duplicated start point

    // 5. return the 3D points by hull indices
    vector<point3> result;
    for (int i : hullIdx) result.push_back(p[i]);
    return result;
}

```

2D Geometric Centroid

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point { ld x, y; };
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// return the polygon centroid, vertices arranged in boundary order
point polygonCentroid(const vector<point>& p) {
    int n = p.size();
    if (n < 3) return { 0, 0 }; // degenerate case

    ld A = 0, cx = 0, cy = 0;
    for (int i = 0; i < n; i++) {
        point a = p[i];
        point b = p[(i + 1) % n];
        ld cr = cross(a, b); // twice the signed area
        A += cr;
        cx += (a.x + b.x) * cr;
        cy += (a.y + b.y) * cr;
    }
    A /= 2; // signed area
    if (fabs(A) < eps) return { 0, 0 }; // area is 0, centroid does not
    exist
}

```

```
    return { cx / (6 * A), cy / (6 * A) };  
}
```

2D Polygon Area (Shoelace Formula)

```
#include<bits/stdc++.h>  
using namespace std;  
using ld = long double;  
using ll = long long;  
  
// 2D  
struct point {  
    ld x, y;  
};  
ld cross(point a, point b) {  
    return a.x * b.y - a.y * b.x;  
}  
  
// triangle area: given three sides (Heron's formula)  
ld triangleAreaBySides(ld a, ld b, ld c) {  
    ld p = (a + b + c) / 2;  
    return sqrt(p * (p - a) * (p - b) * (p - c));  
}  
  
// triangle area: given vertex coordinates  
ld triangleAreaByPoints(point A, point B, point C) {  
    return fabs(A.x * (B.y - C.y) + B.x * (C.y - A.y) + C.x * (A.y - B.y))  
    / 2.0;  
}  
  
// polygon area: shoelace formula, vertices in order (counterclockwise or  
clockwise, convex or concave both OK)  
ld polygonArea(const vector<point>& p) {  
    int n = p.size();  
    if (n < 3) return 0;  
    ld area = 0;  
    for (int i = 0; i < n; i++) {  
        int j = (i + 1) % n;  
        // clockwise: x_later*y_former - x_former*y_later  
        // counterclockwise: x_former*y_later - x_later*y_former  
        area += p[i].x * p[j].y - p[j].x * p[i].y;  
    }  
    return fabs(area) / 2.0;  
}
```

```

struct pointll {
    ll x, y;
};
ll crossLL(pointll a, pointll b) {
    return a.x * b.y - a.y * b.x;
}

// S = I + B / 2 - 1
// Pick's theorem to get the number of boundary lattice points
// boundary lattice points B: each edge contributes gcd(|dx|, |dy|) points
// (only counting the start point, not the end point)
// note: here integer doubled arithmetic is used to avoid precision issues
ll boundaryPoints(const vector<pointll>& p) {
    int n = p.size();
    ll B = 0;
    for (int i = 0; i < n; i++) {
        pointll a = p[i], b = p[(i + 1) % n];
        ll dx = labs(a.x - b.x);
        ll dy = labs(a.y - b.y);
        B += std::gcd(dx, dy);
    }
    return B;
}

```

Convex Hull (Andrew's Algorithm)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 7. Convex hull (Andrew's algorithm): the convex hull is the smallest
// convex polygon that can enclose all points
// the construction direction of the convex hull rotates
// counterclockwise from the leftmost point
// lower hull: scan from left->right, forming the lower half of the
// convex hull (starting from the leftmost point, passing through the lower
// boundary, to the rightmost point)
// upper hull: scan from right->left, forming the upper half of the
// convex hull (starting from the rightmost point, passing through the upper
// boundary, back to the leftmost point)
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) > 0,
// counterclockwise left turn
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) < 0, clockwise
// right turn
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) = 0, three points
// collinear

```

```

struct point { //2D
    ld x, y;
};
ld dot(point a, point b) {
    // 3D = x1 * x2 + y1 * y2 + z1 * z2
    return a.x * b.x + a.y * b.y;
}

point cal(point a, point b) {
    return { a.x - b.x,
            a.y - b.y };
}

ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
}

bool cmp(point a, point b) {
    if (a.x == b.x) return a.y < b.y;
    return a.x < b.x;
}

void Andrew(point* st, point* p) {
    // if the number of points n < 3, all points are the convex hull
    const int n = 10;
    // point st[2 * n]; the size of st needs to be doubled
    sort(p + 1, p + n + 1, cmp); // sort by x ascending, y ascending
    // lower hull
    int top = 0;
    for (int i = 1; i <= n; i++) {
        while (top >= 2 && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--; // pop the top of the stack
        }
        st[++top] = p[i]; // push the current point
    }
    // upper hull: traverse in reverse from n-1 to 1 (skip the rightmost
point, it is already in the lower hull)
    int tmp = top; // record the lower hull size
    for (int i = n - 1; i >= 1; i--) {
        while (top > tmp && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--;
        }
        st[++top] = p[i];
    }
    top--; // pop the extra p[1], maintain counterclockwise order, now st is

```

```
the convex hull vertices
}
```

Determining the Type of Angle

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

//2D given vectors, find the angle
struct point { ld x, y; };
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }

// return value: 1 = acute, 0 = right, -1 = obtuse, -2 = zero vector
// exists, cannot determine
int angleType(point a, point b) {
    ld la = len(a), lb = len(b);
    if (la < eps || lb < eps) return -2;    // zero vector undefined

    ld d = dot(a, b);
    if (d > eps) return 1;                  // acute
    if (d < -eps) return -1;               // obtuse
    return 0;                              // right
}

// 3D
struct point3 { ld x, y, z; };
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }

// return value: 1 = acute, 0 = right, -1 = obtuse, -2 = zero vector
// exists, cannot determine
int angleType3D(point3 a, point3 b) {
    ld la = len3(a), lb = len3(b);
    if (la < eps || lb < eps) return -2;

    ld d = dot3(a, b);
    if (d > eps) return 1;
    if (d < -eps) return -1;
    return 0;
}
```

Angle Between Vectors

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

//2D given vectors, find the angle
struct point { ld x, y; };
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }

// return the angle between vectors a and b (radians, range [0,  $\pi$ ])
ld angleBetween(point a, point b) {
    ld la = len(a), lb = len(b);
    if (la < eps || lb < eps) return 0;    // zero vector undefined, handle
as needed

    ld cosv = dot(a, b) / (la * lb);

    // prevent floating-point error from making cosv exceed [-1, 1]
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;

    return acos(cosv);
}

// 3D
struct point3 { ld x, y, z; };
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }

// return the angle between vectors a and b (radians, range [0,  $\pi$ ])
ld angleBetween3D(point3 a, point3 b) {
    ld la = len3(a), lb = len3(b);
    if (la < eps || lb < eps) return 0;    // zero vector undefined

    ld cosv = dot3(a, b) / (la * lb);

    // prevent floating-point error from making cosv exceed [-1, 1]
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;

    return acos(cosv);
}
```

Circle-Related (2D)

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point { ld x, y; };
struct Circle {
    point O;
    ld R;
};

// three points determine a circle
// return the circle, if the three points are collinear return a special
// value (e.g. radius -1)
Circle circleFrom3Points(point A, point B, point C) {
    ld ax = A.x, ay = A.y;
    ld bx = B.x, by = B.y;
    ld cx = C.x, cy = C.y;

    ld d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by));
    if (fabs(d) < 1e-12) return { {0,0}, -1 }; // three points
    collinear, return radius -1 to indicate failure

    ld ux = ((ax * ax + ay * ay) * (by - cy) +
        (bx * bx + by * by) * (cy - ay) +
        (cx * cx + cy * cy) * (ay - by)) / d;
    ld uy = ((ax * ax + ay * ay) * (cx - bx) +
        (bx * bx + by * by) * (ax - cx) +
        (cx * cx + cy * cy) * (bx - ax)) / d;

    ld R = hypot(ax - ux, ay - uy);
    return { {ux, uy}, R };
}

// tangent points of point P and circle G

// other computation functions needed
point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }
ld dist(point a, point b) { return len(sub(a, b)); }
```

```

// rotate the vector counterclockwise by theta (radians)
// theta = angle * (PI / 180.0)
// PT = acos(-1.0)
point rotate(point v, ld theta) {
    ld c = cos(theta), s = sin(theta);
    return { v.x * c - v.y * s, v.x * s + v.y * c };
}

// find the tangent points of point P to the circle (C, r)
// return value: number of tangent points (0/1/2), tangent points stored
// in the t array
int tangentPoints(point P, point C, ld r, point t[2]) {
    ld d = dist(P, C);

    if (d < r - eps) return 0;           // point inside the circle, no
    tangent
    if (fabs(d - r) < eps) {             // point on the circle, the
    tangent point is itself
        t[0] = P;
        return 1;
    }

    point v = sub(P, C);                 // C -> P
    v = mul(v, 1.0 / d);                 // normalize

    ld theta = acos(r / d);              // rotation angle
    point v1 = rotate(v, theta);
    point v2 = rotate(v, -theta);

    t[0] = add(C, mul(v1, r));           // tangent point 1
    t[1] = add(C, mul(v2, r));           // tangent point 2
    return 2;
}

```

Plane Normal Vector

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;

// 2D
struct point { ld x, y; };

// normal vector of line AB (not unique, take any one)
point normal2D(point A, point B) {

```

```

    point v = {B.x - A.x, B.y - A.y};
    return {-v.y, v.x};    // or {v.y, -v.x}
}

// 3D
struct point3 { ld x, y, z; };

point3 cross(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

point3 cal(point3 a, point3 b) {    // a - b
    return {a.x - b.x, a.y - b.y, a.z - b.z};
}

// normal vector of plane ABC (A,B,C not collinear)
point3 normal3D(point3 A, point3 B, point3 C) {
    return cross(cal(B, A), cal(C, A));
}

```

Radian Angle Type Determination and Conversion

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;
const ld PI = acos(-1.0);
const ld HALF_PI = PI / 2;
const ld DEG_TO_RAD = PI / 180.0;
const ld RAD_TO_DEG = 180.0 / PI;

// degrees to radians
ld degToRad(ld deg) {
    return deg * DEG_TO_RAD;
}

// radians to degrees
ld radToDeg(ld rad) {
    return rad * RAD_TO_DEG;
}

// determine the angle type based on radians

```

```

// return 0=zero angle, 1=acute, 2=right, 3=obtuse, 4=straight, -1=outside
[0,π]
int angleTypeByRad(ld rad) {
    rad = fabs(rad);
    if (rad < eps) return 0;
    if (fabs(rad - HALF_PI) < eps) return 2;
    if (rad < HALF_PI) return 1;
    if (fabs(rad - PI) < eps) return 4;
    if (rad < PI) return 3;
    return -1;
}

// directly determine the angle type from degrees (optional)
int angleTypeByDeg(ld deg) {
    return angleTypeByRad(degToRad(deg));
}

```

Dot Product and Cross Product

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// dot product: the signed projection length of vector a onto vector b *
the length of b
// properties: same direction is maximum, perpendicular is 0, opposite
direction is negated
// functions:

// 1. determine the angle type (= 0 perpendicular, > 0 acute, < 0 obtuse)

// 2. determine the direction relationship

// 3. A!=B, given a third point P, determine the relationship
// draw a line through A (2D) or a plane (3D), using it as the boundary,
determine where P is
// dot(B - A, P - A) = 0, P is on the line or plane through A
perpendicular to AB
// dot(B - A, P - A) > 0, P is on the half-plane or half-space in the AB
direction
// dot(B - A, P - A) < 0, P is on the half-plane or half-space in the BA
direction

// 4. find the projection point: given point P, find the projection point
of P onto AB, t is the ratio coefficient
// ld t = dot(P - A, B - A) / dot(B - A, B - A)
// projection point = A + t * (B - A)

```

```

// t=0 projection point is A, t=1 projection point is B, t<0 on the
extension line of A in the BA direction, t>1 on the extension line of B in
the AB direction

// 5. find the length of vector a: sqrt(dot(a, a))

// 6. find the angle between two vectors a, b: (C is the angle)
// cosC = dot(a, b) / (| a | * | b | )
// ld C = acos(dot(a, b) / (| a | * | b | ))

// 7. 3D determine the facing of a face: dot(face normal vector, view
direction) < 0, the front faces the camera (i.e. facing your eyes)
// view direction: from the eye pointing to the face

struct point { ld x, y; };
// 3D = x1 * x2 + y1 * y2 + z1 * z2
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }

// cross product:
// 2D: the result is a number, equal to the area of the parallelogram
formed by vector a and vector b (with sign)
//      the sign indicates b is in the counterclockwise(+) / clockwise(-)
direction of a, 0 indicates collinear
// 3D: the result is a vector, perpendicular to the plane of vector a and
vector b, length equal to the parallelogram area
//      direction by the right-hand rule: four fingers from vector a to
vector b, the thumb direction is the result direction
// properties: a x b = -b x a; parallel/collinear is 0

// functions (2D cross product):

// 1. determine clockwise/counterclockwise direction
//      cross(a, b) > 0, b is counterclockwise of a
//      cross(a, b) < 0, b is clockwise of a
//      cross(a, b) == 0, a and b are collinear

// 2. determine which side of the segment the point is on
//      cross(B - A, P - A) > 0, P is on the left of AB (counterclockwise
direction)
//      cross(B - A, P - A) < 0, P is on the right of AB (clockwise
direction)
//      cross(B - A, P - A) == 0, P is on line AB

// 3. determine whether two segments intersect (straddle test)
//      the condition for segments AB and CD to intersect:
//      cross(B - A, C - A) and cross(B - A, D - A) have opposite signs and
//      cross(D - C, A - C) and cross(D - C, B - C) have opposite signs

```

```

// (note boundary cases need extra handling, e.g. endpoint on the
// segment, will be misjudged as not intersecting)

// 4. find the triangle area (triangle ABC)
//  $S = | \text{cross}(B - A, C - A) / 2 |$ 

// 5. find the polygon area (shoelace formula)
// area = 0;
// enumerate the polygon endpoints
// for (int i = 0; i < n; i++)
//     area += cross(p[i], p[(i+1)%n]);
// area = fabs(area) / 2;

// 6. polar angle sorting (sort by angle)
// polar angle: the angle between a vector and the positive x-axis,
// counted counterclockwise
// compare the angles of two vectors v1, v2:
// cross(v1, v2) > 0 , v2 > v1
// cross(v1, v2) < 0 , v1 > v2
// cross(v1, v2) = 0, collinear, direction unknown
// dot(v1, v2) > 0 same direction equal, < 0 opposite 180°

// 7. Convex hull (Andrew's algorithm): the convex hull is the smallest
// convex polygon that can enclose all points
// the construction direction of the convex hull rotates
// counterclockwise from the leftmost point
// lower hull: scan from left->right, forming the lower half of the
// convex hull (starting from the leftmost point, passing through the lower
// boundary, to the rightmost point)
// upper hull: scan from right->left, forming the upper half of the
// convex hull (starting from the rightmost point, passing through the upper
// boundary, back to the leftmost point)
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) > 0,
// counterclockwise left turn
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) < 0, clockwise
// right turn
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) = 0, three points
// collinear

point cal(point a, point b) {
    return { a.x - b.x,
            a.y - b.y };
}

ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
}

```

```

}
bool cmp(point a, point b) {}
void Andrew() {
    // if the number of points n < 3, all points are the convex hull
    const int n = 10;
    point st[2 * n];
    point p[n];
    sort(p + 1, p + n + 1, cmp); // sort by x ascending, y ascending
    // lower hull
    int top = 0;
    for (int i = 1; i <= n; i++) {
        while (top >= 2 && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--; // pop the top of the stack
        }
        st[++top] = p[i]; // push the current point
    }
    // upper hull: traverse in reverse from n-1 to 1 (skip the rightmost
point, it is already in the lower hull)
    int tmp = top; // record the lower hull size
    for (int i = n - 1; i >= 1; i--) {
        while (top > tmp && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--;
        }
        st[++top] = p[i];
    }
    top--; // pop the extra p[1], maintain counterclockwise order, now st is
the convex hull vertices
}

// functions (3D cross product):

// 1. find the plane normal vector
//    n = cross(B - A, C - A), perpendicular to the plane of triangle ABC
//    direction determined by the right-hand rule

// 2. determine which side of the plane the point is on
//    dot(cross(B - A, C - A), P - A) > 0, P is on one side of the plane
(the side the normal points to)
//    dot(cross(B - A, C - A), P - A) < 0, P is on the other side of the
plane
//    dot(cross(B - A, C - A), P - A) == 0, P is on the plane

// 3. find the 3D triangle area
//    S = |cross(B - A, C - A)| / 2

```

```

// 4. combine with dot product to determine the face facing
//     normal vector n = cross(B - A, C - A), view direction eye -> face
//     dot(n, eyeDir) < 0 front faces the camera (see dot product item 7)

struct pointt {
    ld x, y, z;
};
pointt Cross(pointt a, pointt b) {
    return { a.y * b.z - a.z * b.y,
            a.z * b.x - a.x * b.z,
            a.x * b.y - a.y * b.x };
}

```

Point-Line and Point-Plane Relationship

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// 2D point-line determination

struct point { ld x, y; };
point cal(point a, point b) { return { a.x - b.x, a.y - b.y }; }
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// return >0 for left side, <0 for right side, 0 on the line
int sideOfLine(point P, point A, point B) {
    ld d = cross(cal(B, A), cal(P, A));
    if (d > eps) return 1;      // left
    if (d < -eps) return -1;    // right
    return 0;                  // on the line
}

//3D point-plane determination

struct point3 { ld x, y, z; };
point3 cal3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 cross3(point3 a, point3 b) {
    return { a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x };
}
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

```

```

// return >0 for the same side as the normal vector, <0 for the opposite
// side of the normal vector, 0 on the plane
int sideOfPlane(point3 P, point3 A, point3 B, point3 C) {
    point3 n = cross3(cal3(B, A), cal3(C, A)); // plane normal vector
    ld d = dot3(n, cal3(P, A));
    if (d > eps) return 1; // same side as the normal vector
    if (d < -eps) return -1; // opposite side of the normal vector
    return 0; // on the plane
}

```

Point-Line and Point-Plane Projection

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

//2D, point-line projection
struct point { ld x, y; };
point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }

// projection point of point P onto line AB (A != B)
point projectToLine2D(point P, point A, point B) {
    point v = sub(B, A);
    point w = sub(P, A);
    ld t = dot(w, v) / dot(v, v);
    return add(A, mul(v, t));
}

//3D, point-line projection
struct point3 { ld x, y, z; };
point3 add3(point3 a, point3 b) { return { a.x + b.x, a.y + b.y, a.z + b.z }; }
point3 sub3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 mul3(point3 a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

// projection point of point P onto line AB (A != B)
point3 projectToLine3D(point3 P, point3 A, point3 B) {
    point3 v = sub3(B, A);
    point3 w = sub3(P, A);
    ld t = dot3(w, v) / dot3(v, v);
}

```

```

        return add3(A, mul3(v, t));
    }

//3D, point-plane projection
point3 cross3(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// projection point of point P onto plane ABC (A, B, C not collinear)
point3 projectToPlane3D(point3 P, point3 A, point3 B, point3 C) {
    point3 n = cross3(sub3(B, A), sub3(C, A));    // normal vector
    point3 w = sub3(P, A);
    ld s = dot3(w, n) / dot3(n, n);
    point3 offset = mul3(n, s);
    return sub3(P, offset);    // P - n * s
}

```

Point-to-Line Distance

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 2D
struct point { ld x, y; };
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
// distance from point P to line AB (A, B are two points on the line, A != B)
ld pointLineDist(point P, point A, point B) {
    point v = { B.x - A.x, B.y - A.y };    // direction vector
    point w = { P.x - A.x, P.y - A.y };    // AP vector
    return fabs(cross(w, v)) / sqrt(dot(v, v));
}

// 3D
struct point3 { ld x, y, z; };
point3 sub(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 cross(point3 a, point3 b) {    // 3D cross product
    return {

```

```

        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}
ld dot(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len(point3 a) { return sqrt(dot(a, a)); }
// distance from point P to 3D line AB, A != B
ld pointLineDist3D(point3 P, point3 A, point3 B) {
    point3 v = sub(B, A);    // direction vector B - A
    point3 w = sub(P, A);    // P - A
    point3 c = cross(w, v); // parallelogram area vector
    return len(c) / len(v);
}

```

Angle Between Lines

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// ----- 2D -----
struct point {
    ld x, y;
};

point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// 2D angle between two lines (direction vectors v1, v2)

ld angleBetweenLines2D(point v1, point v2) {
    ld la = len(v1), lb = len(v2);
    if (la < eps || lb < eps) return 0;    // zero vector undefined
    ld cosv = dot(v1, v2) / (la * lb);
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;
    return acos(cosv);
}

// 2D intersection point of two lines

```

```

// line 1 determined by A1,B1, line 2 determined by A2,B2
// return 0: one normal intersection (returned through res)
// return 1: parallel, no intersection
// return 2: collinear, infinitely many intersections
int intersectLines2D(point A1, point B1, point A2, point B2, point& res) {
    point v1 = sub(B1, A1);
    point v2 = sub(B2, A2);
    ld d = cross(v1, v2);

    if (fabs(d) < eps) {
        // parallel or collinear
        if (fabs(cross(sub(A2, A1), v1)) < eps) return 2; // collinear
        return 1; // parallel,
no intersection
    }

    point w = sub(A2, A1);
    ld t = cross(w, v2) / d;
    res = add(A1, mul(v1, t));
    return 0;
}

// ----- 3D -----
struct point3 {
    ld x, y, z;
};

point3 add3(point3 a, point3 b) { return { a.x + b.x, a.y + b.y, a.z +
b.z }; }
point3 sub3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z -
b.z }; }
point3 mul3(point3 a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }
point3 cross3(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// 3D angle between two lines (direction vectors v1, v2)

ld angleBetweenLines3D(point3 v1, point3 v2) {
    ld la = len3(v1), lb = len3(v2);
    if (la < eps || lb < eps) return 0;
}

```

```

    ld cosv = dot3(v1, v2) / (la * lb);
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;
    return acos(cosv);
}

// 3D shortest distance between two lines (common perpendicular segment)
// and the closest point pair

// line 1 determined by A1,B1, line 2 determined by A2,B2
// return the shortest distance, the closest points returned through p1,p2
ld distanceLines3D(point3 A1, point3 B1, point3 A2, point3 B2, point3& p1,
point3& p2) {
    point3 v1 = sub3(B1, A1);
    point3 v2 = sub3(B2, A2);
    point3 w = sub3(A2, A1);

    point3 n = cross3(v1, v2);
    ld nlen = len3(n);

    if (nlen > eps) {
        // skew lines: common perpendicular method
        ld s = dot3(cross3(w, v2), n) / (nlen * nlen);
        ld t = dot3(cross3(w, v1), n) / (nlen * nlen);

        p1 = add3(A1, mul3(v1, s));
        p2 = add3(A2, mul3(v2, t));
        return len3(sub3(p1, p2));
    }
    else {
        // the two lines are parallel: take the projection of A2 onto line
1
        p1 = A2;
        point3 v = v1;
        point3 ap = sub3(A2, A1);
        ld t = dot3(ap, v) / dot3(v, v);
        p2 = add3(A1, mul3(v, t));
        return len3(sub3(p1, p2));
    }
}

```

Line-Plane Angle and Projection

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

```

```

// ----- 3D basics -----
struct point { ld x, y, z; };
point add(point a, point b) { return { a.x + b.x, a.y + b.y, a.z +
b.z }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y, a.z -
b.z }; }
point mul(point a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len(point a) { return sqrt(dot(a, a)); }
point cross(point a, point b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// ----- line-plane angle -----
// line direction vector d, plane normal vector n
// return the line-plane angle (radians), range [0,  $\pi/2$ ]
ld angleLinePlane(point d, point n) {
    ld ldv = len(d), lnv = len(n);
    if (ldv < eps || lnv < eps) return 0;    // zero vector undefined

    ld sinv = fabs(dot(d, n)) / (ldv * lnv);
    if (sinv > 1) sinv = 1;
    return asin(sinv);
}

// ----- plane normal vector (determined by three points) -----
point normalOfPlane(point A, point B, point C) {
    return cross(sub(B, A), sub(C, A));
}

// ----- point to plane projection -----
// projection point of point P onto the plane (through point A, normal
vector n)
point projectPointToPlane(point P, point A, point n) {
    point w = sub(P, A);
    ld s = dot(w, n) / dot(n, n);
    return sub(P, mul(n, s));
}

// ----- vector to plane projection -----
// project vector v onto the plane with normal vector n, getting the
component within the plane

```

```

point projectVectorToPlane(point v, point n) {
    ld s = dot(v, n) / dot(n, n);
    return sub(v, mul(n, s));
}

// ----- line to plane projection -----
// line L: through point P0, direction vector d
// plane  $\pi$ : through point A, normal vector n
// projected line L' through point P0_proj, direction vector d_proj
// note: if d_proj is close to the zero vector, the original line is
perpendicular to the plane, the projection degenerates to a point
void projectLineToPlane(point P0, point d, point A, point n,
    point& P0_proj, point& d_proj) {
    P0_proj = projectPointToPlane(P0, A, n);
    d_proj = projectVectorToPlane(d, n);
}

```

7. Data Structures

1. Trie

Basic Trie

```

#include<bits/stdc++.h>

//Trie (prefix tree, dictionary tree), build the tree according to prefix
characters
//query based on prefix information
//low time complexity, slightly high space complexity
//template problem:
https://www.nowcoder.com/practice/7f8a8553ddb44eaab749ec988726702b

using std::string;
using std::fill;

const int MAXN = 1e6;
int tree[MAXN + 1][26]; //'a'~'z', 0~25
int pass[MAXN + 1]; //how many strings pass through this point
int end[MAXN + 1]; //how many strings end at this point
int cnt = 1; //node number, allocate and release space

void build() {
    cnt = 1; //1 as the head node
}

void insert(string word) {
    int cur = 1; //cur initialized to 1, head node

```

```

    pass[cur]++;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            tree[cur][path] = ++cnt;
        }
        cur = tree[cur][path];
        pass[cur]++;
    }
    end[cur]++;
}

int search(string word) {
    int cur = 1;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return end[cur];
}

int countPrefix(string word) {
    int cur = 1;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return pass[cur];
}

void erase(string word) {
    if (search(word) < 1) return;
    int cur = 1;
    //decrement pass along the way
    //decrement end at the endpoint
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (--pass[tree[cur][path]] == 0) {
            //all deleted, clear the node number
            tree[cur][path] = 0;
            return;
        }
        cur = tree[cur][path];
    }
}

```

```

    }
    cur = tree[cur][path];
}
end[cur]--;
}

void clear() {
    for (int i = 1; i <= cnt; i++) {
        fill(tree[i], tree[i] + 26, 0);
        pass[i] = end[i] = 0;
    }
}

```

Persistent Trie

```

#include<iostream>
using namespace std;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    return 0;
}

```

2. Handwritten Heap

Handwritten Reverse-Indexed Heap Optimization

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//by establishing a mapping array f[val]=i
//get the mapping position i of value val in the heap
//the following template code maintains a min-heap, and only performs
operations when the inserted value does not exist or is smaller
//if general updates are needed, siftDown must be performed when the value
becomes larger
const int MAXN = 2e5;
int f[MAXN + 1]; //reverse index table of the heap, the position of point i
in the heap
pair<ll, int> heap[MAXN + 1]; //heap: {value, point}
int siz = 0; //size of the heap

int heapIdx;
ll heapVal;

```

```

//float up
void siftUp(int i) {
    while (i > 0 && heap[i].first < heap[(i - 1) >> 1].first) {
        swap(f[heap[i].second], f[heap[(i - 1) >> 1].second]);
        swap(heap[i], heap[(i - 1) >> 1]);
        i = (i - 1) >> 1;
    }
}

//sink down
void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].first < heap[l].first ? r : l;
        if (heap[best].first >= heap[i].first) {
            return;
        }
        swap(f[heap[best].second], f[heap[i].second]);
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
        r = l + 1;
    }
}

//insert a new value, this function implements insert, update, and ignore
checks at the same time
void HeapInsert(ll val, int idx) {
    if (f[idx] != -1) {
        //already in the heap, perform the update operation
        update(idx, val);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].first = val;
    heap[cur].second = idx;
    f[idx] = cur;
    siftUp(cur);
}

//update the value
void update(int idx, ll val) {
    int i = f[idx]; //get the position of this point in the heap
    if (val < heap[i].first) {
        heap[i].first = val;
        siftUp(i);
    }
}

```

```

}

//pop the top of the heap
void pop() {
    if (siz == 0) heapIdx = -1;
    swap(f[heap[0].second], f[heap[--siz].second]);
    swap(heap[0], heap[siz]);
    heapIdx = heap[siz].second;
    heapVal = heap[siz].first;
    f[heap[siz].second] = -1; //delete
    siftDown(0);
}

```

Handwritten Heap

```

#include<bits/stdc++.h>

using namespace std;
using ll = long long;
const int MAXN = 2e5;

// the handwritten heap can be optimized with reverse indexing
// to implement modification/deletion of non-head data
// see the optimization combined with the PrimMst algorithm

// the advantage of building the heap bottom-up is that the complexity of
offline building goes from  $O(n \log n)$  ->  $O(n)$ 
// the dynamic insertion operation is still  $O(\log n)$ 

// the heap structure is based on a binary tree, as follows
//      0
//    1 2
//  3 4 5 6

class MaxHeap_nlogn {
public:
    //build the heap top-down
    //complexity  $O(n \log n)$ 
    int heap[MAXN + 1];
    int siz = 0; //size of the heap

    //max-heap, dynamic insertion
    void heapInsert(int val) {
        int cur = ++siz - 1; //vertex index = 0, so siz-1 is the current
node
        heap[cur] = val;
        //current node cur, parent (cur-1)>>1
    }

```

```

        while (cur > 0 && heap[cur] > heap[(cur - 1) >> 1]) {
            swap(heap[cur], heap[(cur - 1) >> 1]);
            cur = (cur - 1) >> 1;
        }
    }

    //siftUp and siftDown are used for more complex heap functionality
    void siftDown(int i) { //pass small values down
        int l = 2 * i + 1;
        while (l < siz) {
            //max-heap, swap with the largest of the left and right
            children, and it must be greater than the current node value
            int best = l + 1 < siz && heap[l + 1] > heap[l] ? l + 1 : l;
            if (heap[best] <= heap[i]) return;
            swap(heap[i], heap[best]);
            i = best;
            l = 2 * i + 1;
        }
    }

    void siftUp(int i) { //pass large values up
        while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
            swap(heap[i], heap[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }

    int pop() { //dequeue
        if (siz == 0) return -1;
        //swap position head 0 and the tail, discard the tail
        swap(heap[0], heap[--siz]);
        siftDown(0);
        //siz is now the original discarded position
        return heap[siz];
    }

    int top() { //get the head
        if (siz == 0) return -1;
        return heap[0];
    }
};

class MinHeap_n {
public:
    //min-heap

```

```

//build the heap bottom-up
//complexity O(n)
int heap[MAXN + 1];
int siz = 0;

//read data
void getArr(vector<int>& nums) {
    for (int i = 0; i < nums.size(); i++) {
        heap[i] = nums[i];
    }
    siz = nums.size();
}

void siftDown(int i) {
    int l = 2 * i + 1;
    while (l < siz) {
        int best = l + 1 < siz && heap[l + 1] < heap[l] ? l + 1 : l;
        if (heap[best] >= heap[i]) return;
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
    }
}

//build the heap
void heapInsert() {
    for (int i = (siz - 2) / 2; i >= 0; i--) {
        siftDown(i);
    }
}

int pop() {
    if (siz == 0) return -1;
    //swap position head 0 and the tail, discard the tail
    swap(heap[0], heap[--siz]);
    siftDown(0);
    //siz is now the original discarded position
    return heap[siz];
}

};

```

3. Leftist Heap

K-th Shortest Path

```

#include<bits/stdc++.h>
using namespace std;

```

```

using ll = long long;
using ld = long double;

const int MAXN = 5e3;
const int MAXM = 2e5;
const int MAXT = 2e6;

int headGo[MAXM + 1];
int nxtGo[MAXM + 1];
int toGo[MAXM + 1];
int idxGo = 0;

int headBack[MAXM + 1];
int nxtBack[MAXM + 1];
int toBack[MAXM + 1];
int idxBack = 0;

int path[MAXN + 1];

ld weight[MAXM + 1];

bool vis[MAXN + 1];
ld dist[MAXN + 1];

struct {
    int left;
    int right;
    int dist;
    ld addVal;
    int to;
} node[MAXT + 1];

int cnt = 0;
int top[MAXM + 1];

int create(ld addVal, int to) {
    cnt++;
    node[cnt].addVal = addVal;
    node[cnt].to = to;
    node[cnt].left = node[cnt].right = node[cnt].dist = 0;
    return cnt;
}

int clone(int i) {
    cnt++;
    node[cnt] = node[i];
    return cnt;
}

```

```

}

void addEdgeGo(int u, int v, ld w) {
    nxtGo[++idxGo] = headGo[u];
    headGo[u] = idxGo;
    toGo[idxGo] = v;
    weight[idxGo] = w;
}

void addEdgeBack(int u, int v, ld w) {
    nxtBack[++idxBack] = headBack[u];
    headBack[u] = idxBack;
    toBack[idxBack] = v;
    weight[idxBack] = w;
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    if (node[i].addVal > node[j].addVal) {
        swap(i, j);
    }
    int p = clone(i);
    node[p].right = merge(node[p].right, j);
    if (node[node[p].left].dist < node[node[p].right].dist) {
        swap(node[p].left, node[p].right);
    }
    node[p].dist = node[node[p].right].dist + 1;
    return p;
}

int pop(int i) {
    if (node[i].left == 0 && node[i].right == 0) {
        return 0;
    }
    if (node[i].left == 0 || node[i].right == 0) {
        return clone(node[i].left + node[i].right);
    }
    return merge(node[i].left, node[i].right);
}

// run the shortest path algorithm on the reverse graph
void Djk(int n) {
    priority_queue<pair<ld, int>, vector<pair<ld, int>>, greater<pair<ld,
int>>>q;
    fill(vis, vis + n + 1, false);

```

```

fill(dist, dist + n + 1, 1e18);
dist[n] = 0;
q.push({ 0,n });
while (q.size()) {
    auto [w, u] = q.top();
    q.pop();
    if (!vis[u]) {
        vis[u] = true;
        for (int edge = headBack[u]; edge > 0; edge = nxtBack[edge]) {
            int v = toBack[edge];
            if (dist[v] > dist[u] + weight[edge]) {
                dist[v] = dist[u] + weight[edge];
                path[v] = edge;
                q.push({ dist[v],v });
            }
        }
    }
}

void mergeRoad(int n) {
    priority_queue<pair<ld, int>, vector<pair<ld, int>>, greater<pair<ld,
int>>>q;
    for (int i = 1; i <= n; i++) {
        q.push({ dist[i], i });
    }
    while (q.size()) {
        auto [w, u] = q.top();
        q.pop();
        for (int edge = headGo[u]; edge > 0; edge = nxtGo[edge]) {
            int v = toGo[edge];
            if (edge != path[u]) {
                // non-tree edge
                if (top[u] == 0) {
                    top[u] = create(dist[v] + weight[edge] - dist[u], v);
                }
                else {
                    top[u] = merge(top[u], create(dist[v] + weight[edge] -
dist[u], v));
                }
            }
        }
        if (path[u] != 0) {
            // there is a parent
            int v = toGo[path[u]];
            top[u] = merge(top[u], top[v]);
        }
    }
}

```

```

    }
}

struct heapNode {
    ld addVal;
    int to;
    int heapRoot;
};

int compute(int n, ld money) {
    auto cmp = [](heapNode a, heapNode b) { return a.addVal > b.addVal; };
    priority_queue<heapNode, vector<heapNode>, decltype(cmp)>q(cmp);

    int ans = 0;
    if (money >= dist[1]) {
        ans++;
        money -= dist[1];
    }
    else {
        return ans;
    }

    q.push({ node[top[1]].addVal,node[top[1]].to,top[1] });
    while (q.size()) {
        auto [addVal, to, heapRoot] = q.top();
        q.pop();

        if (dist[1] + addVal <= money) {
            money -= (dist[1] + addVal);
            ans++;
        }
        else {
            return ans;
        }

        // replace the influence, method: directly select the left and
        // right children, do not pop the heap top, otherwise MLE
        // leftist heap satisfies: the key value of each node's children
        // is less than the parent
        int ls = node[heapRoot].left;
        int rs = node[heapRoot].right;
        if (ls != 0) {
            ld newAddVal = addVal - node[heapRoot].addVal +
node[ls].addVal;
            q.push({ newAddVal,node[ls].to,ls });
        }
    }
}

```

```

        if (rs != 0) {
            ld newAddVal = addVal - node[heapRoot].addVal +
node[rs].addVal;
            q.push({ newAddVal,node[rs].to,rs });
        }

        // pop the heap top, copy, will cause !!!MLE!!!
        // int newHead = pop(heapRoot);
        // if (newHead != 0) {
        //     ld newAddVal = addVal - node[heapRoot].addVal +
node[newHead].addVal;
        //     q.push({ newAddVal,node[newHead].to,newHead });
        // }

        // add new influence

        int newHead = top[to];
        if (newHead != 0) {
            ld newAddVal = addVal + node[newHead].addVal;
            q.push({ newAddVal,node[newHead].to,newHead });
        }

    }
    return ans;
}

void solve() {

    // https://www.luogu.com.cn/problem/P2483

    // core: the shortest path tree generated by the djikstra algorithm
    // for non-tree edges, they must be edges connecting two tree nodes
    // for edge (u, v), the extra cost of choosing this edge is dist[v] +
weight(u, v) - dist[u]
    // use a persistent leftist heap, recurse point by point and level by
level

    int n, m;
    ld money;
    cnt = idxGo = idxBack = 0;
    cin >> n >> m >> money;
    node[0].dist = -1;
    fill(top, top + n + 1, 0);
    fill(path, path + n + 1, 0);
    fill(headGo, headGo + n + 1, 0);
    fill(headBack, headBack + n + 1, 0);

```

```

    for (int i = 1; i <= m; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        // stop when reaching the endpoint, so there should not be any
edge connections
        // avoid extra computation
        if (u != n) {
            addEdgeGo(u, v, w);
            addEdgeBack(v, u, w);
        }
    }
    Djk(n);
    mergeRoad(n);
    cout << compute(n, money) << '\n';
}

```

Persistent Leftist Heap

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;
const int MAXT = 4e5;
int cnt = 0;
int version[MAXN + 1];
int verSize[MAXN + 1];
struct leftTree {
    ll key;
    int left;
    int right;
    int dist;
    int size;
} node[MAXT + 1];

void cleanNode(int i) {
    node[i].key = 0;
    node[i].left = node[i].right = node[i].dist = 0;
    node[i].size = 1;
}

void clear(int n) {
    for (int i = 0; i <= cnt; i++) {
        cleanNode(i);
    }
}

```

```

    fill(version, version + n + 1, 0);
    fill(verSize, verSize + n + 1, 0);
    node[0].dist = -1;
    node[0].size = 0;
    cnt = 0;
}

int clone(int i) {
    int j = ++cnt;
    node[j] = node[i];
    return j;
}

int create(ll val) {
    int i = ++cnt;
    node[i].key = val;
    node[i].left = node[i].right = node[i].dist = 0;
    node[i].size = 1;
    return i;
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    if (node[i].key > node[j].key) {
        swap(i, j);
    }
    int p = clone(i);
    node[p].right = merge(node[p].right, j);
    if (node[node[p].left].dist < node[node[p].right].dist) {
        swap(node[p].left, node[p].right);
    }
    node[p].dist = node[node[p].right].dist + 1;
    node[p].size = node[node[p].left].size + node[node[p].right].size + 1;
    return p;
}

int pop(int i) {
    if (node[i].left == 0 && node[i].right == 0) {
        return 0;
    }
    if (node[i].left == 0 || node[i].right == 0) {
        return clone(node[i].left + node[i].right);
    }
    return merge(node[i].left, node[i].right);
}

```

```

// insert v based on version x
void treeAdd(int x, ll v, int i) {
    if (version[x] == 0) {
        version[i] = create(v);
        verSize[i] = 1;
    }
    else {
        version[i] = merge(version[x], create(v));
        verSize[i] = verSize[x] + 1;
    }
}

// merge based on versions x and y
void treeMerge(int x, int y, int i) {
    int v1 = version[x];
    int v2 = version[y];
    if (v1 == 0 && v2 == 0) {
        version[i] = 0;
        verSize[i] = 0;
        return;
    }
    if (v1 == 0 || v2 == 0) {
        version[i] = clone(v1 + v2);
        verSize[i] = verSize[x] + verSize[y];
        return;
    }
    version[i] = merge(v1, v2);
    verSize[i] = verSize[x] + verSize[y];
}

// pop the head based on version x
void treePop(int x, int i) {
    if (version[x] != 0) {
        version[i] = pop(version[x]);
        verSize[i] = verSize[x] - 1;
    }
    else {
        version[i] = 0;
        verSize[i] = 0;
    }
}

```

Leftist Heap

```

#include<bits/stdc++.h>
using namespace std;

```

```

using ll = long long;
const int MAXN = 2e5;

// Leftist heap: a mergeable heap structure
// heap merge time complexity O(logn)
// the leftist heap relies on the dist value to maintain balance
// dist[i]: the minimum number of nodes passed from i to reach an empty
node
// each node of the leftist heap satisfies left child's dist >= right
child's dist
// node dist = right child dist + 1

struct tree {
    ll val;
    int left, right;
    int dist;
    int father;// auxiliary for the remove deletion operation
}node[MAXN + 1];

int fa[MAXN + 1];

void prepare(int n) {
    for (int i = 0; i <= n; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
        fa[i] = i;
    }
    node[0].dist = -1;
}

int find(int u) {
    return u == fa[u] ? u : fa[u] = find(fa[u]);
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    // maintain a min-heap
    // make i the one with the smaller value, if values are equal make it
the one with the smaller index
    if (node[i].val > node[j].val || (node[i].val == node[j].val && i >
j)) {
        swap(i, j);
    }
    node[i].right = merge(node[i].right, j);
    node[node[i].right].father = i;
}

```

```

        if (node[node[i].left].dist < node[node[i].right].dist) {
            swap(node[i].left, node[i].right);
        }
        node[i].dist = node[node[i].right].dist + 1;
        fa[node[i].left] = i;
        fa[node[i].right] = i;
        return i;
    }

    // return the head node of the new tree
    int pop(int i) {
        fa[node[i].left] = node[i].left;
        fa[node[i].right] = node[i].right;
        int newHead = merge(node[i].left, node[i].right);
        fa[i] = newHead;
        node[i].left = node[i].right = node[i].dist = node[i].father = 0;
        return newHead;
    }

    int remove(int i) {
        int head = find(i);
        int f = node[i].father;
        int s = pop(i);
        node[s].father = f;

        if (head != i) {
            fa[s] = head;
            if (node[f].left == i) {
                node[f].left = s;
            }
            else {
                node[f].right = s;
            }
            for (int d = node[s].dist; node[f].dist > d + 1; f = node[f].father,
d++) {
                node[f].dist = d + 1;
                if (node[node[f].left].dist < node[node[f].right].dist) {
                    swap(node[f].left, node[f].right);
                }
            }
        }
        // if i is the head, then = 0, fa[s] = 0
        // if i is not the head, then s exists, update
        return fa[s];
    }

    // adjust i to to

```

```

void update(int i, ll to) {
    int h = remove(i);
    node[i].val = to;
    fa[i] = i;
    node[i].dist = 0;
    fa[h] = fa[i] = merge(h, i);
}

```

4. Disjoint Set Union (DSU)

Weighted DSU

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
// https://www.luogu.com.cn/problem/P8779

const int MAXN = 2e5;
int fa[MAXN + 1];
ll dist[MAXN + 1];

void init(int n) {
    for (int i = 0; i <= n; i++) {
        fa[i] = i;
        dist[i] = 0;
    }
}

int find(int u) {
    if (u != fa[u]) {
        int tmp = fa[u];
        fa[u] = find(tmp);
        dist[u] += dist[tmp];
    }
    return fa[u];
}

void merge(int u, int v, ll d) {
    int ru = find(u);
    int rv = find(v);
    if (ru != rv) {
        fa[ru] = rv;
        // u -> ru
        // v -> rv
        // transformed into: ru -> u -> v -> rv
        dist[ru] = d + dist[v] - dist[u];
    }
}

```

```

}

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    init(n + 1);
    for (int i = 1; i <= m; i++) {
        int l, r; ll d;
        cin >> l >> r >> d;
        r = r + 1;
        merge(l, r, d);
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        r = r + 1;
        if (find(l) != find(r)) {
            cout << "UNKNOWN" << '\n';
        }
        else {
            cout << dist[l] - dist[r] << '\n';
        }
    }
}
}

```

Disjoint Set Union

```

#include<iostream>
using namespace std;
const int MAXN = 2e5;

//union-find
int father[MAXN + 1];
int siz[MAXN + 1];
int road[MAXN + 1]; //auxiliary for path optimization

//initialize
void initial(int m) {
    for (int i = 1; i <= m; i++) {
        father[i] = i;
        siz[i] = 1;
    }
}

//find the root node
int find(int u) {
    //iterative + flattening initialization

```

```

    //i.e. has the path optimization of recursion, and avoids overly deep
    recursion
    int cnt = -1;
    while (father[u] != u) {
        road[++cnt] = u;
        u = father[u];
    }
    //path optimization
    while (cnt >= 0) father[road[cnt--]] = u;
    return u;
}

//merge by rank, avoid degradation
void merge(int u, int v) {
    int ru = find(u);
    int rv = find(v);
    if (ru == rv) return;
    if (siz[ru] >= siz[rv]) {
        siz[ru] += siz[rv];
        father[rv] = ru;
    }
    else {
        siz[rv] += siz[ru];
        father[ru] = rv;
    }
}
}

```

5. Ordered Map Topics

AVL Balanced Tree

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// AVL: balanced binary search tree
// the height difference between the left and right subtrees of each node
is <= 1

// ordered map:
// can implement: insert, delete, modify, query
// single operation time complexity O(logn)
// each key maps to a unique value
// organized by the order of keys, keys cannot be duplicated (unique)

// https://www.luogu.com.cn/problem/P3369

const int MAXN = 2e5;

```

```

const ll INF = 1e18;
int head = 0;
int cnt = 0;
// can use a use table to maintain available memory numbers, avoiding
memory waste
// canUse[0] represents the number of available nodes
// int canUse[MAXN + 1];
struct {
    ll key;// key
    int height;// tree height
    int left;// left child
    int right;// right child
    int cnt;// key frequency
    int siz;// subtree size
}node[MAXN + 1];

void clear() {
    for (int i = 1;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0 };
    }
    cnt = 0;
    head = 0;
}

// aggregate information
void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].siz = node[l].siz + node[r].siz + node[i].cnt;
    node[i].height = max(node[l].height, node[r].height) + 1;
}

// left rotation, the parent node sinks to the left
// the left child of the right child becomes the right child of the parent
// the parent becomes the left child of the right child
int leftRotate(int i) {
    int r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;
    up(i);
    up(r);
    return r;
}

// right rotation
int rightRotate(int i) {
    int l = node[i].left;
    node[i].left = node[l].right;

```

```

    node[l].right = i;
    up(i);
    up(l);
    return l;
}

// check whether the subtree violates the rule and adjust
int maintain(int i) {
    int l = node[i].left, r = node[i].right;
    int lh = node[l].height;
    int rh = node[r].height;
    if (lh - rh > 1) {
        // LL
        if (node[node[l].left].height >= node[node[l].right].height) {
            i = rightRotate(i);
        }
        else {
            // LR
            node[i].left = leftRotate(l);
            i = rightRotate(i);
        }
    }
    else if (rh - lh > 1) {
        // RR
        if (node[node[r].right].height >= node[node[r].left].height) {
            i = leftRotate(i);
        }
        else {
            // RL
            node[i].right = rightRotate(r);
            i = leftRotate(i);
        }
    }
    return i;
}

// query the number of keys < val
int small(int i, int val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {

```

```

        return node[node[i].left].siz + node[i].cnt + small(node[i].right,
val);
    }
}

// query the rank
int rank(int val) {
    return small(head, val) + 1;
}

// query the k-th smallest number on subtree i
ll index(int i, int k) {
    int l = node[i].left;
    if (node[l].siz >= k) {
        return index(l, k);
    }
    else if (node[l].siz + node[i].cnt < k) {
        return index(node[i].right, k - node[l].siz - node[i].cnt);
    }
    return node[i].key;
}

ll index(int k) {
    return index(head, k);
}

// find the predecessor of val on subtree i (the maximum < val)
ll pre(int i, int val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll pre(int val) {
    return pre(head, val);
}

ll suf(int i, ll val) {
    if (i == 0) {
        return INF;
    }

```

```

    }
    if (node[i].key <= val) {
        return suf(node[i].right, val);
    }
    else {
        return min(node[i].key, suf(node[i].left, val));
    }
}

// suf(int val) {
//     return suf(head, val);
// }

// add value val to subtree i
int add(int i, int val) {
    if (i == 0) {
        node[++cnt].key = val;
        node[cnt].cnt = node[cnt].siz = node[cnt].height = 1;
        return cnt;
    }
    if (node[i].key == val) {
        node[i].cnt++;
    }
    else if (node[i].key > val) {
        node[i].left = add(node[i].left, val);
    }
    else {
        node[i].right = add(node[i].right, val);
    }
    up(i);
    return maintain(i);
}

// add value val to the whole tree, return the new head
void add(int val) {
    head = add(head, val);
}

// delete the low node on subtree i
int removeLow(int i, int low) {
    if (i == low) {
        return node[i].right;
    }
    else {
        node[i].left = removeLow(node[i].left, low);
        up(i);
        return maintain(i);
    }
}

```

```

    }
}

// delete val on subtree i
int remove(int i, int val) {
    if (node[i].key < val) {
        node[i].right = remove(node[i].right, val);
    }
    else if (node[i].key > val) {
        node[i].left = remove(node[i].left, val);
    }
    else {
        if (node[i].cnt > 1) {
            node[i].cnt--;
        }
        else {
            if (node[i].left == 0 && node[i].right == 0) {
                return 0;
            }
            else if (node[i].left == 0 && node[i].right != 0) {
                i = node[i].right;
            }
            else if (node[i].left != 0 && node[i].right == 0) {
                i = node[i].left;
            }
            else {
                int low = node[i].right;
                while (node[low].left != 0) {
                    low = node[low].left;
                }
                node[i].right = removeLow(node[i].right, low);
                node[low].left = node[i].left;
                node[low].right = node[i].right;
                i = low;
            }
        }
    }
    up(i);
    return maintain(i);
}

// delete key from the whole tree once
void remove(int val) {
    if (::rank(val) != ::rank(val + 1)) {
        head = remove(head, val);
    }
}
}

```

FHQ Persistent Treap

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 5e5;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

int version[MAXN + 1];
int cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size;
}node[MAXN * 50 + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0,0,0,0,0 };
    }
    cnt = 0;
}

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int create(int val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].priority = dist(gen);
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

void split(int l, int r, int i, ll val) {
    if (i == 0) {
```

```

        node[l].right = node[r].left = 0;
    }
    else {
        i = clone(i);
        if (node[i].key <= val) {
            node[l].right = i;
            split(i, r, node[i].right, val);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, val);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        l = clone(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        r = clone(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

int remove(int v, ll val) {
    // based on version v, delete one val
    int head = version[v];
    split(0, 0, head, val);
    //   <=val               >val
    int l = node[0].right, r = node[0].left;
    node[0].left = node[0].right = 0;
    split(0, 0, l, val - 1);
    //   <val               =val
    int lm = node[0].right, rm = node[0].left;
    node[0].left = node[0].right = 0;
    int ans = merge(merge(lm, merge(node[rm].left, node[rm].right)), r);
    node[0].left = node[0].right = 0;
}

```

```

    return ans;
}

int add(int v, ll val) {
    int head = version[v];
    split(0, 0, head, val);
    //   <=val               >val
    int l = node[0].right, r = node[0].left;
    node[0].left = node[0].right = 0;
    int ans = merge(merge(l, create(val)), r);
    return ans;
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + 1 + small(node[i].right, val);
    }
}

int rank(int v, ll val) {
    return small(version[v], val) + 1;
}

ll index(int i, int rk) {
    if (i == 0) {
        return 0;
    }
    if (node[node[i].left].size >= rk) {
        return index(node[i].left, rk);
    }
    else if (node[node[i].left].size + 1 < rk) {
        return index(node[i].right, rk - node[node[i].left].size - 1);
    }
    return node[i].key;
}

ll pre(int i, ll val) {
    if (i == 0) {
        return INT_MIN + 1;
    }
    if (node[i].key >= val) {

```

```

        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll post(int i, ll val) {
    if (i == 0) {
        return INT_MAX;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

void solve() {
    int n;
    cin >> n;
    clear();
    for (int i = 1; i <= n; i++) {
        int v, op;
        ll val;
        cin >> v >> op >> val;
        if (op == 1) {
            version[i] = add(v, val);
        }
        else if (op == 2) {
            version[i] = remove(v, val);
        }
        else {
            version[i] = version[v];
            if (op == 3) {
                cout << ::rank(i, val) << '\n';
            }
            else if (op == 4) {
                cout << index(version[i], val) << '\n';
            }
            else if (op == 5) {
                cout << pre(version[i], val) << '\n';
            }
            else {
                cout << post(version[i], val) << '\n';
            }
        }
    }
}

```

```

    }
}
}

```

Splay Tree

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5;
const ll INF = 1e18;

// in a splay tree, the splay-to-root operation must be performed in every
// insert, delete, modify, and query operation
// because the splay-to-root operation can shorten the height of the tree

// among the methods below, only find does not contain the splay-to-root
// operation
// because remove relies on consecutive find assistance, once splay-to-
// root changes the tree structure, the remove operation becomes invalid

int head = 0, cnt = 0;
struct {
    ll key;
    int left;
    int right;
    int father;
    int size;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0,0,0,0,0 };
    }
    head = cnt = 0;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

```

```

int side(int i) {
    return node[node[i].father].left == i ? 1 : 2;
}

// while performing left/right rotation, maintain the correct parent-child
// relationship
// node 0 has no parent
int leftRotate(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;

    node[r].father = node[i].father;
    node[i].father = r;
    if (node[i].right != 0) {
        // node 0 has no parent
        node[node[i].right].father = i;
    }
    return r;
}

int rightRotate(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].left = node[l].right;
    node[l].right = i;

    node[l].father = node[i].father;
    node[i].father = l;
    if (node[i].left != 0) {
        node[node[i].left].father = i;
    }
    return l;
}

// let i rise one step
void rotate(int i) {
    // father grandfather
    int fa = node[i].father, gfa = node[fa].father;
    // no parent, the root node does not perform the operation
    if (fa != 0) {
        int s1 = side(i), s2 = side(fa);
        if (s1 == 1) {
            rightRotate(fa);
        }
        else {

```

```

        leftRotate(fa);
    }
    // there is a grandfather, update
    if (gfa != 0) {
        if (s2 == 1) {
            node[gfa].left = i;
        }
        else {
            node[gfa].right = i;
        }
    }
    else { // no grandfather, now i becomes the root
        head = i;
    }
    up(fa);
    up(i);
}
}

// let i become the child of goal
void splay(int i, int goal) {
    int fa = node[i].father, gfa = node[fa].father;
    while (fa != goal) {
        // if the grandfather is the target point, i rises one step
        if (gfa != goal) {
            // zig-zig
            if (side(i) == side(fa)) {
                rotate(fa);
            }
            else { // zig-zag
                rotate(i);
            }
        }
        rotate(i);
        fa = node[i].father;
        gfa = node[fa].father;
    }
    if (goal == 0) {
        // i becomes the root node
        head = i;
    }
}

// query the node with rank k
int find(int k) {
    int i = head;
    while (i != 0) {

```

```

        // satisfied
        if (node[node[i].left].size + 1 == k) {
            return i;
        }
        else if (node[node[i].left].size >= k) {
            // located on the left
            i = node[i].left;
        }
        else { // located on the right
            k -= (node[node[i].left].size + 1);
            i = node[i].right;
        }
    }
    return 0; // does not exist
}

int small(ll val) {
    int i = head, last = head;
    int ans = 0;
    while (i != 0) {
        last = i;
        if (node[i].key >= val) {
            i = node[i].left;
        }
        else {
            ans += node[node[i].left].size + 1;
            i = node[i].right;
        }
    }
    splay(last, 0);
    return ans;
}

int rank(ll val) {
    return small(val) + 1;
}

void add(ll val) {
    int i = create(val);
    if (head == 0) {
        head = i;
    }
    else {
        int fa = 0, u = head, side = 0;
        while (u != 0) {
            // left <= middle <= right
            if (node[u].key <= val) {

```

```

        side = 2;
        fa = u;
        u = node[u].right;
    }
    else {
        side = 1;
        fa = u;
        u = node[u].left;
    }
}
if (side == 1) {
    node[fa].left = i;
}
else {
    node[fa].right = i;
}
node[i].father = fa;
splay(i, 0);
}
}

void remove(ll val) {
    int kth = ::rank(val);
    if (kth != ::rank(val + 1)) {
        int i = find(kth);
        splay(i, 0);
        if (node[i].left == 0) {
            head = node[i].right;
        }
        else if (node[i].right == 0) {
            head = node[i].left;
        }
        else {
            // i has rank kth, kth+1 must be in i's right subtree
            // splay j to be the right child of i, at this point j must
            have no left child
            // (because kth is the in-order traversal order, so j as rank
            kth+1 must have no left child)
            // give i's left child to j, let j become the head
            int j = find(kth + 1);
            splay(j, i);
            node[j].left = node[i].left;
            node[node[j].left].father = j;
            up(j);
            head = j;
        }
        node[head].father = 0;
    }
}

```

```

    }
}

ll index(int x) {
    int i = find(x);
    splay(i, 0);
    return node[i].key;
}

ll pre(ll val) {
    int i = head, last = head;
    ll ans = -INF;
    while (i != 0) {
        last = i;
        if (node[i].key >= val) {
            i = node[i].left;
        }
        else {
            ans = max(ans, node[i].key);
            i = node[i].right;
        }
    }
    splay(last, 0);
    return ans;
}

ll post(ll val) {
    int i = head, last = head;
    ll ans = INF;
    while (i != 0) {
        last = i;
        if (node[i].key <= val) {
            i = node[i].right;
        }
        else {
            ans = min(ans, node[i].key);
            i = node[i].left;
        }
    }
    splay(last, 0);
    return ans;
}

```

Treap FHQ (Non-Frequency-Compressed)

```

#include<bits/stdc++.h>
using namespace std;

```

```

using ll = long long;

// FHQTreap
// core: split according to the key value, merge according to priority

// merge(l,r): after merging, the in-order traversal order of the tree is
first the l tree, then the r tree
// therefore, the FHQ tree can cleverly maintain a specific order of in-
order traversal
// and at the same time, because of the priority setting, the tree height
is also maintained

// if you only care about a specific order, the combination of merge and
split is more flexible, at this point the so-called rank is the in-order
traversal order
// figuratively, if the in-order traversal result is placed in a 1D array,
then rank is the index, i.e. what is maintained is the index

// if you care about the size relationship of key values, then for the
merge operation on the left/right subtrees after split, the order cannot
be changed, rank not only represents the index, but also the k-th smallest
// otherwise rank loses the comparison function about element size, and
can only represent the traversal order, i.e. only maintains the index

const int MAXN = 2e5;
const ll INF = 1e18;
int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size;// total number of nodes
}node[MAXN + 1];

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0 };
    }
    head = cnt = 0;
}

void up(int i) {

```

```

    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

// if the current curv <= key, then the left subtree needs no change,
// recurse to the right subtree
// conversely, the right subtree needs no change, recurse to the left
// subtree

// split by key value
void splitByKey(int l, int r, int i, ll key) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[i].key <= key) {
            node[l].right = i;
            splitByKey(i, r, node[i].right, key);
        }
        else {
            node[r].left = i;
            splitByKey(l, i, node[i].left, key);
        }
        up(i);
    }
}

// classic non-frequency-compressed, can be used normally
void splitByRank(int l, int r, int i, ll rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            splitByRank(i, r, node[i].right, rk - node[node[i].left].size
- 1);
        }
        else {
            node[r].left = i;
            splitByRank(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {

```

```

        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

void add(ll val) {
    splitByKey(0, 0, head, val);
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].priority = dist(gen);
    head = merge(merge(node[0].right, cnt), node[0].left);
}

void remove(ll val) {
    splitByKey(0, 0, head, val);
    int lm = node[0].right; // <=val
    int r = node[0].left; // >val
    splitByKey(0, 0, lm, val - 1);
    int l = node[0].right; // <=val-1
    int m = node[0].left; // =val
    // remove one (the head) from =val, keep all of the val-1 part
    and >val part
    head = merge(merge(l, merge(node[m].left, node[m].right)), r);
}

int small(ll val) {
    splitByKey(0, 0, head, val - 1);
    int ans = node[node[0].right].size;
    head = merge(node[0].right, node[0].left);
    return ans;
}

int rank(ll val) {
    return small(val) + 1;
}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {

```

```

        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + 1 < x) {
        return index(node[i].right, x - node[node[i].left].size - 1);
    }
    return node[i].key;
}

ll index(int x) {
    return index(head, x);
}

ll pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll pre(ll val) {
    return pre(head, val);
}

ll post(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

ll post(ll val) {
    return post(head, val);
}

```

Treap FHQ (Frequency-Compressed)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// FHQTreap
// core: split according to the key value, merge according to priority

const int MAXN = 2e5;
const ll INF = 1e18;
int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size;// total element count (sum of frequencies)
    int count;
}node[MAXN + 1];

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0 };
    }
    head = cnt = 0;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size +
node[i].count;
}

// if the current curv <= key, then the left subtree needs no change,
recurve to the right subtree
// conversely, the right subtree needs no change, recurse to the left
subtree

// split by key value
void splitByKey(int l, int r, int i, ll key) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
}
```

```

    else {
        if (node[i].key <= key) {
            node[l].right = i;
            splitByKey(i, r, node[i].right, key);
        }
        else {
            node[r].left = i;
            splitByKey(l, i, node[i].left, key);
        }
        up(i);
    }
}

// split by rank
// this rank split can only handle the case where each position has count
= 1
// if duplicate counts exist, it needs customization
void splitByRank(int l, int r, int i, ll rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            splitByRank(i, r, node[i].right, rk - node[node[i].left].size
- 1);
        }
        else {
            node[r].left = i;
            splitByRank(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        node[r].left = merge(l, node[r].left);
        up(r);
    }
}

```

```

        return r;
    }
}

int find(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key == val) {
        return i;
    }
    else if (node[i].key < val) {
        return find(node[i].right, val);
    }
    else {
        return find(node[i].left, val);
    }
}

void changeCount(int i, ll val, int change) {
    if (node[i].key == val) {
        node[i].count += change;
    }
    else if (node[i].key > val) {
        changeCount(node[i].left, val, change);
    }
    else {
        changeCount(node[i].right, val, change);
    }
    up(i);
}

void add(ll val) {
    if (find(head, val) != 0) {
        changeCount(head, val, 1);
    }
    else {
        splitByKey(0, 0, head, val);
        node[++cnt].key = val;
        node[cnt].count = 1;
        node[cnt].size = 1;
        node[cnt].priority = dist(gen);
        head = merge(merge(node[0].right, cnt), node[0].left);
    }
}

void remove(ll val) {

```

```

int i = find(head, val);
if (i != 0) {
    if (node[i].count > 1) {
        changeCount(head, val, -1);
    }
    else {
        splitByKey(0, 0, head, val);
        int lm = node[0].right;
        int r = node[0].left;
        splitByKey(0, 0, lm, val - 1);
        int l = node[0].right;
        head = merge(l, r);
    }
}
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + node[i].count < x) {
        return index(node[i].right, x - node[node[i].left].size -
node[i].count);
    }
    return node[i].key;
}

ll index(int x) {
    return index(head, x);
}

```

```

ll pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll pre(ll val) {
    return pre(head, val);
}

ll post(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

ll post(ll val) {
    return post(head, val);
}

```

Treap Rotation Tree

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

// Treap tree:
// each node stores {key, priority}
// maintain the binary search tree order according to the key value
(left<middle<right)
// then maintain the heap structure (max/min-heap) according to priority
// priority is generated with the random function, guaranteeing the
balance of the Treap

```

```

const int MAXN = 2e5;
const ll INF = 1e18;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(0, INT_MAX);

int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left, right;
    int size;
    int count;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0, 0 };
    }
    cnt = head = 0;
}

void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].size = node[l].size + node[r].size + node[i].count;
}

int rightRotate(int i) {
    int l = node[i].left;
    node[i].left = node[l].right;
    node[l].right = i;
    up(i);
    up(l);
    return l;
}

int leftRotate(int i) {
    int r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;
    up(i);
    up(r);
    return r;
}

```

```

int add(int i, ll val) {
    if (i == 0) {
        node[++cnt].key = val;
        node[cnt].count = node[cnt].size = 1;
        node[cnt].priority = dist(gen);
        return cnt;
    }
    if (node[i].key == val) {
        node[i].count++;
    }
    else if (node[i].key > val) {
        node[i].left = add(node[i].left, val);
    }
    else {
        node[i].right = add(node[i].right, val);
    }
    up(i);
    if (node[i].left != 0 && node[node[i].left].priority >
node[i].priority) {
        return rightRotate(i);
    }
    if (node[i].right != 0 && node[node[i].right].priority >
node[i].priority) {
        return leftRotate(i);
    }
    return i;
}

void add(ll val) {
    head = add(head, val);
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

```

```

}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + node[i].count < x) {
        return index(node[i].right, x - node[node[i].left].size -
node[i].count);
    }
    return node[i].key;
}

ll index(int x) {
    return index(head, x);
}

ll pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(pre(node[i].right, val), node[i].key);
    }
}

ll pre(ll val) {
    return pre(head, val);
}

ll suf(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return suf(node[i].right, val);
    }
    else {
        return min(suf(node[i].left, val), node[i].key);
    }
}

ll suf(ll val) {

```

```

    return suf(head, val);
}

int remove(int i, ll val) {
    if (node[i].key < val) {
        node[i].right = remove(node[i].right, val);
    }
    else if (node[i].key > val) {
        node[i].left = remove(node[i].left, val);
    }
    else {
        if (node[i].count > 1) {
            node[i].count--;
        }
        else {
            if (node[i].left == 0 && node[i].right == 0) {
                return 0;
            }
            else if (node[i].left == 0 && node[i].right != 0) {
                return node[i].right;
            }
            else if (node[i].right == 0 && node[i].left != 0) {
                return node[i].left;
            }
            else {
                if (node[node[i].left].priority >=
node[node[i].right].priority) {
                    i = rightRotate(i);
                    node[i].right = remove(node[i].right, val);
                }
                else {
                    i = leftRotate(i);
                    node[i].left = remove(node[i].left, val);
                }
            }
        }
    }
    up(i);
    return i;
}

void remove(ll val) {
    if (::rank(val) != ::rank(val + 1)) {
        head = remove(head, val);
    }
}

```

Persistent Literary Balanced Tree

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

const int MAXN = 2e5;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

int cnt = 0;
int version[MAXN + 1];
struct {
    ll key;
    ll sum;
    int left;
    int right;
    int size;
    int priority;
    bool isRev;
}node[MAXN * 50 + 1];

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0,0 };
    }
    cnt = 0;
}

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].sum = val;
    node[cnt].priority = dist(gen);
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
```

```

    node[i].sum = node[node[i].left].sum + node[node[i].right].sum +
node[i].key;
}

void down(int i) {
    if (node[i].isRev) {
        // if it exists, then copy
        if (node[i].left != 0) {
            node[i].left = clone(node[i].left);
            node[node[i].left].isRev ^= 1;
        }
        if (node[i].right != 0) {
            node[i].right = clone(node[i].right);
            node[node[i].right].isRev ^= 1;
        }
        // perform lazy update on the copy
        swap(node[i].left, node[i].right);
        node[i].isRev = false;
    }
}

void split(int l, int r, int i, int rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        i = clone(i);
        down(i);
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            split(i, r, node[i].right, rk - node[node[i].left].size - 1);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        l = clone(l);

```

```

        down(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        r = clone(r);
        down(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

// based on version v, reverse the interval [jobl, jobr]
int reverse(int v, int jobl, int jobr) {
    // the head of the new version
    int head = version[v];

    split(0, 0, head, jobr);
    //   <=jobr           >jobr
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    split(0, 0, l, jobl - 1);
    //   <jobl           [jobl, jobr]
    int lm = node[0].right, rm = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    int ans;
    node[rm].isRev ^= 1;
    lm = merge(lm, rm);
    ans = merge(lm, r);

    return ans;
}

// based on version v, delete the x-th element
int remove(int v, int x) {
    int head = version[v];

    split(0, 0, head, x);
    //   <=x           >x
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    split(0, 0, l, x - 1);

```

```

// <x                =x
int lm = node[0].right, rm = node[0].left;
node[0] = { 0,0,0,0,0,0,0,0 };

lm = merge(lm, merge(node[rm].left, node[rm].right));
int ans = merge(lm, r);

return ans;
}

// based on version v, insert val after the x-th element
int insert(int v, int x, ll val) {
    int head = version[v];

    split(0, 0, head, x);
    // <=x                >x
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    l = merge(l, create(val));
    int ans = merge(l, r);

    return ans;
}

// based on version v, query the sum of [l,r]
int query(int v, int jobl, int jobr, ll& sum) {
    int head = version[v];

    split(0, 0, head, jobr);
    // <=jobr                >jobr
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    split(0, 0, l, jobl - 1);
    // <jobl                [jobl,jobr]
    int lm = node[0].right, rm = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    sum = node[rm].sum;
    lm = merge(lm, rm);
    int ans = merge(lm, r);
    return ans;
}

void solve() {
    int n;

```

```

cin >> n;
ll lastAns = 0;
clear();
for (int i = 1; i <= n; i++) {
    int v, op;
    cin >> v >> op;
    if (op == 1) {
        ll p, x;
        cin >> p >> x;
        p ^= lastAns, x ^= lastAns;
        version[i] = insert(v, p, x);
    }
    else if (op == 2) {
        ll p;
        cin >> p;
        p ^= lastAns;
        version[i] = remove(v, p);
    }
    else if (op == 3) {
        ll l, r;
        cin >> l >> r;
        l ^= lastAns, r ^= lastAns;
        version[i] = reverse(v, l, r);
    }
    else {
        ll l, r;
        cin >> l >> r;
        l ^= lastAns, r ^= lastAns;
        ll sum;
        version[i] = query(v, l, r, sum);
        cout << sum << '\n';
        lastAns = sum;
    }
}
}

```

Leftist Heap

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e5;

// Leftist heap: a mergeable heap structure
// heap merge time complexity O(logn)
// the leftist heap relies on the dist value to maintain balance

```

```

// dist[i]: the minimum number of nodes passed from i to reach an empty
node
// each node of the leftist heap satisfies left child's dist >= right
child's dist
// node dist = right child dist + 1

struct tree {
    ll val;
    int left, right;
    int dist;
    int father;
}node[MAXN + 1];

int fa[MAXN + 1];

void prepare(int n) {
    for (int i = 0; i <= n; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
        fa[i] = i;
    }
    node[0].dist = -1;
}

int find(int u) {
    return u == fa[u] ? u : fa[u] = find(fa[u]);
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    // maintain a min-heap
    // make i the one with the smaller value, if values are equal make it
the one with the smaller index
    if (node[i].val > node[j].val || (node[i].val == node[j].val && i >
j)) {
        swap(i, j);
    }
    node[i].right = merge(node[i].right, j);
    node[node[i].right].father = i;
    if (node[node[i].left].dist < node[node[i].right].dist) {
        swap(node[i].left, node[i].right);
    }
    node[i].dist = node[node[i].right].dist + 1;
    fa[node[i].left] = i;
    fa[node[i].right] = i;
    return i;
}

```

```

}

// return the head node of the new tree
int pop(int i) {
    fa[node[i].left] = node[i].left;
    fa[node[i].right] = node[i].right;
    fa[i] = merge(node[i].left, node[i].right);
    node[i].left = node[i].right = node[i].dist = node[i].father = 0;
    return fa[i];
}

int remove(int i) {
    int head = find(i);
    int f = node[i].father;
    int s = pop(i);
    node[s].father = f;
    if (head != i) {
        fa[s] = head;
        if (node[f].left == i) {
            node[f].left = s;
        }
        else {
            node[f].right = s;
        }
        for (int d = node[s].dist; node[f].dist > d + 1; f = node[f].father,
d++) {
            node[f].dist = d + 1;
            if (node[node[f].left].dist < node[node[f].right].dist) {
                swap(node[f].left, node[f].right);
            }
        }
    }
    return fa[s];
}

// adjust i to to
void update(int i, ll to) {
    int h = remove(i);
    node[i].val = to;
    fa[i] = i;
    node[i].dist = 0;
    fa[h] = fa[i] = merge(h, i);
}

```

Literary Balanced Tree (FHQ Lazy Tag)

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 1e5;
int head = 0, cnt = 0;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

struct {
    int key;
    int priority;
    int left;
    int right;
    int size;
    bool isRev;
}node[MAXN + 1];

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0 };
    }
}

int create(int val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].isRev = false;
    node[cnt].priority = dist(gen);
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

void down(int i) {
    if (node[i].isRev) {
        swap(node[i].left, node[i].right);
        node[node[i].left].isRev ^= 1;
        node[node[i].right].isRev ^= 1;
        node[i].isRev = false;
    }
}
```

```

void split(int l, int r, int i, int rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        down(i);
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            split(i, r, node[i].right, rk - node[node[i].left].size - 1);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        down(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        down(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

void printTree(int i) {
    if (i != 0) {
        down(i);
        printTree(node[i].left);
        cout << node[i].key << ' ';
        printTree(node[i].right);
    }
}

void reverse(int jobl, int jobr) {

```

```

    split(0, 0, head, jobr);
    //    <=jobr                >jobr
    int l = node[0].right, r = node[0].left;
    split(0, 0, l, jobl - 1);
    //    <jobl                jobl<=&&<=jobr
    int lm = node[0].right, rm = node[0].left;
    node[rm].isRev ^= 1;
    l = merge(lm, rm);
    head = merge(l, r);
}

void solve() {
    int n, m;
    cin >> n >> m;
    clear();
    for (int i = 1; i <= n; i++) {
        head = merge(head, create(i));
    }
    while (m--) {
        int l, r;
        cin >> l >> r;
        reverse(l, r);
    }
    printTree(head);
}

```

Scapegoat Tree (Frequency-Compressed)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// Scapegoat tree:
// balance factor Alpha, the optimal value is 0.7
// when the number of nodes in the left or right subtree > n * Alpha
// rebuild the entire subtree
// if multiple violating subtrees exist, choose the subtree at the highest
node

const int MAXN = 2e5;
const double Alpha = 0.7; // balance factor
const ll INF = 1e18;
int head = 0, cnt = 0;
int collect[MAXN + 1]; // collect nodes during in-order traversal
// the highest unbalanced point, top's parent, the relationship between
top and its parent
int top, father, side;

```

```

struct {
    ll key;// key value
    int count;// current key, frequency
    int left;// left child
    int right;// right child
    int diff;// maintain the number of different keys on the subtree (i.e.
node count), different
    int size;// the number of collected elements (sum of frequencies)
}node[MAXN + 1];

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].count = 1;
    node[cnt].left = node[cnt].right = 0;
    node[cnt].diff = 1;
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].size = node[l].size + node[r].size + node[i].count;
    // because of the lazy deletion mechanism of the scapegoat tree, here
diff needs to check whether node i has a frequency
    node[i].diff = node[l].diff + node[r].diff + (node[i].count > 0 ? 1 :
0);
}

// in-order traversal
// left -> middle -> right
void inorder(int i) {
    if (i != 0) {
        inorder(node[i].left);
        if (node[i].count > 0) {
            collect[++collect[0]] = i;
        }
        inorder(node[i].right);
    }
}

int build(int l, int r) {
    if (l > r) {
        return 0;
    }
    int mid = (l + r) >> 1;
    int h = collect[mid];
    node[h].left = build(l, mid - 1);

```

```

        node[h].right = build(mid + 1, r);
        up(h);
        return h;
    }

// binary rebuild
void rebuild() {
    if (top != 0) {
        collect[0] = 0;
        inorder(top);
        if (collect[0] > 0) {
            if (father == 0) {
                head = build(1, collect[0]);
            }
            else if (side == 1) {
                node[father].left = build(1, collect[0]);
            }
            else {
                node[father].right = build(1, collect[0]);
            }
        }
    }
}

bool isBalance(int i) {
    return Alpha * node[i].diff >= max(node[node[i].left].diff,
node[node[i].right].diff);
}

// current node, parent, parent-child relationship, key value
void add(int i, int fa, int s, ll val) {
    if (i == 0) {
        if (fa == 0) {
            head = create(val);
        }
        else if (s == 1) {
            node[fa].left = create(val);
        }
        else {
            node[fa].right = create(val);
        }
    }
    else {
        if (node[i].key == val) {
            node[i].count++;
        }
        else if (node[i].key > val) {

```

```

        add(node[i].left, i, 1, val);
    }
    else {
        add(node[i].right, i, 2, val);
    }
}
up(i);
if (!isBalance(i)) {
    top = i;
    father = fa;
    side = s;
}
}

void add(ll val) {
    top = father = side = 0;
    add(head, 0, 0, val);
    rebuild();
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

void remove(int i, int fa, int s, ll val) {
    if (node[i].key == val) {
        node[i].count--;
    }
    else if (node[i].key > val) {
        remove(node[i].left, i, 1, val);
    }
    else {
        remove(node[i].right, i, 2, val);
    }
}

```

```

    up(i);
    if (!isBalance(i)) {
        top = i;
        father = fa;
        side = s;
    }
}

void remove(ll val) {
    if (::rank(val) != ::rank(val + 1)) {
        top = father = side = 0;
        remove(head, 0, 0, val);
        rebuild();
    }
}

ll index(int i, ll x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + node[i].count < x) {
        return index(node[i].right, x - node[node[i].left].size -
node[i].count);
    }
    return node[i].key;
}

ll index(ll x) {
    return index(head, x);
}

ll pre(ll val) {
    int kth = ::rank(val);
    if (kth == 1) {
        return -INF;
    }
    return index(kth - 1);
}

ll suf(ll val) {
    int kth = ::rank(val + 1);
    if (kth == node[head].size + 1) {
        return INF;
    }
    return index(kth);
}

```

```

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0, 0 };
    }
    cnt = 0;
    head = 0;
}

```

Scapegoat Tree (Non-Frequency-Compressed)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
using ld = long double;
const int MAXN = 2e6;
const ld Alpha = 0.7;
const ll INF = 1e18;

int top, father, side;
int collects[MAXN + 1];
int ci = 0;

int head = 0, cnt = 0;
struct tree {
    ll key;
    int aliveSiz;
    int left;
    int right;
    bool alive;
} node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
    }
    head = cnt = 0;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].aliveSiz = 1;
    node[cnt].alive = true;
    node[cnt].left = node[cnt].right = 0;
    return cnt;
}

```

```

void up(int i) {
    node[i].aliveSiz = node[node[i].left].aliveSiz +
node[node[i].right].aliveSiz + (node[i].alive ? 1 : 0);
}

int build(int l, int r) {
    if (l > r) {
        return 0;
    }
    int mid = (l + r) >> 1;
    int p = collects[mid];
    node[p].left = build(l, mid - 1);
    node[p].right = build(mid + 1, r);
    up(p);
    return p;
}

void inorder(int i) {
    if (i != 0 && node[i].aliveSiz > 0) {
        inorder(node[i].left);
        if (node[i].alive) {
            collects[++ci] = i;
        }
        inorder(node[i].right);
    }
}

void rebuild() {
    if (top != 0) {
        ci = 0;
        inorder(top);
        int u = build(1, ci);
        if (side == 1) {
            node[father].left = u;
        }
        else if (side == 2) {
            node[father].right = u;
        }
        else {
            head = u;
        }
        top = father = side = 0;
    }
}

bool isBalance(int i) {

```

```

        return node[i].aliveSiz * Alpha >= max(node[node[i].left].aliveSiz,
node[node[i].right].aliveSiz);
    }

int add(int i, int f, int s, ll val) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return create(val);
    }
    if (node[i].key >= val) {
        node[i].left = add(node[i].left, i, 1, val);
    }
    else {
        node[i].right = add(node[i].right, i, 2, val);
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = f;
        side = s;
    }
    return i;
}

void add(ll val) {
    top = father = side = 0;
    head = add(head, 0, 0, val);
    rebuild();
}

int samll(int i, ll val) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return samll(node[i].left, val);
    }
    else {
        return node[node[i].left].aliveSiz + (node[i].alive ? 1 : 0) +
samll(node[i].right, val);
    }
}

int rank(ll val) {
    return samll(head, val) + 1;
}

```

```

ll index(int i, int rank) {
    int cur = node[i].alive ? 1 : 0;
    if (node[node[i].left].aliveSiz >= rank) {
        return index(node[i].left, rank);
    }
    else if (node[node[i].left].aliveSiz + cur < rank) {
        return index(node[i].right, rank - node[node[i].left].aliveSiz -
cur);
    }
    return node[i].key;
}

ll index(int rank) {
    return index(head, rank);
}

int remove(int i, int f, int s, int rank) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return 0;
    }
    int lsiz = node[node[i].left].aliveSiz;
    int cur = node[i].alive ? 1 : 0;
    if (rank <= lsiz) {
        node[i].left = remove(node[i].left, i, 1, rank);
    }
    else if (node[i].alive && rank == lsiz + cur) {
        node[i].alive = false;
    }
    else {
        node[i].right = remove(node[i].right, i, 2, rank - lsiz - cur);
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = f;
        side = s;
    }
    return i;
}

void remove(ll val) {
    int k1 = ::rank(val);
    int k2 = ::rank(val + 1);
    if (k1 != k2) {
        top = father = side = 0;
        head = remove(head, 0, 0, k1);
        rebuild();
    }
}

```

```

    }
}

ll post(ll val) {
    int kth = ::rank(val + 1);
    if (kth == node[head].aliveSiz + 1) {
        return INF;
    }
    else {
        return index(kth);
    }
}

ll pre(ll val) {
    int kth = ::rank(val);
    if (kth == 1) {
        return -INF;
    }
    else {
        return index(kth - 1);
    }
}
}

```

Cartesian Tree

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

// Cartesian tree:
// the core of the Cartesian tree is:
// 1. according to the feature value a[i] of each element i, naturally
maintain the monotonic order of that feature value
// 2. the result of in-order traversal is the insertion order of element i

// features:
// each node stores value
// the node number is the key
// node numbers are organized according to the binary search tree
left<root<right
// value satisfies that the head node of each subtree is the
maximum/minimum
// the whole tree maintains a generalized (max/min) heap structure

// the tree building process relies on a monotonic stack:

```

```

// traverse in ascending order of key values, if pushed onto the stack, it
// becomes the right child of the pushed element (i.e. the old stack top)
// if nodes are popped, record the number of the last popped node as the
// left child of the newly pushed node

const int MAXN = 1e7;
ll a[MAXN + 1];
int head = 0;
struct {
    ll value;
    int left, right;
} node[MAXN + 1];

void link(int i, int fa, int side) {
    // connect i to the side side of fa
    if (side == 1) {
        node[fa].left = i;
    }
    else {
        node[fa].right = i;
    }
}

void clear(int n) {
    head = 0;
    for (int i = 1; i <= n; i++) {
        node[i] = { 0, 0, 0 };
    }
}

void build(int n) {
    clear(n);
    stack<pair<int, ll>> sta;
    for (int i = 1; i <= n; i++) {
        node[i].value = a[i];
        if (!sta.size()) {
            head = i;
            sta.push({ i, a[i] });
        }
        else {
            if (sta.top().second < a[i]) {
                link(i, sta.top().first, 2);
            }
            else {
                int last = 0; // the last popped node
                while (sta.size() && sta.top().second > a[i]) {
                    last = sta.top().first;
                }
            }
        }
    }
}

```

```

        sta.pop();
    }
    if (sta.size()) {
        link(i, sta.top().first, 2);
    }
    else {
        head = i;
    }
    link(last, i, 1);
}
sta.push({ i,a[i] });
}
}
}

```

Skip List

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const ll INF = 1e18;

// Skip list:
// the core is to maintain the number of interval elements covered by the
// level pointers of each node
// use the random function random
// simulate binary search, the expected structure is similar to a balanced
// tree
// achieve logn time complexity

// https://www.luogu.com.cn/problem/P3369

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, 100);
int cnt = 0;
// the maximum level limit log2(n) is enough
int UpLimit = 20;
struct {
    ll key;// key value
    int count;// frequency
    int level;// number of levels
    int next[21];// pointer (points to the next)
    int len[21];// the number of elements spanned by each pointer
}node[MAXN + 1];

```

```

void build() {
    cnt = 1;
    node[cnt].key = -INF;
    node[cnt].level = UpLimit;
}

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i].key = node[i].count = node[i].level = 0;
        fill(node[i].next, node[i].next + UpLimit + 1, 0);
        fill(node[i].len, node[i].len + UpLimit + 1, 0);
    }
    cnt = 0;
}

// random number of levels
int randomLevel() {
    int ans = 1;
    while (ans <= UpLimit && dist(gen) < 50) {
        ans++;
    }
    return min(ans, UpLimit);
}

// starting from level h of node i, find the space number where val is
// located
int find(int i, int h, ll val) {
    // horizontal jump -> vertical jump -> return upon reaching level 1
    int* next = node[i].next; // pointer array
    while (next[h] != 0 && node[next[h]].key < val) { // this pointer
        // points to a valid space, and the key value of that space < val
        i = next[h];
        next = node[i].next;
    }
    if (h == 1) {
        if (next[h] != 0 && node[next[h]].key == val) {
            return next[h];
        }
        else { // val does not exist
            return 0;
        }
    }
    // go to the next level
    return find(i, h - 1, val);
}

```

```

void addCount(int i, int h, ll val) {
    // increase the frequency
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        // the current position is the position < val, the next is val
        node[node[i].next[h]].count++;
    }
    else {
        addCount(i, h - 1, val);
    }
    node[i].len[h]++; // the number of elements spanned by the pointer + 1
}

// actually add a node
// return value: the number of elements spanned by the bottom pointer
int addNode(int i, int h, int idx) {
    int rightCnt = 0;
    // the cumulative number of elements on the path
    while (node[i].next[h] != 0 && node[node[i].next[h]].key <
node[idx].key) {
        rightCnt += node[i].len[h]; // pointer count
        i = node[i].next[h];
    }
    if (h == 1) {
        node[idx].next[h] = node[i].next[h];
        node[i].next[h] = idx;
        node[idx].len[h] = node[i].len[h];
        node[i].len[h] = node[idx].count;
        return rightCnt;
    }
    else {
        // how many values < node[idx].val are between (i ~ idx)
        int downCnt = addNode(i, h - 1, idx);
        if (h > node[idx].level) {
            // this level cannot set a node
            node[i].len[h]++;
        }
        else {
            node[idx].next[h] = node[i].next[h];
            node[i].next[h] = idx;
            node[idx].len[h] = node[i].len[h] + 1 - downCnt -
node[idx].count;
            node[i].len[h] = downCnt + node[idx].count;
        }
        return rightCnt + downCnt;
    }
}

```

```

    }
}

void add(ll val) {
    if (find(1, UpLimit, val) != 0) {
        addCount(1, UpLimit, val);
    }
    else {
        node[++cnt].key = val;
        node[cnt].count = 1;
        node[cnt].level = randomLevel();
        addNode(1, UpLimit, cnt);
    }
}

void removeCount(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        node[node[i].next[h]].count--;
    }
    else {
        removeCount(i, h - 1, val);
    }
    node[i].len[h]--;
}

void removeNode(int i, int h, int idx) {
    if (h < 1) {
        return;
    }
    while (node[i].next[h] != 0 && node[node[i].next[h]].key <
node[idx].key) {
        i = node[i].next[h];
    }
    if (h > node[idx].level) {
        node[i].len[h]--;
    }
    else {
        node[i].next[h] = node[idx].next[h];
        node[i].len[h] += node[idx].len[h] - 1;
    }
    removeNode(i, h - 1, idx);
}

void remove(ll val) {

```

```

int idx = find(1, UpLimit, val);
if (idx != 0) {
    if (node[idx].count > 1) {
        removeCount(1, UpLimit, val);
    }
    else {
        removeNode(1, UpLimit, idx);
    }
}
}

int small(int i, int h, ll val) {
    int rightCnt = 0;
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        rightCnt += node[i].len[h];
        i = node[i].next[h];
    }
    if (h == 1) {
        return rightCnt;
    }
    else {
        return rightCnt + small(i, h - 1, val);
    }
}

int rank(ll val) { // query the rank
    return small(1, UpLimit, val) + 1;
}

ll index(int i, int h, int x) {
    int c = 0;
    while (node[i].next[h] != 0 && c + node[i].len[h] < x) {
        c += node[i].len[h];
        i = node[i].next[h];
    }
    if (h == 1) {
        return node[node[i].next[h]].key;
    }
    else {
        return index(i, h - 1, x - c);
    }
}

ll index(int x) {
    return index(1, UpLimit, x);
}

```

```

11 pre(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        return i == 1 ? -INF : node[i].key;
    }
    else {
        return pre(i, h - 1, val);
    }
}

11 pre(ll val) {
    return pre(1, UpLimit, val);
}

11 suf(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        if (node[i].next[h] == 0) {
            return INF;
        }
        if (node[node[i].next[h]].key > val) {
            return node[node[i].next[h]].key;
        }
        i = node[i].next[h];
        if (node[i].next[h] == 0) {
            return INF;
        }
        else {
            return node[node[i].next[h]].key;
        }
    }
    else {
        return suf(i, h - 1, val);
    }
}

11 suf(ll val) {
    return suf(1, UpLimit, val);
}

```

6. Fenwick Tree

Point Update, Range Query (2D)

```
#include<bits/stdc++.h>
using namespace std;
//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...

// 2D dependent matrix summation
class SingleAndRange {
    //point update + range query
    static const int MAXN = 2000;
    static int tree[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }
    //range add (1,1)->(x,y)
    void add(int x, int y, int v, int n, int m) {
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree[i][j] += v;
            }
        }
    }
    //range sum (1,1)->(x,y)
    int sum(int x, int y) {
        int ans = 0;
        for (int i = x; i > 0; i -= lowbit(i)) {
            for (int j = y; j > 0; j -= lowbit(j)) {
                ans += tree[i][j];
            }
        }
        return ans;
    }
    //range sum (l,r)->(x,y)
    int queryRange(int l, int r, int x, int y) {
        return sum(x, y) - sum(x, r - 1) - sum(l - 1, y) + sum(l - 1, r -
1);
    }
};
```

Point Update, Range Query

```
#include<bits/stdc++.h>
using namespace std;
```

```

//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...

class Single_AND_Range {
private:
    static const int MAXN = 2e5;
    static int tree[MAXN + 1];
    //get the lowest bit 1
    int lowbit(int x) {
        return x & -x;
    }
    //point update
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n is the right boundary
            tree[i] += val;
            i += lowbit(i);
        }
    }

    //ordinary sum [1...i]
    int sum(int i) {
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }
    //range query
    int queryRange(int l, int r) {
        return sum(r) - sum(l - 1);
    }
};

```

Fenwick Tree

```

#include<bits/stdc++.h>
using namespace std;
//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...
//1. 1D array: point/range update, point/range query
//2. 2D array: point/range update, range query
//1D Fenwick tree
//1D array: point update + range query

```

```

class Single_AND_Range {
private:
    static const int MAXN = 2e5;
    static int tree[MAXN + 1];
    //get the lowest bit 1
    int lowbit(int x) {
        return x & -x;
    }
    //point update
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n is the right boundary
            tree[i] += val;
            i += lowbit(i);
        }
    }

    //ordinary sum [1...i]
    int sum(int i) {
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }
    //range query
    int queryRange(int l, int r) {
        return sum(r) - sum(l - 1);
    }
};

//1D Fenwick tree: range update + point query
class Range_AND_Single {
    static const int MAXN = 2e5;
    static int treeDiff[MAXN + 1]; //range update relies on a 1D difference
    array

    int lowbit(int x) {
        return x & -x;
    }
    //point update, based on the difference array
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n is the right boundary
            treeDiff[i] += val;
            i += lowbit(i);
        }
    }
}

```

```

//range add
void addRange(int n, int l, int r, int val) {
    addSingle(n, l, val);
    addSingle(n, r + 1, -val);
}

//point query
int querySingle(int i) {
    int ans = 0;
    while (i > 0) {
        ans += treeDiff[i];
        i -= lowbit(i);
    }
    return ans;
}

};

//1D Fenwick tree: range update + range query
class Range_AND_Range {
    static const int MAXN = 2e5;
    static int tree1[MAXN + 1]; //1D difference array
    static int tree2[MAXN + 1]; //tree2[i]=(i-1)*d[i], auxiliary array
    int lowbit(int x) {
        return x & -x;
    }

    //point update
    void add(int n, int* tree, int i, int val) {
        while (i <= n) {
            tree[i] += val;
            i += lowbit(i);
        }
    }

    //tree range sum [1...i]
    int sum(int* tree, int i) {
        //1...i
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }

    //tree range modification
    void addRange(int n, int l, int r, int v) {
        add(n, tree1, l, v);
        add(n, tree1, r + 1, -v);
        add(n, tree2, l, (l - 1) * v);
        add(n, tree2, r + 1, -r * v);
    }
}

```

```

    }
    //find the range sum of the original array
    int queryRange(int l, int r) {
        return sum(tree1, r) * r - sum(tree2, r) - sum(tree1, l - 1) * (l
- 1) + sum(tree2, l - 1);
    }
};

class TwoSingleAndRange {
    //point update + range query
    static const int MAXN = 2000;
    static int tree[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }
    //range add (1,1)->(x,y)
    void add(int x, int y, int v, int n, int m) {
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree[i][j] += v;
            }
        }
    }
    //range sum (1,1)->(x,y)
    int sum(int x, int y) {
        int ans = 0;
        for (int i = x; i > 0; i -= lowbit(i)) {
            for (int j = y; j > 0; j -= lowbit(j)) {
                ans += tree[i][j];
            }
        }
        return ans;
    }
    //range sum (1,r)->(x,y)
    int queryRange(int l, int r, int x, int y) {
        return sum(x, y) - sum(x, r - 1) - sum(l - 1, y) + sum(l - 1, r -
1);
    }
};

class TwoRangeAndRange {
    //range update + range query
    static const int MAXN = 2000;
    //2D difference array, tree maintains four pieces of information

```

```

//diff[i][j]
static int tree1[MAXN + 1][MAXN + 1];
//diff[i][j]*i
static int tree2[MAXN + 1][MAXN + 1];
//diff[i][j]*j
static int tree3[MAXN + 1][MAXN + 1];
//diff[i][j]*i*j
static int tree4[MAXN + 1][MAXN + 1];

int lowbit(int x) {
    return x & -x;
}

//point update at position (x,y)
void addSingle(int x, int y, int v, int n, int m) {
    int v1 = v;
    int v2 = v * x;
    int v3 = v * y;
    int v4 = x * y * v;
    for (int i = x; i <= n; i += lowbit(i)) {
        for (int j = y; j <= m; j += lowbit(j)) {
            tree1[i][j] += v1;
            tree2[i][j] += v2;
            tree3[i][j] += v3;
            tree4[i][j] += v4;
        }
    }
}

//sum (1,1)->(x,y)
int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            //four pieces of information
            ans += (x + 1) * (y + 1) * tree1[i][j]
                - (y + 1) * tree2[i][j]
                - (x + 1) * tree3[i][j]
                + tree4[i][j];
        }
    }
    return ans;
}

//range add
void addRange(int a, int b, int c, int d, int v, int n, int m) {
    addSingle(a, b, v, n, m);
    addSingle(a, d + 1, -v, n, m);
    addSingle(c + 1, b, -v, n, m);

```

```

        addSingle(c + 1, d + 1, v, n, m);
    }
    //range query
    int queryRange(int a, int b, int c, int d, int n, int m) {
        int ans = 0;
        ans = sum(c, d) - sum(a - 1, d) - sum(c, b - 1) + sum(a - 1, b -
1);
        return ans;
    }
};

```

Range Update, Point Query

```

#include<bits/stdc++.h>
using namespace std;
//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...

//1D Fenwick tree: range update + point query
//range update relies on maintaining a 1D difference array

class Range_AND_Single {
    static const int MAXN = 2e5;
    static int treeDiff[MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }
    //point update, based on the difference array
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n is the right boundary
            treeDiff[i] += val;
            i += lowbit(i);
        }
    }
    //range add
    void addRange(int n, int l, int r, int val) {
        addSingle(n, l, val);
        addSingle(n, r + 1, -val);
    }
    //point query
    int querySingle(int i) {
        int ans = 0;
        while (i > 0) {
            ans += treeDiff[i];

```

```

        i -= lowbit(i);
    }
    return ans;
}
};

```

Range Update, Range Query (2D)

```

#include<bits/stdc++.h>
using namespace std;
//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...

// 2D query relies on the difference array to maintain information
class RangeAndRange {
    //range update + range query
    static const int MAXN = 2000;
    //2D difference array, tree maintains four pieces of information
    //diff[i][j]
    static int tree1[MAXN + 1][MAXN + 1];
    //diff[i][j]*i
    static int tree2[MAXN + 1][MAXN + 1];
    //diff[i][j]*j
    static int tree3[MAXN + 1][MAXN + 1];
    //diff[i][j]*i*j
    static int tree4[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }

    //point update at position (x,y)
    void addSingle(int x, int y, int v, int n, int m) {
        int v1 = v;
        int v2 = v * x;
        int v3 = v * y;
        int v4 = x * y * v;
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree1[i][j] += v1;
                tree2[i][j] += v2;
                tree3[i][j] += v3;
                tree4[i][j] += v4;
            }
        }
    }
}

```

```

//sum (1,1)->(x,y)
int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            //four pieces of information
            ans += (x + 1) * (y + 1) * tree1[i][j]
                - (y + 1) * tree2[i][j]
                - (x + 1) * tree3[i][j]
                + tree4[i][j];
        }
    }
    return ans;
}

//range add
void addRange(int a, int b, int c, int d, int v, int n, int m) {
    addSingle(a, b, v, n, m);
    addSingle(a, d + 1, -v, n, m);
    addSingle(c + 1, b, -v, n, m);
    addSingle(c + 1, d + 1, v, n, m);
}

//range query
int queryRange(int a, int b, int c, int d, int n, int m) {
    int ans = 0;
    ans = sum(c, d) - sum(a - 1, d) - sum(c, b - 1) + sum(a - 1, b -
1);
    return ans;
}

};

```

Range Update, Range Query

```

#include<bits/stdc++.h>
using namespace std;
//Fenwick tree, the index of the Fenwick tree must be based on 1
//function: mainly used to maintain differentiable information, such as
cumulative sum, product...

//1D Fenwick tree: range update + range query
//relies on two 1D arrays for maintenance
class Range_AND_Range {
    static const int MAXN = 2e5;
    static int tree1[MAXN + 1]; //1D difference array
    static int tree2[MAXN + 1]; //tree2[i]=(i-1)*d[i], auxiliary array
    int lowbit(int x) {

```

```

        return x & -x;
    }
    //point update
    void add(int n, int* tree, int i, int val) {
        while (i <= n) {
            tree[i] += val;
            i += lowbit(i);
        }
    }
    //tree range sum [1...i]
    int sum(int* tree, int i) {
        //1...i
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }
    //tree range modification
    void addRange(int n, int l, int r, int v) {
        add(n, tree1, l, v);
        add(n, tree1, r + 1, -v);
        add(n, tree2, l, (l - 1) * v);
        add(n, tree2, r + 1, -r * v);
    }
    //find the range sum of the original array
    int queryRange(int l, int r) {
        return sum(tree1, r) * r - sum(tree2, r) - sum(tree1, l - 1) * (l
- 1) + sum(tree2, l - 1);
    }
};

```

7. Segment Tree

Persistent Segment Tree (Range MEX)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e5;

// lateLeft represents
// within the managed interval, the leftmost position among the latest
occurrence positions of all appearing elements
// core judgment: if the lateLeft of a node's left child >= L, it means
the mex value of the query range is on the right side

```

```

// the segment tree nodes maintain the value range interval under
management, the situation of the positions where values appear

int n, m;
int a[MAXN + 5];
int version[MAXN + 5];

int cnt = 0;
struct tree {
    int left;
    int right;
    int lateLeft;
}node[(MAXN << 2 | 1) + MAXN * 60];

int clone(int i) {
    cnt++;
    node[cnt] = node[i];
    return cnt;
}

void up(int i) {
    node[i].lateLeft = min(node[node[i].left].lateLeft,
node[node[i].right].lateLeft);
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].left = node[rt].right = node[rt].lateLeft = 0;
    if (l != r) {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

int update(int jobi, int jobv, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].lateLeft = jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = update(jobi, jobv, l, mid, node[rt].left);
        }
    }
}

```

```

        else {
            node[rt].right = update(jobi, jobv, mid + 1, r,
node[rt].right);
        }
        up(rt);
    }
    return rt;
}

int query(int L, int l, int r, int i) {
    if (l == r) {
        return l;
    }
    int mid = (l + r) >> 1;
    if (node[node[i].left].lateLeft >= L) {
        return query(L, mid + 1, r, node[i].right);
    }
    else {
        return query(L, l, mid, node[i].left);
    }
}

void prepare() {
    cnt = 0;
    version[0] = build(1, n);
    for (int v, i = 1; i <= n; i++) {
        // [0, n - 1] a[i] > n
        // mapping [1, n] a[i] + 1 > n + 1
        v = a[i] + 1;
        if (v > n + 1) {
            version[i] = version[i - 1];
        }
        else {
            version[i] = update(v, i, 1, n + 1, version[i - 1]);
        }
    }
}

void solve() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    prepare();
    while (m--) {
        int l, r;
        cin >> l >> r;
    }
}

```

```

        cout << query(1, 1, n + 1, version[r]) - 1 << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    solve();
    return 0;
}

```

Persistent Segment Tree (K-th Smallest)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;

int a[MAXN + 1];
int version[MAXN + 1];
int d[MAXN + 1];
int m = 0;
int cnt = 0;
struct tree {
    int left;
    int right;
    int size;
} node[(MAXN << 2 | 1) + MAXN * 20];

int clone(int i) {
    ++cnt;
    node[cnt] = node[i];
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size;
}

int id(int val) {
    int l = 1, r = m;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (d[mid] == val) {

```

```

        return mid;
    }
    else if (d[mid] > val) {
        r = mid - 1;
    }
    else {
        l = mid + 1;
    }
}
return l;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].size = 0;
    if (l == r) {
        return rt;
    }
    int mid = (l + r) >> 1;
    node[rt].left = build(l, mid);
    node[rt].right = build(mid + 1, r);
    up(rt);
    return rt;
}

int insert(int jobi, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].size++;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = insert(jobi, l, mid, node[rt].left);
        }
        else {
            node[rt].right = insert(jobi, mid + 1, r, node[rt].right);
        }
        up(rt);
    }
    return rt;
}

int query(int jobk, int l, int r, int h1, int h2) {
    if (l == r) {
        return d[l];
    }

```

```

    int mid = (l + r) >> 1;
    int lsiz = node[node[h2].left].size - node[node[h1].left].size;
    if (lsiz >= jobk) {
        return query(jobk, l, mid, node[h1].left, node[h2].left);
    }
    else {
        return query(jobk - lsiz, mid + 1, r, node[h1].right,
node[h2].right);
    }
}

void prepare() {
    int c = m;
    m = 0;
    sort(d + 1, d + c + 1);
    d[0] = -1;
    for (int i = 1; i <= c; i++) {
        if (d[i] != d[m]) {
            d[++m] = d[i];
        }
    }
}

// https://www.luogu.com.cn/problem/P3834

void solve() {
    int n, q;
    cin >> n >> q;
    cnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        d[i] = a[i];
    }
    m = n;
    prepare();
    version[0] = build(1, m);
    for (int i = 1; i <= n; i++) {
        // based on the previous version, at the position with rank
id(a[i]), frequency + 1
        version[i] = insert(id(a[i]), 1, m, version[i - 1]);
    }
    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << query(k, 1, m, version[l - 1], version[r]) << '\n';
    }
}

```

Persistent Segment Tree (Lazy Update)

```
#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;

ll a[MAXN + 1];
int version[MAXN + 1];

int cnt = 0;
struct tree {
    int left;
    int right;
    ll sum;
    ll lazyAdd;
}node[(MAXN << 2 | 1) + MAXN * 70];

void up(int i) {
    node[i].sum = node[node[i].left].sum + node[node[i].right].sum;
}

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].lazyAdd = 0;
    if (l == r) {
        node[rt].sum = a[l];
    }
    else {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

void lazy(int i, int siz, ll jobv) {
    node[i].sum += siz * jobv;
```

```

    node[i].lazyAdd += jobv;
}

void down(int i, int lsiz, int rsiz) {
    if (node[i].lazyAdd != 0) {
        node[i].left = clone(node[i].left);
        node[i].right = clone(node[i].right);
        lazy(node[i].left, lsiz, node[i].lazyAdd);
        lazy(node[i].right, rsiz, node[i].lazyAdd);
        node[i].lazyAdd = 0;
    }
}

ll add(int jobl, int jobr, ll jobv, int l, int r, int i) {
    int rt = clone(i);
    if (jobl <= l && r <= jobr) {
        lazy(rt, r - l + 1, jobv);
    }
    else {
        int mid = (l + r) >> 1;
        down(rt, mid - l + 1, r - mid);
        if (jobl <= mid) {
            node[rt].left = add(jobl, jobr, jobv, l, mid, node[rt].left);
        }
        if (jobr > mid) {
            node[rt].right = add(jobl, jobr, jobv, mid + 1, r,
node[rt].right);
        }
        up(rt);
    }
    return rt;
}

ll query(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = 0;
        if (jobl <= mid) {
            ans += query(jobl, jobr, l, mid, node[i].left);
        }
        if (jobr > mid) {
            ans += query(jobl, jobr, mid + 1, r, node[i].right);
        }
    }
}

```

```

        up(i);
        return ans;
    }
}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    int time = 0;
    version[0] = build(1, n);
    while (m--) {
        char op;
        cin >> op;
        if (op == 'C') {
            int l, r;
            ll v;
            cin >> l >> r >> v;
            version[time + 1] = add(l, r, v, 1, n, version[time]);
            time++;
        }
        else if (op == 'Q') {
            int l, r;
            cin >> l >> r;
            cout << query(l, r, 1, n, version[time]) << '\n';
        }
        else if (op == 'H') {
            int l, r, ver;
            cin >> l >> r >> ver;
            cout << query(l, r, 1, n, version[ver]) << '\n';
        }
        else {
            cin >> time;
        }
    }
}

```

Persistent Segment Tree (Marker Permanence)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e5;

int version[MAXN + 1];

```

```

ll a[MAXN + 1];

// https://acm.hdu.edu.cn/showproblem.php?pid=4348

struct tree {
    int left;
    int right;
    ll sum;
    ll addTag;
}node[(MAXN << 2 | 1) + MAXN * 20];
int cnt = 0;

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].addTag = 0;
    node[rt].sum = 0;
    node[rt].left = node[rt].right = 0;
    if (l == r) {
        node[rt].sum = a[l];
    }
    else {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        node[rt].sum = node[node[rt].left].sum + node[node[rt].right].sum;
    }
    return rt;
}

int add(int jobl, int jobr, ll tag, int l, int r, int i) {
    int rt = clone(i);
    ll a = max(l, jobl), b = min(jobr, r);
    node[rt].sum += tag * (b - a + 1);
    // only when the node is completely covered, mark the tag information
    if (jobl <= l && r <= jobr) {
        node[rt].addTag += tag;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            node[rt].left = add(jobl, jobr, tag, l, mid, node[rt].left);
        }
    }
}

```

```

        if (jobr > mid) {
            node[rt].right = add(jobl, jobr, tag, mid + 1, r,
node[rt].right);
        }
    }
    return rt;
}

ll query(int jobl, int jobr, ll addHistory, int l, int r, int i) {
    ll a = max(l, jobl), b = min(jobr, r);
    if (jobl <= l && r <= jobr) {
        return node[i].sum + addHistory * (b - a + 1LL);
    }
    else {
        int mid = (l + r) >> 1;
        // the ans here is initialized to 0, obtain the correct cumulative
sum value on the child nodes
        ll ans = 0;
        if (jobl <= mid) {
            ans += query(jobl, jobr, addHistory + node[i].addTag, l, mid,
node[i].left);
        }
        if (jobr > mid) {
            ans += query(jobl, jobr, addHistory + node[i].addTag, mid + 1,
r, node[i].right);
        }
        return ans;
    }
}

void solve() {
    int n, m;
    cin >> n >> m;
    int time = 0;
    cnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    version[0] = build(1, n);
    while (m--) {
        char op;
        int l, r, ver;
        ll v;
        cin >> op;
        if (op == 'C') {
            cin >> l >> r >> v;
            version[time + 1] = add(1, r, v, 1, n, version[time]);

```

```

        time++;
    }
    else if (op == 'Q') {
        cin >> l >> r;
        cout << query(l, r, 0, 1, n, version[time]) << '\n';
    }
    else if (op == 'H') {
        cin >> l >> r >> ver;
        cout << query(l, r, 0, 1, n, version[ver]) << '\n';
    }
    else {
        cin >> time;
    }
}
}

```

Persistent Segment Tree (Floating Upper Median)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e4;
const ll INF = 1e18;

// query the upper median of the interval [l, r]
// where l belongs to [a, b], r belongs to [c, a]
// a < b < c < d

// solving strategy: sort the original array a by value, while storing the
indices
// generate a persistent segment tree for each position according to the
sorted array
// for the sorted array a, for position i, the index stored at position j
with j < i is marked as -1 on the segment tree
// binary search for the answer on the sorted array a

int cnt = 0;
int n, m;
int version[MAXN + 1];
int query[4];

struct Arr {
    int val;
    int pos;
}arr[MAXN + 1];

```

```

struct tree {
    int left;
    int right;
    ll pre;
    ll suf;
    ll sum;
}node[(MAXN << 2 | 1) + MAXN * 60];

int clone(int i) {
    int rt = ++cnt;
    node[rt] = node[i];
    return rt;
}

void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].sum = node[l].sum + node[r].sum;
    node[i].pre = max(node[l].pre, node[l].sum + node[r].pre);
    node[i].suf = max(node[r].suf, node[r].sum + node[l].suf);
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].pre = node[rt].suf = node[rt].sum = r - l + 1;
    node[rt].left = node[rt].right = 0;
    if (l != r) {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

// update the value at position jobi to -1
int update(int jobi, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].pre = node[rt].suf = node[rt].sum = -1;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = update(jobi, l, mid, node[rt].left);
        }
        else {
            node[rt].right = update(jobi, mid + 1, r, node[rt].right);
        }
    }
}

```

```

    }
    up(rt);
}
return rt;
}

// query the maximum prefix sum of interval [jobl, jobr] starting at jobl
array<ll, 2> queryMaxPreSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return { node[i].pre, node[i].sum };
    }
    else {
        int mid = (l + r) >> 1;
        if (jobr <= mid) { // completely covered by the left side
            return queryMaxPreSum(jobl, jobr, l, mid, node[i].left);
        }
        else if (jobl > mid) { // completely covered by the right side
            return queryMaxPreSum(jobl, jobr, mid + 1, r, node[i].right);
        }
        else {
            array<ll, 2> left = queryMaxPreSum(jobl, jobr, l, mid,
node[i].left);
            array<ll, 2> right = queryMaxPreSum(jobl, jobr, mid + 1, r,
node[i].right);
            ll pre, sum;
            pre = max(left[0], left[1] + right[0]);
            sum = left[1] + right[1];
            return { pre, sum };
        }
    }
}

// query the maximum suffix sum of interval [jobl, jobr] ending at jobr
array<ll, 2> queryMaxSufSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return { node[i].suf, node[i].sum };
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl > mid) { // completely covered by the right side
            return queryMaxSufSum(jobl, jobr, mid + 1, r, node[i].right);
        }
        else if (jobr <= mid) { // completely covered by the left side
            return queryMaxSufSum(jobl, jobr, l, mid, node[i].left);
        }
        else {
            array<ll, 2> left = queryMaxSufSum(jobl, jobr, l, mid,

```

```

node[i].left);
    array<ll, 2>right = queryMaxSufSum(jobl, jobr, mid + 1, r,
node[i].right);
    ll suf, sum;
    suf = max(right[0], right[1] + left[0]);
    sum = left[1] + right[1];
    return { suf, sum };
}
}
}

// query the range sum
ll queryRangeSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        ll ans = 0;
        if (jobl <= mid) {
            ans += queryRangeSum(jobl, jobr, l, mid, node[i].left);
        }
        if (jobr > mid) {
            ans += queryRangeSum(jobl, jobr, mid + 1, r, node[i].right);
        }
        return ans;
    }
}

void prepare() {
    cnt = 0;
    sort(arr + 1, arr + n + 1, [](Arr& a, Arr& b) {return a.val <
b.val;});
    version[1] = build(1, n);
    for (int i = 2; i <= n; i++) {
        version[i] = update(arr[i - 1].pos, 1, n, version[i - 1]);
    }
}

bool check(int a, int b, int c, int d, int pos) {
    ll sum = queryMaxSufSum(a, b, 1, n, version[pos])[0];
    sum += queryMaxPreSum(c, d, 1, n, version[pos])[0];
    sum += queryRangeSum(b + 1, c - 1, 1, n, version[pos]);
    return sum >= 0;
}

```

```

ll compute(int a, int b, int c, int d) {
    int l = 1, r = n;
    int ans = 0;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (check(a, b, c, d, mid)) {
            ans = mid;
            l = mid + 1;
        }
        else {
            r = mid - 1;
        }
    }
    return arr[ans].val;
}

```

// <https://www.luogu.com.cn/problem/P2839>

```

void solve() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i].val;
        arr[i].pos = i;
    }
    prepare();
    ll lastAns = 0;
    cin >> m;
    while (m--) {
        cin >> query[0] >> query[1] >> query[2] >> query[3];
        query[0] = (query[0] + lastAns) % n;
        query[1] = (query[1] + lastAns) % n;
        query[2] = (query[2] + lastAns) % n;
        query[3] = (query[3] + lastAns) % n;
        sort(query, query + 4);
        // convert the index to 1-based
        lastAns = compute(query[0] + 1, query[1] + 1, query[2] + 1,
query[3] + 1);
        cout << lastAns << '\n';
    }
}

```

Segment Tree Range Min/Max

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

const ll INF = 1e18;
const int MAXN = 1e6;
ll a[MAXN + 1];
ll sum[4 * MAXN + 1];
ll maxval[4 * MAXN + 1];
ll cnt[4 * MAXN + 1];
ll cmaxval[4 * MAXN + 1];

void up(int i) {
    int l = 2 * i, r = l + 1;
    sum[i] = sum[l] + sum[r];
    maxval[i] = max(maxval[l], maxval[r]);
    if (maxval[l] > maxval[r]) {
        cmaxval[i] = max(maxval[r], cmaxval[l]);
        cnt[i] = cnt[l];
    }
    else if (maxval[r] > maxval[l]) {
        cmaxval[i] = max(maxval[l], cmaxval[r]);
        cnt[i] = cnt[r];
    }
    else {
        cmaxval[i] = max(cmaxval[l], cmaxval[r]);
        cnt[i] = cnt[l] + cnt[r];
    }
}

void build(int l, int r, int i) {
    if (l == r) {
        maxval[i] = a[l];
        cmaxval[i] = -INF;
        sum[i] = a[l];
        cnt[i] = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, 2 * i);
        build(mid + 1, r, 2 * i + 1);
        up(i);
    }
}

void lazy(int i, ll jobv) {
    if (jobv < maxval[i]) {
        sum[i] -= (maxval[i] - jobv) * cnt[i];
        maxval[i] = jobv;
    }
}

void down(int i) {
    int l = 2 * i, r = l + 1;

```

```

        lazy(l, maxval[i]);
        lazy(r, maxval[i]);
    }
    void setMin(int jobl, int jobr, ll jobv, int l, int r, int i) {
        if (jobv >= maxval[i]) {
            return;
        }
        if (jobl <= l && r <= jobr && cmaxval[i] < jobv) {
            //cmax<jobv<=max, lazy tag
            lazy(i, jobv);
        }
        else {
            //1. the task is not fully covered
            //2. jobv <= cmax, must pass down
            int mid = (l + r) >> 1;
            down(i);
            if (jobl <= mid) {
                setMin(jobl, jobr, jobv, l, mid, 2 * i);
            }
            if (jobr > mid) {
                setMin(jobl, jobr, jobv, mid + 1, r, 2 * i + 1);
            }
            up(i);
        }
    }
}
ll querySum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return sum[i];
    }
    int mid = (l + r) >> 1;
    down(i);
    ll ans = 0;
    if (jobl <= mid) {
        ans += querySum(jobl, jobr, l, mid, 2 * i);
    }
    if (jobr > mid) {
        ans += querySum(jobl, jobr, mid + 1, r, 2 * i + 1);
    }
    up(i);
    return ans;
}

ll queryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return maxval[i];
    }
    int mid = (l + r) >> 1;

```

```

    down(i);
    ll ans = -INF;
    if (jobl <= mid) {
        ans = max(ans, queryMax(jobl, jobr, l, mid, 2 * i));
    }
    if (jobr > mid) {
        ans = max(ans, queryMax(jobl, jobr, mid + 1, r, 2 * i + 1));
    }
    up(i);
    return ans;
}

```

Segment Tree Historical Min/Max

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;
const int MAXN = 5e5;
ll a[MAXN + 1];
struct {
    ll sum, max, cnt, cmax;
    ll maxAdd, otherAdd;
    ll maxHistory, maxAddTop;
    ll otherAddTop;
    // sum          total sum
    // max          maximum value
    // cnt          number of maximum values
    // cmax         second maximum value
    // maxHistory   historical maximum value
    // the following are lazy information tag contents:
    // maxAdd       maximum value increment (current)
    // otherAdd     other value increment (current)
    // maxAddTop    the historical maximum increment of the maximum value
    // otherAddTop  the historical maximum increment of other values
} node[4 * MAXN + 1];

void up(int i) {
    int l = i << 1;
    int r = i << 1 | 1;
    node[i].sum = node[l].sum + node[r].sum;
    node[i].max = max(node[l].max, node[r].max);
    node[i].maxHistory = max(node[l].maxHistory, node[r].maxHistory);
    if (node[l].max > node[r].max) {
        node[i].cnt = node[l].cnt;
        node[i].cmax = max(node[l].cmax, node[r].max);
    }
}

```

```

    else if (node[l].max < node[r].max) {
        node[i].cnt = node[r].cnt;
        node[i].cmax = max(node[r].cmax, node[l].max);
    }
    else {
        node[i].cnt = node[l].cnt + node[r].cnt;
        node[i].cmax = max(node[l].cmax, node[r].cmax);
    }
}

//parameters: node, size, maximum value increment, other value increment,
maximum value historical maximum increment, other value historical maximum
increment
void lazy(int i, int n, ll maxAddv, ll otherAddv, ll maxUpv, ll otherUpv)
{
    node[i].maxHistory = max(node[i].maxHistory, node[i].max + maxUpv);
    node[i].maxAddTop = max(node[i].maxAddTop, node[i].maxAdd + maxUpv);
    node[i].otherAddTop = max(node[i].otherAddTop, node[i].otherAdd +
otherUpv);
    node[i].sum += maxAddv * node[i].cnt + otherAddv * (n - node[i].cnt);
    node[i].max += maxAddv;
    node[i].cmax += (node[i].cmax == -INF) ? 0 : otherAddv;
    node[i].maxAdd += maxAddv;
    node[i].otherAdd += otherAddv;
}

void down(int i, int ln, int rn) {
    int l = i << 1;
    int r = i << 1 | 1;
    ll tmp = max(node[l].max, node[r].max); //the parent's maximum comes
from its children, the parent may have changed
    if (node[l].max == tmp) {
        lazy(l, ln, node[i].maxAdd, node[i].otherAdd, node[i].maxAddTop,
node[i].otherAddTop);
    }
    else {
        lazy(l, ln, node[i].otherAdd, node[i].otherAdd,
node[i].otherAddTop, node[i].otherAddTop);
    }
    if (node[r].max == tmp) {
        lazy(r, rn, node[i].maxAdd, node[i].otherAdd, node[i].maxAddTop,
node[i].otherAddTop);
    }
    else {
        lazy(r, rn, node[i].otherAdd, node[i].otherAdd,
node[i].otherAddTop, node[i].otherAddTop);
    }
    node[i].maxAdd = node[i].otherAdd = node[i].maxAddTop =
node[i].otherAddTop = 0;
}

```

```

}
void build(int l, int r, int i) {
    if (l == r) {
        node[i].sum = node[i].max = node[i].maxHistory = a[l];
        node[i].cmax = -INF;
        node[i].cnt = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1);
        build(mid + 1, r, i << 1 | 1);
        up(i);
    }
    //lazy information
    node[i].maxAdd = node[i].otherAdd = node[i].maxAddTop =
node[i].otherAddTop = 0;
}
//range add
void add(int jobl, int jobr, ll jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        lazy(i, r - l + 1, jobv, jobv, jobv, jobv);
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
        up(i);
    }
}
//set the range maximum
void setMin(int jobl, int jobr, ll jobv, int l, int r, int i) {
    if (jobv >= node[i].max) {
        return;
    }
    if (jobl <= l && r <= jobr && node[i].cmax < jobv) {
        lazy(i, r - l + 1, jobv - node[i].max, 0, jobv - node[i].max, 0);
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        if (jobl <= mid) {
            setMin(jobl, jobr, jobv, l, mid, i << 1);

```

```

    }
    if (jobr > mid) {
        setMin(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
    }
    up(i);
}
}
//query the range sum
ll querySum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = 0;
        if (jobl <= mid) {
            ans += querySum(jobl, jobr, l, mid, i << 1);
        }
        if (jobr > mid) {
            ans += querySum(jobl, jobr, mid + 1, r, i << 1 | 1);
        }
        return ans;
    }
}
}
//query the range maximum
ll queryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].max;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = -INF;
        if (jobl <= mid) {
            ans = max(ans, queryMax(jobl, jobr, l, mid, i << 1));
        }
        if (jobr > mid) {
            ans = max(ans, queryMax(jobl, jobr, mid + 1, r, i << 1 | 1));
        }
        return ans;
    }
}
}
//query the range historical maximum
ll queryHistoryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].maxHistory;
    }

```

```

    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - 1 + 1, r - mid);
        ll ans = -INF;
        if (jobl <= mid) {
            ans = max(ans, queryHistoryMax(jobl, jobr, l, mid, i << 1));
        }
        if (jobr > mid) {
            ans = max(ans, queryHistoryMax(jobl, jobr, mid + 1, r, i << 1
| 1));
        }
        return ans;
    }
}

```

Segment Tree and Sweep Line (Perimeter Union)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 5e3;
array<int, 4>infox[2 * MAXN + 5];
array<int, 4>infoy[2 * MAXN + 5];
int fx[2 * MAXN + 5];
int fy[2 * MAXN + 5];
int n, m, cntx, cnty;

struct {
    ll len;
    ll cover;
    int cnt;
}node[8 * MAXN + 5];

void up(int i, int l, int r) {
    if (l == r) {
        node[i].cover = node[i].cnt > 0 ? node[i].len : 0;
    }
    else {
        node[i].cover = node[i].cnt > 0 ? node[i].len : node[i << 1].cover
+ node[i << 1 | 1].cover;
    }
}

void build(int l, int r, int i, int* f) {
    node[i].len = f[r + 1] - f[l];
}

```

```

node[i].cover = 0;
node[i].cnt = 0;
if (l != r) {
    int mid = (l + r) >> 1;
    build(l, mid, i << 1, f);
    build(mid + 1, r, i << 1 | 1, f);
    up(i, l, r);
}
}

void add(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        node[i].cnt += jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
    }
    up(i, l, r);
}

void Disc() {
    sort(fy + 1, fy + m + 1);
    fy[0] = -1;
    int k = m; m = 0;
    for (int i = 1; i <= k; i++) {
        if (fy[i] != fy[m]) {
            fy[++m] = fy[i];
        }
    }
    fy[m + 1] = fy[m];

    sort(fx + 1, fx + n + 1);
    fx[0] = -1;
    k = n; n = 0;
    for (int i = 1; i <= k; i++) {
        if (fx[i] != fx[n]) {
            fx[++n] = fx[i];
        }
    }
    fx[n + 1] = fx[n];
}
}

```

```

int rankk(int* disc, int R, int v) {
    int l = 1, r = R;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (disc[mid] == v) {
            return mid;
        }
        else if (disc[mid] > v) {
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return l;
}

bool cmp(array<int, 4>a, array<int, 4>b) {
    if (a[0] == b[0]) {
        return a[3] > b[3];
    }
    return a[0] < b[0];
}

void solve() {
    int N;
    cin >> N;
    m = n = cntx = cnty = 0;
    for (int i = 1; i <= N; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        x1 += 1e5, x2 += 1e5, y1 += 1e5, y2 += 1e5;
        infoy[++cnty] = { x1, y1, y2, 1 };
        infoy[++cnty] = { x2, y1, y2, -1 };
        fy[++m] = y1;
        fy[++m] = y2;

        infox[++cntx] = { y1, x1, x2, 1 };
        infox[++cntx] = { y2, x1, x2, -1 };
        fx[++n] = x1;
        fx[++n] = x2;
    }
    sort(infox + 1, infox + cntx + 1, cmp);
    sort(infoy + 1, infoy + cnty + 1, cmp);
    Disc();
    ll ans = 0;
}

```

```

//process y
build(1, m, 1, fy);
for (int i = 1, pre = 0; i <= cnty; i++) {
    pre = node[1].cover;
    add(rankk(fy, m, infoy[i][1]), rankk(fy, m, infoy[i][2]) - 1,
infoy[i][3], 1, m, 1);
    ans += llabs(node[1].cover - pre);
}
//process x
build(1, n, 1, fx);
for (int i = 1, pre = 0; i <= cntx; i++) {
    pre = node[1].cover;
    add(rankk(fx, n, infox[i][1]), rankk(fx, n, infox[i][2]) - 1,
infox[i][3], 1, n, 1);
    ans += llabs(node[1].cover - pre);
}
cout << ans << '\n';
}

```

Segment Tree and Sweep Line (Area Union)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 1e5;
array<int, 4>info[2 * MAXN + 5];
int fy[2 * MAXN + 5];
int m, cnt;

struct {
    ll len;
    ll cover;
    int cnt;
}node[8 * MAXN + 5];

void up(int i, int l, int r) {
    if (l == r) {
        node[i].cover = node[i].cnt > 0 ? node[i].len : 0;
    }
    else {
        node[i].cover = node[i].cnt > 0 ? node[i].len : node[i << 1].cover
+ node[i << 1 | 1].cover;
    }
}

void build(int l, int r, int i) {

```

```

node[i].len = fy[r + 1] - fy[l];
node[i].cover = 0;
node[i].cnt = 0;
if (l != r) {
    int mid = (l + r) >> 1;
    build(l, mid, i << 1);
    build(mid + 1, r, i << 1 | 1);
    up(i, l, r);
}
}

void add(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        node[i].cnt += jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
    }
    up(i, l, r);
}

void Disc() {
    sort(fy + 1, fy + m + 1);
    fy[0] = -1;
    int k = m; m = 0;
    for (int i = 1; i <= k; i++) {
        if (fy[i] != fy[m]) {
            fy[++m] = fy[i];
        }
    }
    fy[m + 1] = fy[m];
}

int rankk(int* disc, int n, int v) {
    int l = 1, r = n;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (disc[mid] == v) {
            return mid;
        }
        else if (disc[mid] > v) {

```

```

        r = mid - 1;
    }
    else {
        l = mid + 1;
    }
}
return l;
}

bool cmp(array<int, 4>a, array<int, 4>b) {
    if (a[0] == b[0])return a[3] > b[3]; //handle the same coordinate
    problem
    return a[0] < b[0];
}

void solve() {
    int n;
    cin >> n;
    m = cnt = 0;
    for (int i = 1; i <= n; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        info[++cnt] = { x1, y1, y2, 1 };
        info[++cnt] = { x2, y1, y2, -1 };
        fy[++m] = y1;
        fy[++m] = y2;
    }
    sort(info + 1, info + cnt + 1, cmp);
    Disc();
    ll ans = 0;
    build(1, m, 1);
    for (int i = 1; i <= cnt; i++) {
        if (i > 1) {
            ans += ll(info[i][0] - info[i - 1][0]) * ll(node[1].cover);
        }
        add(rankk(fy, m, info[i][1]), rankk(fy, m, info[i][2]) - 1,
info[i][3], 1, m, 1);
    }
    cout << ans << '\n';
}

```